98 System Design Concepts

Deep Q&A with Java 17 & Spring Boot

HLD & LLD • Code Examples • When To Use • Trade-offs

Category	Key Concepts
Scalability & HA	Scalability · Fault Tolerance · Circuit Breaker · Autoscaling
Databases & Caching	SQL vs NoSQL · Sharding · Indexing · Caching · Vector DB
APIs & Communication	REST · gRPC · GraphQL · WebSockets · Service Discovery
Distributed Patterns	CAP · Consensus · Saga · CQRS · Event Sourcing · Outbox
Reliability & Ops	Retry · Timeout · Bulkhead · Backpressure · Load Shedding
Data Engineering	ETL · Stream Processing · Data Lake · Data Warehouse · HLL
Security	Authentication · Authorization · RBAC · SSO · Encryption
Advanced Algorithms	Merkle Tree · Bloom Filter · LSM Tree · HyperLogLog

All examples use Java 17 (records, text blocks, sealed classes) with Spring Boot 3.x, Resilience4j, Spring Data JPA, Spring Security, Kafka, and production-grade patterns.

98 System Design Concepts — Deep Q&A with Java 17 & Spring Boot

■ 1. SCALABILITY

■ Q: What is Scalability and why does it matter?

A: Scalability is the system's ability to handle increased load (users, data, requests) without degrading performance. It is the foundation of all distributed system design.

Types:

- **Vertical Scaling (Scale Up):** Add more CPU/RAM to existing machine. Simple but has hard limits and single point of failure.
- **Horizontal Scaling (Scale Out):** Add more machines. Unlimited ceiling, more complex to implement.

When to use Vertical: Early-stage startup, stateful apps (databases), quick win before architectural changes.

When to use Horizontal: High traffic web services, stateless services, microservices.

Advantages of Horizontal: No SPOF, cost-effective at scale, elastic.

Disadvantages: Distributed complexity, data consistency challenges, network latency.

HLD: Load Balancer → [App Server 1, App Server 2, App Server N] → Shared DB / Cache

LLD (Spring Boot Stateless Service):

```
@SpringBootApplication
@EnableCaching
public class ScalableApp {
    public static void main(String[] args) {
        SpringApplication.run(ScalableApp.class, args);
    }
}

// Store session in Redis, NOT in-memory — enables horizontal scale
// spring.session.store-type=redis (in application.yml)

@RestController
@RequestMapping("/api/users")
public class UserController {
    // No instance variables holding state — fully stateless
    private final UserService userService;

    public UserController(UserService userService) {
        this.userService = userService;
    }

    @GetMapping("/{id}")
    public ResponseEntity<User> getUser(@PathVariable Long id) {
        return ResponseEntity.ok(userService.findById(id));
    }
}
```

Key Rule: Design services to be stateless first — they can be horizontally scaled trivially.

■ 2. DATABASE

■ Q: How do you choose the right database for a system?

A: Database choice is one of the most critical architectural decisions. Wrong choice = expensive migration later.

Categories:

Type	Examples	Use Case
Relational (RDBMS)	PostgreSQL, MySQL	ACID transactions, complex queries
Document	MongoDB	Flexible schema, JSON data
Key-Value	Redis, DynamoDB	Cache, sessions, leaderboards
Column-Family	Cassandra, HBase	Time-series, write-heavy
Graph	Neo4j	Social networks, recommendation
Search	Elasticsearch	Full-text search, log analysis
Time-Series	InfluxDB	Metrics, IoT, monitoring

When to use PostgreSQL: E-commerce orders, banking, user accounts — anywhere ACID matters.

When to use Cassandra: IoT sensor data, activity feeds — write heavy, eventual consistency OK.

When to use Redis: Sessions, cache, rate limiting, pub/sub.

```java
// Entity for PostgreSQL
@Entity
@Table(name = "orders")
public class Order {
    @Id
    @GeneratedValue(strategy = GenerationType.SEQUENCE)
    private Long id;

    @Column(nullable = false)
    private String customerId;

    @Column(precision = 19, scale = 4)
    private BigDecimal amount;

    @Enumerated(EnumType.STRING)
    private OrderStatus status;

    @CreatedDate
    private Instant createdAt;
}

// Repository with JPQL and native queries
public interface OrderRepository extends JpaRepository<Order, Long> {
    @Query("SELECT o FROM Order o WHERE o.customerId = :cid AND o.status = :status")
    List<Order> findByCustomerAndStatus(
        @Param("cid") String customerId,
        @Param("status") OrderStatus status
    );
```

```
    @Query(value = """
        SELECT DATE_TRUNC('day', created_at) as day, SUM(amount) as total
        FROM orders GROUP BY day ORDER BY day DESC LIMIT 30
        """, nativeQuery = true)
    List<Object[]> getDailyRevenue();
}
```

■ 3. API DESIGN

■ Q: What makes a good REST API design?

A: A good API is consistent, intuitive, versioned, and backward-compatible.

REST Principles:

- Resources are nouns, not verbs (`/users`, not `/getUser`)
- HTTP methods: GET=read, POST=create, PUT=replace, PATCH=partial update, DELETE=remove
- Stateless — every request self-contained
- Use proper HTTP status codes (200, 201, 204, 400, 401, 403, 404, 409, 422, 500)
- Versioning: `/api/v1/users`
- Pagination: `?page=0&size=20&sort=createdAt,desc`
- Consistent error response format

Richardson Maturity Model:

- Level 0: HTTP tunnel (SOAP-style)
- Level 1: Resources (`/users/{id}`)
- Level 2: HTTP verbs correctly used
- Level 3: HATEOAS (hypermedia links in responses)

```
@RestController
@RequestMapping("/api/v1/users")
@RequiredArgsConstructor
@Validated
public class UserController {

    private final UserService userService;

    @GetMapping
    public ResponseEntity<PagedResponse<UserDTO>> getUsers(
            @RequestParam(defaultValue = "0") int page,
            @RequestParam(defaultValue = "20") @Max(100) int size,
            @RequestParam(required = false) String status) {
        Pageable pageable = PageRequest.of(page, size, Sort.by("createdAt").descending());
        Page<UserDTO> users = userService.findAll(status, pageable);
        return ResponseEntity.ok(PagedResponse.of(users));
    }

    @PostMapping
    public ResponseEntity<UserDTO> createUser(
            @Valid @RequestBody CreateUserRequest request,
            @RequestHeader(value = "Idempotency-Key", required = false) String key) {
        UserDTO created = userService.create(request, key);
        URI location = ServletUriComponentsBuilder.fromCurrentRequest()
                .path("/{id}").buildAndExpand(created.id()).toUri();
        return ResponseEntity.created(location).body(created);
    }

    @PatchMapping("/{id}")
    public ResponseEntity<UserDTO> updateUser(
            @PathVariable Long id,
```

```
            @Valid @RequestBody UpdateUserRequest request) {
        return ResponseEntity.ok(userService.update(id, request));
    }

    @DeleteMapping("/{id}")
    public ResponseEntity<Void> deleteUser(@PathVariable Long id) {
        userService.delete(id);
        return ResponseEntity.noContent().build();
    }
}

// Global exception handler — consistent error format
@RestControllerAdvice
public class GlobalExceptionHandler {

    @ExceptionHandler(ResourceNotFoundException.class)
    public ResponseEntity<ErrorResponse> handleNotFound(ResourceNotFoundException ex) {
        return ResponseEntity.status(404)
                .body(new ErrorResponse("NOT_FOUND", ex.getMessage(), Instant.now()));
    }

    @ExceptionHandler(MethodArgumentNotValidException.class)
    public ResponseEntity<ErrorResponse> handleValidation(MethodArgumentNotValidException ex) {
        String errors = ex.getBindingResult().getFieldErrors().stream()
                .map(e -> e.getField() + ": " + e.getDefaultMessage())
                .collect(Collectors.joining(", "));
        return ResponseEntity.badRequest()
                .body(new ErrorResponse("VALIDATION_ERROR", errors, Instant.now()));
    }
}

public record ErrorResponse(String code, String message, Instant timestamp) {}
```

■ 4. FAULT TOLERANCE

■ Q: What patterns implement fault tolerance in distributed systems?

A: Fault Tolerance is the system's ability to continue operating correctly even when components fail.

Core Patterns:

- **Retry:** Re-attempt transient failures with exponential backoff
- **Circuit Breaker:** Stop calling failing service, fail fast
- **Fallback:** Return cached/default response on failure
- **Bulkhead:** Isolate failures to prevent cascade
- **Timeout:** Never wait forever for a response
- **Redundancy:** Multiple instances of every critical component

Circuit Breaker States:

- CLOSED → OPEN (when failure rate > threshold)
- OPEN → HALF_OPEN (after wait duration)
- HALF_OPEN → CLOSED/OPEN (based on test calls)

```
// Resilience4j with Spring Boot
// build.gradle: implementation 'io.github.resilience4j:resilience4j-spring-boot3'

@Service
@RequiredArgsConstructor
public class PaymentService {

    private final PaymentGateway gateway;

    @CircuitBreaker(name = "payment", fallbackMethod = "paymentFallback")
    @Retry(name = "payment")
    @TimeLimiter(name = "payment")
    @Bulkhead(name = "payment", type = Bulkhead.Type.SEMAPHORE)
    public CompletableFuture<PaymentResult> processPayment(PaymentRequest req) {
        return CompletableFuture.supplyAsync(() -> gateway.charge(req));
    }

    public CompletableFuture<PaymentResult> paymentFallback(
            PaymentRequest req, Exception ex) {
        log.warn("Payment fallback triggered: {}", ex.getMessage());
        // Queue for async retry
        eventQueue.publish(new PendingPaymentEvent(req));
        return CompletableFuture.completedFuture(PaymentResult.pending(req.getOrderId()));
    }
}
```

```
# application.yml
resilience4j:  circuitbreaker:    instances:      payment:        slidingWindowType: COUNT_BASED        slidingWindowS
```

■ 5. ERASURE CODING

■ Q: What is Erasure Coding and when to use it over replication?

A: Erasure Coding splits data into `k` data chunks + `m` parity chunks (total k+m), stored across nodes. Any `k` chunks can reconstruct original data. Tolerate up to `m` failures.

RS(6,3) Example: 6 data + 3 parity = 9 chunks. Tolerate 3 failures. Storage overhead = 9/6 = **1.5x**.

3x Replication: Same fault tolerance but **3x** storage overhead.

When to use Erasure Coding: Cold storage/archival (AWS S3 Glacier, HDFS), large objects, storage cost is primary concern.

When to use Replication: Hot data needing fast reads, real-time systems, small objects.

Disadvantage of EC: Higher CPU for encode/decode; higher read latency (must gather k chunks from different nodes).

Real-world: Facebook f4 uses RS(10,4): tolerates 4 failures at only 1.4x overhead, saving petabytes vs 3x replication.

```
// Conceptual RS encoding illustration
public class ErasureCodingSimulator {
    private static final int DATA_SHARDS = 4;
    private static final int PARITY_SHARDS = 2;

    // In production: use ISA-L or Jerasure library via JNI
    public List<byte[]> encode(byte[] data) {
        int shardSize = (int) Math.ceil((double) data.length / DATA_SHARDS);
        List<byte[]> shards = new ArrayList<>();

        // Split data into DATA_SHARDS chunks
        for (int i = 0; i < DATA_SHARDS; i++) {
            byte[] shard = new byte[shardSize];
            int start = i * shardSize;
            int len = Math.min(shardSize, data.length - start);
            if (len > 0) System.arraycopy(data, start, shard, 0, len);
            shards.add(shard);
        }
        // Generate parity shards via XOR (real RS uses Galois Field GF(2^8))
        shards.add(xorShards(shards.subList(0, 2)));
        shards.add(xorShards(shards.subList(2, 4)));
        return shards;
    }

    private byte[] xorShards(List<byte[]> shards) {
        byte[] result = new byte[shards.get(0).length];
        for (byte[] s : shards)
            for (int i = 0; i < result.length; i++) result[i] ^= s[i];
        return result;
    }
}
```

■ 6. SHARDING

■ Q: What are the database sharding strategies and when does each shine?

A: Sharding is horizontal partitioning — splitting data across multiple DB instances (shards) where each shard holds a subset.

Strategies:

Strategy	How	Pros	Cons
Range-Based	Shard by value range (IDs 1-1M)	Easy range queries	Hot partitions
Hash-Based	`shard = hash(key) % N`	Even distribution	No range queries; resharding painful
Directory-Based	Lookup table: key → shard	Flexible	Lookup table is SPOF
Geographic	By region/country	Data locality, compliance	Cross-region queries slow

When to shard: Single DB can't handle write throughput; dataset exceeds server capacity; query latency too high even with indexing.

Problems: Cross-shard JOINs (scatter-gather), rebalancing, distributed transactions, schema migrations on all shards.

Consistent Hashing solves the resharding problem — adding a node moves only K/N keys (not all).

```
// Consistent Hash Ring — virtual nodes for even distribution
public class ConsistentHashRouter<T> {
    private final SortedMap<Integer, T> ring = new TreeMap<>();
    private final int virtualNodes;

    public ConsistentHashRouter(List<T> nodes, int virtualNodes) {
        this.virtualNodes = virtualNodes;
        nodes.forEach(this::addNode);
    }

    public void addNode(T node) {
        for (int i = 0; i < virtualNodes; i++)
            ring.put(hash(node + "#" + i), node);
    }

    public void removeNode(T node) {
        for (int i = 0; i < virtualNodes; i++)
            ring.remove(hash(node + "#" + i));
    }

    public T getNode(String key) {
        if (ring.isEmpty()) throw new IllegalStateException("No nodes");
        int hash = hash(key);
        SortedMap<Integer, T> tail = ring.tailMap(hash);
        return (tail.isEmpty() ? ring : tail).get(
            tail.isEmpty() ? ring.firstKey() : tail.firstKey());
    }

    private int hash(String key) {
```

```
        // Use MurmurHash3 in production for better distribution
        return Math.abs(key.hashCode());
    }
}

// ShardingSphere JDBC — transparent sharding (application.yml)
/*
spring:
  shardingsphere:
    rules:
      sharding:
        tables:
          orders:
            actual-data-nodes: ds${0..1}.orders_${0..3}
            database-strategy:
              standard:
                sharding-column: customer_id
                sharding-algorithm-name: customer-hash
        sharding-algorithms:
          customer-hash:
            type: HASH_MOD
            props:
              sharding-count: 2
*/
```

■ 7. EVENT-DRIVEN ARCHITECTURE

■ Q: When should you choose Event-Driven Architecture?

A: EDA is a paradigm where services communicate by emitting and consuming events asynchronously through an event broker.

Core Components:

- **Producer:** Emits events (order-service → "OrderPlaced")
- **Broker:** Routes and persists events (Kafka, RabbitMQ)
- **Consumer:** Reacts to events (payment-service, inventory-service)

When to use EDA:

- Decoupled microservices (services shouldn't directly call each other)
- Fan-out: one event triggers many consumers
- Workflows spanning multiple services (order → payment → shipping → notification)
- Audit trail / event sourcing
- Real-time data pipelines

Advantages: Loose coupling, high throughput, temporal decoupling, easy to add consumers without modifying producers.

Disadvantages: Eventual consistency, hard to debug (need distributed tracing), duplicate events (idempotency required), ordering challenges.

Outbox Pattern: Atomically save event + business data in same DB transaction, then publish separately — ensures no event is lost.

```
// Producer with Outbox Pattern (atomicity guarantee)
@Service
@RequiredArgsConstructor
public class OrderService {

    private final OrderRepository orderRepo;
    private final OutboxRepository outboxRepo;
    private final KafkaTemplate<String, Object> kafka;

    @Transactional  // Both order and outbox saved atomically
    public Order placeOrder(CreateOrderRequest req) {
        Order order = orderRepo.save(Order.from(req));

        // Save to outbox table — same transaction, guaranteed delivery
        outboxRepo.save(OutboxEvent.builder()
            .aggregateId(order.getId().toString())
            .eventType("OrderPlaced")
            .payload(toJson(order))
            .status(OutboxStatus.PENDING)
            .build());

        return order;
    }

    // Scheduled job or CDC picks up PENDING outbox events and publishes
```

```
    @Scheduled(fixedDelay = 1000)
    @Transactional
    public void publishPendingEvents() {
        List<OutboxEvent> pending = outboxRepo.findByStatus(OutboxStatus.PENDING);
        pending.forEach(event -> {
            kafka.send("orders", event.getAggregateId(), event.getPayload());
            event.setStatus(OutboxStatus.PUBLISHED);
            outboxRepo.save(event);
        });
    }
}

// Consumer with idempotency
@Service
public class PaymentEventConsumer {

    private final Set<String> processedIds = Collections.synchronizedSet(
        new LinkedHashSet<>() {
            protected boolean removeEldestEntry(Map.Entry e) { return size() > 10000; }
        });

    @KafkaListener(topics = "orders", groupId = "payment-service")
    public void handleOrderPlaced(@Payload OrderEvent event, Acknowledgment ack) {
        try {
            if (!processedIds.contains(event.getEventId())) {
                paymentService.initiate(event);
                processedIds.add(event.getEventId());
            }
            ack.acknowledge();
        } catch (Exception e) {
            log.error("Processing failed, will retry: {}", event.getEventId(), e);
            // Don't ack — Kafka redelivers
        }
    }
}
```

■ 8. BULKHEAD PATTERN

■ Q: What is the Bulkhead pattern and why is it critical in microservices?

A: Bulkhead isolates resources so a failure in one area doesn't cascade to others — like watertight compartments preventing a ship from sinking when one compartment floods.

Without Bulkhead: Slow payment service exhausts entire thread pool → entire application becomes unresponsive for all operations.

With Bulkhead: Payment has its own thread pool (10 threads). If payment hangs, inventory (separate 20 threads) continues serving normally.

Types:

1. **Thread Pool Bulkhead:** Dedicated thread pool per dependency (best isolation)
2. **Semaphore Bulkhead:** Limit concurrent calls (lighter, shares thread pool)

```
@Service
public class OrderFulfillmentService {

    // Separate thread pool per dependency = full isolation
    private final ExecutorService paymentPool = new ThreadPoolExecutor(
        5, 10, 60, TimeUnit.SECONDS,
        new LinkedBlockingQueue<>(50),                          // Queue 50 requests
        new ThreadFactoryBuilder().setNameFormat("payment-%d").build(),
        new ThreadPoolExecutor.AbortPolicy());                  // Reject when full → fast fail

    private final ExecutorService inventoryPool = new ThreadPoolExecutor(
        10, 20, 60, TimeUnit.SECONDS,
        new LinkedBlockingQueue<>(100),
        new ThreadFactoryBuilder().setNameFormat("inventory-%d").build(),
        new ThreadPoolExecutor.CallerRunsPolicy());

    public CompletableFuture<FulfillmentResult> fulfill(Order order) {
        // Payment hangs? Only paymentPool threads affected — inventory unaffected
        CompletableFuture<PaymentResult> payment =
            CompletableFuture.supplyAsync(() -> paymentService.charge(order), paymentPool)
                .orTimeout(5, TimeUnit.SECONDS)
                .exceptionally(ex -> PaymentResult.failed(ex.getMessage()));

        CompletableFuture<InventoryResult> inventory =
            CompletableFuture.supplyAsync(() -> inventoryService.reserve(order), inventoryPool)
                .orTimeout(3, TimeUnit.SECONDS)
                .exceptionally(ex -> InventoryResult.failed(ex.getMessage()));

        return CompletableFuture.allOf(payment, inventory)
            .thenApply(v -> FulfillmentResult.of(payment.join(), inventory.join()));
    }
}
```

```
# application.yml — Resilience4j bulkhead
resilience4j:  bulkhead:    instances:      payment:        maxConcurrentCalls: 10        maxWaitDuration
```

■ 9. BLUE-GREEN DEPLOYMENT

■ Q: What is Blue-Green Deployment and when to use it vs Canary?

A: Blue-Green maintains two identical production environments. Blue = current, Green = new version. Traffic switches instantly (100% at once) from Blue to Green.

Flow: Deploy Green → Test Green → Switch LB 100% → Blue becomes standby → Decommission Blue.

Blue-Green vs Canary:

Blue-Green	Canary	
Traffic switch	100% instant	Gradual (1% → 5% → 100%)
Rollback	Instant (flip LB back)	Slow (drain canary)
Cost	Double infra during transition	Minimal extra
Risk	All users see new version at once	Small blast radius
DB changes	Must be backward compatible	Same requirement

When to use Blue-Green: Need instant rollback, compliance requirements, database migration scenarios.

When to use Canary: Gradual confidence building, A/B testing, cost-sensitive.

```
# Kubernetes: Switch traffic by changing Service selector
apiVersion: v1kind: Servicemetadata:  name: app-servicespec:  selector:    app: myapp    version: blue   # Change to '
    targetPort: 8080---
# Green deployment
apiVersion: apps/v1kind: Deploymentmetadata:  name: app-greenspec:  replicas: 3  selector:    matchLabels
        image: myapp:2.0.0        readinessProbe:          httpGet:            path: /actuator/health/readiness
```

```
// Spring Boot — graceful shutdown to drain Blue
// application.yml: server.shutdown=graceful
// spring.lifecycle.timeout-per-shutdown-phase=30s

@Component
public class DeploymentHealthIndicator implements HealthIndicator {
    @Value("${app.version}")
    private String version;

    @Override
    public Health health() {
        return Health.up()
            .withDetail("version", version)
            .withDetail("db", checkDatabase() ? "connected" : "failed")
            .build();
    }
}
```

■ 10. CORRELATION ID

■ Q: What is a Correlation ID and how do you propagate it across microservices?

A: A Correlation ID (Trace ID) is a unique identifier attached to every request, flowing through all services, enabling end-to-end request tracing across a distributed system.

Why critical: Without it, correlating logs across 10 services for a single user request is nearly impossible.

Propagation points: HTTP headers, Kafka message headers, gRPC metadata, async tasks.

```
// 1. Filter to generate/extract Correlation ID
@Component
@Order(1)
public class CorrelationIdFilter implements Filter {

    public static final String CORRELATION_HEADER = "X-Correlation-Id";
    public static final String MDC_KEY = "correlationId";

    @Override
    public void doFilter(ServletRequest req, ServletResponse res,
                         FilterChain chain) throws IOException, ServletException {
        HttpServletRequest httpReq = (HttpServletRequest) req;
        String id = httpReq.getHeader(CORRELATION_HEADER);
        if (id == null || id.isBlank()) id = UUID.randomUUID().toString();

        MDC.put(MDC_KEY, id);
        ((HttpServletResponse) res).setHeader(CORRELATION_HEADER, id);

        try {
            chain.doFilter(req, res);
        } finally {
            MDC.clear();  // CRITICAL: prevent MDC leak in thread pools
        }
    }
}

// 2. Propagate via WebClient (reactive HTTP calls)
@Bean
public WebClient webClient() {
    return WebClient.builder()
        .filter((request, next) -> {
            String id = MDC.get(CorrelationIdFilter.MDC_KEY);
            return next.exchange(ClientRequest.from(request)
                .header(CorrelationIdFilter.CORRELATION_HEADER,
                    id != null ? id : UUID.randomUUID().toString())
                .build());
        }).build();
}

// 3. Propagate via Kafka headers
@Service
public class EventPublisher {
    public void publish(String topic, Object event) {
```

```
        String id = MDC.get(CorrelationIdFilter.MDC_KEY);
        kafkaTemplate.send(MessageBuilder.withPayload(event)
            .setHeader("X-Correlation-Id", id)
            .setHeader(KafkaHeaders.TOPIC, topic)
            .build());
    }
}

// 4. Logback pattern — include correlationId in every log line
// logback-spring.xml:
// <pattern>%d{ISO8601} [%thread] %-5level %logger - [%X{correlationId}] %msg%n</pattern>
```

■ 11. QUERY OPTIMIZATION

■ Q: What are the key techniques for database query optimization?

A: Query optimization is the #1 lever for improving application performance. Poor queries cause most production incidents.

Optimization Hierarchy (most impactful first):

1. **Proper Indexing** — can reduce query from full scan (O(n)) to index seek (O(log n))
2. **Eliminate N+1** — replace N+1 queries with JOIN FETCH or batch fetch
3. **Select Only Needed Columns** — avoid `SELECT *`
4. **Use Pagination** — never fetch unbounded rows
5. **EXPLAIN ANALYZE** — always profile before and after
6. **Covering Index** — all needed columns in index = no table lookup
7. **Partial Index** — index subset of rows (e.g., only ACTIVE records)
8. **Read Replicas** — route SELECT queries to replica
9. **Denormalization** — pre-join data for read-heavy queries
10. **Query Result Caching** — cache expensive aggregates

```
// N+1 Problem — most common mistake
// BAD: 1 query for orders + N queries for each order's items
List<Order> orders = orderRepository.findAll();
orders.forEach(o -> o.getItems().size()); // Each triggers SELECT

// GOOD: Single query with JOIN FETCH
@Query("SELECT o FROM Order o JOIN FETCH o.items WHERE o.status = :status")
List<Order> findWithItems(@Param("status") OrderStatus status);

// EntityGraph for flexible eager loading
@EntityGraph(attributePaths = {"items", "items.product", "customer"})
List<Order> findByStatus(OrderStatus status);

// Projection — only fetch needed columns
public interface OrderSummary {
    Long getId();
    BigDecimal getAmount();
    String getStatus();
    Instant getCreatedAt();
}
List<OrderSummary> findByCustomerId(String customerId);

// Dynamic query with Specification pattern
@Service
public class OrderQueryService {
    public Page<Order> search(OrderSearchRequest req, Pageable pageable) {
        Specification<Order> spec = Specification.where(null);

        if (req.getCustomerId() != null)
            spec = spec.and((root, q, cb) ->
                cb.equal(root.get("customerId"), req.getCustomerId()));
        if (req.getStatus() != null)
            spec = spec.and((root, q, cb) ->
```

```
                cb.equal(root.get("status"), req.getStatus()));
        if (req.getFromDate() != null)
            spec = spec.and((root, q, cb) ->
                cb.greaterThanOrEqualTo(root.get("createdAt"), req.getFromDate()));

        return orderRepository.findAll(spec, pageable);
    }
}

// Partial + covering index via Flyway migration
// V3__indexes.sql:
// CREATE INDEX CONCURRENTLY idx_active_orders
//     ON orders (customer_id, created_at DESC)
//     WHERE status NOT IN ('COMPLETED','CANCELLED');
//
// CREATE INDEX CONCURRENTLY idx_covering
//     ON orders (customer_id, status) INCLUDE (id, amount, created_at);
```

■ 12. DESERIALIZATION

■ Q: What are the security risks of deserialization and how do you prevent them?

A: Insecure deserialization is OWASP Top 10. Attackers craft malicious payloads that execute arbitrary code when deserialized. Java's native `ObjectInputStream` is notoriously vulnerable (Log4Shell-adjacent exploits).

Risks:

- Java native deserialization → Remote Code Execution (RCE)
- JSON polymorphic typing → type confusion
- XML External Entity (XXE) expansion

Prevention:

1. Never use Java native serialization (`ObjectInputStream`) over network
2. Use JSON/Protobuf with strict schema validation
3. Disable default typing in Jackson globally
4. Validate all input after deserialization
5. Use allowlists for acceptable types

```
// NEVER DO THIS
ObjectInputStream ois = new ObjectInputStream(socket.getInputStream());
Object obj = ois.readObject();  // RCE vulnerability!

// SAFE: Jackson with strict configuration
@Configuration
public class JacksonConfig {
    @Bean
    public ObjectMapper objectMapper() {
        ObjectMapper mapper = new ObjectMapper();
        mapper.deactivateDefaultTyping();         // Prevent polymorphic type attacks
        mapper.registerModule(new JavaTimeModule());
        mapper.configure(DeserializationFeature.FAIL_ON_UNKNOWN_PROPERTIES, false);
        return mapper;
    }
}

// Bean Validation on deserialized input
public record CreateOrderRequest(
    @NotNull @Size(min=1, max=100) String customerId,
    @NotEmpty @Size(max=50) List<@Valid OrderItemRequest> items,
    @NotNull @Positive @DecimalMax("999999.99") BigDecimal totalAmount
) {}

public record OrderItemRequest(
    @NotNull Long productId,
    @Positive @Max(1000) int quantity,
    @NotNull @Positive BigDecimal price
) {}

// Custom deserializer for sensitive fields
public class MoneyDeserializer extends JsonDeserializer<BigDecimal> {
    @Override
    public BigDecimal deserialize(JsonParser p, DeserializationContext ctx) throws IOException {
```

```
        String value = p.getText().trim();
        if (!value.matches("\\d{1,12}(\\.\\d{1,4})?"))
            throw new InvalidFormatException(p, "Invalid money format", value, BigDecimal.class);
        return new BigDecimal(value).setScale(4, RoundingMode.HALF_UP);
    }
}
```

■ 13. BFF (BACKEND FOR FRONTEND)

■ Q: What is the BFF pattern and when should you apply it?

A: BFF creates a dedicated backend for each client type (mobile, web, smart TV). Each BFF aggregates and transforms data specifically for its client's needs.

Problem it solves:

- Mobile needs compact responses (battery + bandwidth constrained)
- Web dashboard needs rich, paginated, multi-joined data
- A single generic API either over-fetches for mobile or under-fetches for web

When to use: Multiple distinct client types with very different data needs; teams aligned by product (frontend team owns their BFF).

When NOT to use: Single client type; simple APIs; small teams (complexity overhead not worth it).

```
Client Architecture:
  Mobile App  →  Mobile BFF   ■■■
  Web App     →  Web BFF      ■■■→ [UserSvc] [OrderSvc] [ProductSvc]
  3rd Party   →  Public API   ■■■
```

```java
// Mobile BFF — single call aggregates data from 3 services
@RestController
@RequestMapping("/mobile/v1")
public class MobileBFF {

    @GetMapping("/dashboard")
    public Mono<MobileDashboard> getDashboard(@RequestHeader("User-Id") String userId) {
        // Parallel calls to 3 downstream services
        Mono<UserSummary> user = userClient.getSummary(userId)
            .timeout(Duration.ofSeconds(2))
            .onErrorReturn(UserSummary.guest());

        Mono<List<OrderSummary>> orders = orderClient.getRecent(userId, 5)
            .timeout(Duration.ofSeconds(2))
            .onErrorReturn(List.of());

        Mono<List<ProductCard>> recs = productClient.getRecommendations(userId, 3)
            .timeout(Duration.ofSeconds(2))
            .onErrorReturn(List.of());

        return Mono.zip(user, orders, recs)
            .map(t -> new MobileDashboard(t.getT1(), t.getT2(), t.getT3()));
    }
}

// Mobile response — minimal, compact
public record MobileDashboard(
    UserSummary user,                  // Only 4 fields
    List<OrderSummary> recentOrders,   // Only id, status, amount per order
    List<ProductCard> recommendations  // Only id, name, image, price per product
) {}
```

```
// Web BFF — richer paginated response
@RestController
@RequestMapping("/web/v1")
public class WebBFF {
    @GetMapping("/orders")
    public Mono<PagedResponse<OrderDetail>> getOrders(
            @RequestHeader("User-Id") String userId,
            Pageable pageable) {
        // Web gets full detail with pagination, sorting, filtering
        return orderClient.getAll(userId, pageable);
    }
}
```

■ 14. MAPREDUCE

■ Q: What is MapReduce and when should you use it vs Spark?

A: MapReduce is a batch processing paradigm: split → map (parallel processing) → shuffle/sort → reduce (aggregate). Designed for petabyte-scale batch jobs on commodity hardware.

Phases:

1. **Split:** Input divided into chunks (e.g., 128MB HDFS blocks)
2. **Map:** Each chunk processed in parallel, outputs `(key, value)` pairs
3. **Shuffle & Sort:** Framework groups all values by key across all mappers
4. **Reduce:** Aggregate values per key → final output

MapReduce vs Spark:

MapReduce	Apache Spark	
Processing	Disk-based	In-memory
Speed	Slower	10-100x faster for iterative
Fault tolerance	Checkpoint to disk	RDD lineage
Use case	Single-pass batch ETL	ML, iterative, SQL
Learning curve	Simple	Moderate

Use MapReduce when: Simple single-pass ETL over petabytes, HDFS-native jobs, cost matters.

Use Spark when: Multiple passes over data, ML training, real-time, SQL analytics.

```java
// Spring Batch — MapReduce-style batch in Java
@Configuration
@EnableBatchProcessing
public class RevenueAnalyticsBatch {

    @Bean
    public Job revenueJob(JobBuilderFactory jbf, StepBuilderFactory sbf) {
        return jbf.get("revenueJob")
            .start(mapStep(sbf))
            .next(reduceStep(sbf))
            .build();
    }

    // MAP: Read orders, emit (customerId, amount) pairs
    @Bean
    public Step mapStep(StepBuilderFactory sbf) {
        return sbf.get("mapStep")
            .<Order, CustomerRevenue>chunk(1000)
            .reader(new JpaPagingItemReaderBuilder<Order>()
                .name("orderReader")
                .queryString("SELECT o FROM Order o WHERE o.status='COMPLETED'")
                .pageSize(1000)
                .build())
            .processor(order -> new CustomerRevenue(order.getCustomerId(), order.getAmount()))
            .writer(interimWriter())
```

```
                .build();
        }

        // REDUCE: Group by customerId, sum amounts
        @Bean
        public Step reduceStep(StepBuilderFactory sbf) {
            return sbf.get("reduceStep")
                .<CustomerRevenue, CustomerRevenueSummary>chunk(100)
                .reader(interimReader())
                .processor(new CustomerRevenueAggregator()) // Groups and sums
                .writer(summaryWriter())
                .build();
        }
    }
```

■ 15. AVAILABILITY

■ Q: How do you achieve and calculate system availability?

A: Availability = uptime / total time, expressed as "nines."

Availability Tiers:

SLA	Downtime/Year	Downtime/Month
99% (2 nines)	3.65 days	7.3 hours
99.9% (3 nines)	8.76 hours	43.8 min
99.99% (4 nines)	52.6 min	4.4 min
99.999% (5 nines)	5.26 min	26.3 sec

Series vs Parallel:

- Series: `A = A1 × A2 × A3` (each component degrades overall availability)
- Parallel: `A = 1 - (1-A1)(1-A2)` (redundancy dramatically improves availability)

Example: Two 99.9% servers in parallel = 1 - (0.001)² = **99.9999%**

Strategies: No SPOF (redundancy), health checks + auto-healing, multi-AZ, graceful degradation, chaos engineering.

```java
// Liveness vs Readiness probes (critical for Kubernetes HA)
@RestController
public class HealthController {

    @Autowired private ApplicationAvailability availability;

    // Liveness: Am I alive? (if fails, k8s restarts pod)
    @GetMapping("/health/live")
    public ResponseEntity<String> liveness() {
        return ResponseEntity.ok("OK");
    }

    // Readiness: Am I ready for traffic? (if fails, removed from LB rotation)
    @GetMapping("/health/ready")
    public ResponseEntity<String> readiness() {
        if (availability.getReadinessState() == ReadinessState.ACCEPTING_TRAFFIC) {
            return ResponseEntity.ok("READY");
        }
        return ResponseEntity.status(503).body("NOT_READY");
    }
}

// Graceful shutdown — drain requests before pod termination
// application.yml:
// server.shutdown=graceful
// spring.lifecycle.timeout-per-shutdown-phase=30s

// Distributed lock for single-leader scheduling in HA cluster
@Service
public class LeaderAwareScheduler {
```

```
    @Autowired private RedissonClient redisson;

    @Scheduled(fixedDelay = 60_000)
    public void runScheduledJob() {
        RLock lock = redisson.getLock("scheduler-lock");
        try {
            if (lock.tryLock(0, 55, TimeUnit.SECONDS)) {
                try {
                    doScheduledWork();
                } finally { lock.unlock(); }
            }
        } catch (InterruptedException e) {
            Thread.currentThread().interrupt();
        }
    }
}
```

■ 16. SQL vs NoSQL

■ Q: How do you decide between SQL and NoSQL?

A: There is no universal winner. The choice depends on access patterns, consistency needs, and scale.

Choose SQL when:

- ACID transactions required (payments, inventory adjustments, transfers)
- Complex relationships and JOINs needed
- Schema is well-defined and stable
- Strong consistency required
- Ad-hoc reporting and analytics

Choose NoSQL when:

- Document: Flexible/evolving schema, hierarchical data (user profiles, product catalogs)
- Key-Value: Ultra-fast simple lookups, sessions, counters
- Column-Family: Massive write throughput, time-series, IoT data streams
- Graph: Highly connected data, social graphs, recommendation engines

CAP Theorem positions:

- PostgreSQL → CP (Consistency + Partition tolerance)
- Cassandra → AP (Availability + Partition tolerance, eventual consistency)
- MongoDB → CP (default) with tunable consistency

```
// Service layer using multiple stores appropriately
@Service
public class UserService {
    // Primary data: PostgreSQL (ACID, relationships)
    private final UserJpaRepository userRepo;
    // Fast lookups: Redis (sub-millisecond)
    private final UserRedisRepository cache;
    // Activity feed: MongoDB (flexible schema)
    private final UserActivityMongoRepo activityRepo;
    // Full-text search: Elasticsearch
    private final UserSearchRepository searchRepo;

    // Cache-aside pattern
    public User findById(Long id) {
        return cache.findById(id)
            .orElseGet(() -> {
                User user = userRepo.findById(id)
                    .orElseThrow(() -> new UserNotFoundException(id));
                cache.put(id, user, Duration.ofMinutes(30));
                return user;
            });
    }

    // Write-through: update all stores on mutation
    @Transactional
    public User update(Long id, UpdateUserRequest req) {
        User user = userRepo.findById(id).orElseThrow();
        user.update(req);
```

```
        userRepo.save(user);
        cache.evict(id);                    // Invalidate cache
        searchRepo.index(user);             // Update search index
        activityRepo.recordUpdate(id, req); // Log activity
        return user;
    }
}
```

■ 17. REST

■ Q: What makes an API truly RESTful and what are the HTTP method semantics?

A: REST (Roy Fielding, 2000) has 6 architectural constraints: Stateless, Client-Server, Cacheable, Uniform Interface, Layered System, Code on Demand (optional).

HTTP Method Semantics:

Method	Safe	Idempotent	Use
GET	Yes	Yes	Read resource
HEAD	Yes	Yes	Check resource existence
POST	No	No	Create (non-idempotent)
PUT	No	Yes	Replace entire resource
PATCH	No	No*	Partial update
DELETE	No	Yes	Delete

REST vs GraphQL vs gRPC:

REST	GraphQL	gRPC	
Over-fetching	Common	None	None
Caching	Easy (HTTP)	Complex	Complex
Type safety	Manual	Schema	Protobuf
Browser native	Yes	Yes	Limited
Real-time	Polling/SSE	Subscriptions	Streaming
Best for	Public APIs	Mobile clients	Internal services

```
// HATEOAS — Level 3 REST (Richardson Maturity Model)
@GetMapping("/{id}")
public EntityModel<OrderDTO> getOrder(@PathVariable Long id) {
    Order order = orderService.findById(id);
    EntityModel<OrderDTO> model = EntityModel.of(orderMapper.toDTO(order));

    model.add(linkTo(methodOn(OrderController.class).getOrder(id)).withSelfRel());

    // Dynamic links based on order state
    if (order.getStatus() == OrderStatus.PLACED) {
        model.add(linkTo(methodOn(OrderController.class)
            .cancelOrder(id)).withRel("cancel"));
        model.add(linkTo(methodOn(PaymentController.class)
            .getPayment(order.getPaymentId())).withRel("payment"));
    }
    if (order.getStatus() == OrderStatus.SHIPPED) {
        model.add(linkTo(methodOn(TrackingController.class)
            .track(order.getTrackingId())).withRel("tracking"));
    }
```

```
    return model;
}
```

■ 18. HIGH AVAILABILITY

■ Q: What is the difference between High Availability and Fault Tolerance?

A:

- **Fault Tolerance:** System continues WITHOUT interruption after failure (zero downtime, transparent to users). Requires real-time failover — expensive.
- **High Availability:** System recovers QUICKLY after failure (minimal downtime — seconds to minutes). Automated recovery — less expensive.

HA Patterns:

- **Active-Active:** All instances serving traffic. Failure seamless — others absorb load.
- **Active-Passive:** Primary active, standby warm and ready. Failover = ~10-30s.

Active-Active use: Stateless services, global distribution.

Active-Passive use: Stateful services (primary DB), where synchronization complexity is high.

```
// Multi-region HA with Spring Cloud + Kubernetes
// k8s deployment with anti-affinity (spread pods across nodes/zones)
/*
spec:
  affinity:
    podAntiAffinity:
      requiredDuringSchedulingIgnoredDuringExecution:
      - topologyKey: "topology.kubernetes.io/zone"  # Spread across availability zones
        labelSelector:
          matchLabels:
            app: myapp
*/

// Circuit breaker for Active-Passive failover between regions
@Service
public class RegionalOrderService {

    @CircuitBreaker(name = "primary-region", fallbackMethod = "useSecondaryRegion")
    public Order createOrder(CreateOrderRequest req) {
        return primaryRegionClient.createOrder(req);
    }

    public Order useSecondaryRegion(CreateOrderRequest req, Exception ex) {
        log.warn("Primary region failed, failing over to secondary");
        metricsService.recordFailover("primary→secondary");
        return secondaryRegionClient.createOrder(req);
    }
}
```

■ 19. CONSENSUS

■ Q: What is distributed consensus and which algorithms solve it?

A: Consensus is getting all nodes in a distributed system to agree on a single value despite node failures and network partitions.

FLP Impossibility: In an async distributed system, no algorithm can guarantee consensus if even one node can fail. Real systems choose safety over liveness under extreme partitions.

Algorithms:

- **Raft:** Most understandable. Leader election + log replication. Used by etcd, CockroachDB, TiKV.
- **Paxos:** Foundational but complex. Used by Google Chubby, Spanner.
- **ZAB:** ZooKeeper Atomic Broadcast — used by Kafka (metadata), HDFS NameNode.
- **PBFT:** Byzantine fault tolerant — handles malicious nodes. Used in blockchain.

Raft Phases:

1. Leader Election: Candidate requests votes → majority wins → becomes leader
2. Log Replication: Leader appends to log, replicates to followers, commits when majority ACK
3. Safety: Only committed entries applied to state machine

Where used: etcd (Kubernetes), ZooKeeper (Kafka), CockroachDB, Consul service discovery.

```
// Using Redisson for distributed consensus primitives
@Service
public class DistributedCoordinator {

    private final RedissonClient redisson;

    // Leader election via Redisson (uses Raft internally in cluster mode)
    public void campaignForLeadership(String serviceName) {
        RLeaderLatch latch = redisson.getLeaderLatch("election:" + serviceName);
        latch.addListener(() -> {
            if (latch.hasLeadership()) {
                log.info("This node is now LEADER for {}", serviceName);
                onLeadershipAcquired(serviceName);
            }
        });
        latch.start();
    }

    // Distributed atomic counter (Raft-replicated across Redis cluster)
    public long incrementCounter(String counterName) {
        return redisson.getAtomicLong(counterName).incrementAndGet();
    }

    // Distributed semaphore (consensus on available permits)
    public boolean tryAcquirePermit(String resource, int maxPermits, Duration timeout) {
        RSemaphore semaphore = redisson.getSemaphore("semaphore:" + resource);
        semaphore.trySetPermits(maxPermits);
        try {
            return semaphore.tryAcquire(timeout.toMillis(), TimeUnit.MILLISECONDS);
        } catch (InterruptedException e) {
```

```
            Thread.currentThread().interrupt();
            return false;
        }
    }
}
```

■ 20. INDEXING

■ Q: What index types exist and how do you choose the right one?

A: Indexes speed up reads at the cost of write performance and storage. Choosing the right type and column order is critical.

Index Types:

Type	Best For	Notes
B-Tree	Equality + range, sorting	Default for most RDBMS
Hash	Equality only (`=`)	Faster for equality, no range support
Composite	Multi-column WHERE/ORDER BY	Column order matters!
Partial	Subset of rows	Smaller, faster, uses WHERE clause
Covering	All query columns	No table lookup needed
GIN/GiST	Full-text, arrays, JSON	PostgreSQL specific

Composite Index Rule: Put most selective column first; range column last.

`INDEX(status, created_at)` works for `WHERE status='ACTIVE' ORDER BY created_at` but NOT `WHERE created_at > ?` alone.

Don't index: Low-cardinality columns (boolean); tables with heavy bulk inserts; small tables.

```
// JPA index annotations
@Entity
@Table(name = "orders", indexes = {
    @Index(name = "idx_customer_status_created",
           columnList = "customer_id, status, created_at"),
    @Index(name = "idx_tracking_number",
           columnList = "tracking_number", unique = true)
})
public class Order { ... }

// Flyway migration for advanced index types
/*
-- Partial index: only active orders (much smaller index)
CREATE INDEX CONCURRENTLY idx_active_orders
    ON orders (customer_id, created_at DESC)
    WHERE status NOT IN ('COMPLETED', 'CANCELLED');

-- Covering index: serves query entirely from index
CREATE INDEX CONCURRENTLY idx_order_covering
    ON orders (customer_id, status) INCLUDE (id, amount, created_at);

-- GIN index for JSONB fields
CREATE INDEX CONCURRENTLY idx_metadata
    ON orders USING gin(metadata jsonb_path_ops);

-- Check index usage (find unused indexes)
SELECT indexname, idx_scan, idx_tup_read
```

```
FROM pg_stat_user_indexes WHERE tablename = 'orders'
ORDER BY idx_scan ASC;
*/
```

■ 21. MESSAGE QUEUE

■ Q: When do you use a Message Queue vs Pub/Sub vs Kafka?

A: All solve decoupling but have different delivery semantics.

Queue (Point-to-Point): One message, one consumer. Work distribution, guaranteed single processing.

Pub/Sub (Fan-out): One message, ALL subscribers receive it. Event notification.

Kafka Log: Messages retained (configurable), consumer groups can replay. Event sourcing, audit.

Kafka vs RabbitMQ:

Kafka	RabbitMQ	
Model	Append-only log	Queue (delete on ack)
Replay	Yes	No
Throughput	Millions/sec	Tens of thousands/sec
Ordering	Per-partition	Per-queue
Routing	Topic/partition	Flexible exchanges

```
// Kafka consumer group (each instance processes different partitions)
@KafkaListener(
    topics = "orders",
    groupId = "payment-service",    // Same group = one instance per partition
    concurrency = "3"               // 3 concurrent listener threads
)
public void handleOrder(@Payload OrderEvent event, Acknowledgment ack) {
    try {
        paymentService.process(event); // Idempotent
        ack.acknowledge();
    } catch (RetryableException e) {
        // No ack = redelivery
    } catch (PermanentException e) {
        ack.acknowledge(); // Ack but send to DLQ
        dlqTemplate.send("orders.dlq", event);
    }
}

// RabbitMQ work queue with dead letter queue
@Configuration
public class RabbitConfig {
    @Bean
    public Queue orderQueue() {
        return QueueBuilder.durable("order.processing")
            .withArgument("x-dead-letter-exchange", "order.dlx")
            .withArgument("x-message-ttl", 3_600_000)
            .build();
    }

    @Bean
    public Queue deadLetterQueue() {
```

```
        return QueueBuilder.durable("order.failed").build();
    }
}
```

■ 22. RETRY LOGIC

■ Q: How do you implement robust retry logic without causing thundering herd?

A: Retry logic re-attempts transient failures. Key rules: exponential backoff + jitter (avoid all clients retrying simultaneously), idempotency (retries must be safe), retry budget (bound retry overhead).

Retry vs Circuit Breaker: Retry handles transient failures; Circuit Breaker handles prolonged outages. Use both together.

What to retry: Network timeouts, 503 errors, transient DB errors.

Never retry: 4xx client errors (bug in request), business logic exceptions, non-idempotent operations without idempotency key.

```
@Configuration
@EnableRetry
public class RetryConfig {}

@Service
public class ExternalApiService {

    @Retryable(
        retryFor = {HttpServerErrorException.class, ResourceAccessException.class},
        noRetryFor = {HttpClientErrorException.class},  // Don't retry 4xx
        maxAttempts = 3,
        backoff = @Backoff(
            delay = 1000,        // Initial delay 1s
            multiplier = 2.0,    // Double each attempt
            random = true,       // Add jitter
            maxDelay = 30000     // Cap at 30s
        )
    )
    public ApiResponse callExternalApi(ApiRequest request) {
        return restTemplate.postForObject("/external/api", request, ApiResponse.class);
    }

    @Recover  // Called when all retries exhausted
    public ApiResponse recoverFromFailure(Exception ex, ApiRequest request) {
        log.error("All retries exhausted for {}", request.getId(), ex);
        return ApiResponse.defaultResponse();
    }
}

// Manual retry with full jitter backoff (best for avoiding thundering herd)
public <T> T executeWithFullJitter(Supplier<T> op, int maxAttempts, long baseDelayMs) {
    Random rng = new Random();
    for (int attempt = 1; attempt <= maxAttempts; attempt++) {
        try {
            return op.get();
        } catch (RetryableException e) {
            if (attempt == maxAttempts) throw e;
            // Full jitter: sleep random duration from 0 to cap
            long cap = Math.min(baseDelayMs * (1L << (attempt - 1)), 30_000L);
```

```
            long sleep = (long) (cap * rng.nextDouble());
            LockSupport.parkNanos(TimeUnit.MILLISECONDS.toNanos(sleep));
        }
    }
    throw new RuntimeException("Should not reach here");
}
```

■ 23. CANARY RELEASE

■ Q: How do you implement Canary Release and what metrics signal rollback?

A: Canary gradually rolls out new version to small user percentage, monitors metrics, expands if healthy.

Typical Stages: 1% → 5% → 25% → 50% → 100% (15-30 min monitoring at each stage).

Rollback Signals:

- Error rate increase > baseline + threshold (e.g., 0.1% → 1%)
- p99 latency regression > 20%
- Business metric degradation (checkout completion, conversion rate)
- CPU/memory spike on canary pods

```
# Istio VirtualService for Canary traffic splitting
apiVersion: networking.istio.io/v1alpha3kind: VirtualServicemetadata:  name: myapp-vsspec:  http:  - match:
    - headers:
        x-canary:          exact: "true"    route:    - destination:
        host: myapp        subset: canary  - route:
    - destination:
        host: myapp        subset: stable      weight: 95    - destination:
        host: myapp        subset: canary      weight: 5
```

```
// Canary metrics comparison with Micrometer
@RestController
public class CheckoutController {

    @Value("${app.canary:false}")
    private boolean isCanary;

    private final MeterRegistry registry;

    @PostMapping("/checkout")
    public ResponseEntity<CheckoutResult> checkout(@RequestBody CheckoutRequest req) {
        String variant = isCanary ? "canary" : "stable";
        return Timer.builder("checkout.duration")
            .tag("variant", variant)
            .publishPercentiles(0.5, 0.95, 0.99)
            .register(registry)
            .record(() -> {
                try {
                    CheckoutResult result = checkoutService.process(req);
                    registry.counter("checkout.success", "variant", variant).increment();
                    return ResponseEntity.ok(result);
                } catch (Exception e) {
                    registry.counter("checkout.error", "variant", variant,
                        "reason", e.getClass().getSimpleName()).increment();
                    throw e;
                }
            });
    }
}
// Monitor in Grafana: compare error_rate{variant="canary"} vs error_rate{variant="stable"}
```

■ 24. MONITORING

■ Q: What is the difference between monitoring and observability? What should you monitor?

A:

- **Monitoring:** Track predefined metrics. Answers "Is something wrong?"
- **Observability:** Understand WHY something is wrong from logs + metrics + traces.
- **Alerting:** Notify humans/automation when metrics breach thresholds.

Google SRE Golden Signals (monitor these 4 for any service):

1. **Latency:** How long requests take (p50, p95, p99)
2. **Traffic:** Requests per second, events per second
3. **Errors:** Error rate, error count by type
4. **Saturation:** CPU%, memory%, queue depth, connection pool usage

Three Pillars of Observability:

- Metrics → Prometheus + Grafana
- Logs → ELK Stack or Grafana Loki
- Traces → Jaeger or Zipkin

```
// Micrometer — production metrics in Spring Boot
@Service
public class OrderMetrics {
    private final MeterRegistry registry;

    @PostConstruct
    public void init() {
        // Gauge: live reading from application state
        Gauge.builder("orders.active.count", orderRepo, OrderRepository::countActive)
            .description("Orders currently in PROCESSING state")
            .register(registry);

        // Histogram: latency distribution
        Timer.builder("order.creation.time")
            .publishPercentiles(0.5, 0.95, 0.99)
            .publishPercentileHistogram()
            .serviceLevelObjectives(
                Duration.ofMillis(100), Duration.ofMillis(500), Duration.ofSeconds(1))
            .register(registry);
    }

    public Order createOrder(CreateOrderRequest req) {
        return registry.timer("order.creation.time").record(() -> {
            Order order = doCreateOrder(req);
            registry.counter("orders.created",
                "type", order.getType().name()).increment();
            registry.summary("order.amount.dollars")
                .record(order.getAmount().doubleValue());
            return order;
        });
    }
}
```

```
// application.yml — expose Prometheus scrape endpoint
// management.endpoints.web.exposure.include=prometheus,health,info
// management.metrics.export.prometheus.enabled=true
```

■ 25. CONNECTION POOLING

■ Q: How do you tune HikariCP connection pool for production?

A: Connection pooling reuses database connections, avoiding ~50-200ms connection setup overhead per request. HikariCP is Spring Boot's default and fastest JVM pool.

Tuning Formula: Pool size = `(core count × 2) + effective_disk_spindles`

HikariCP recommendation: For most apps, pool size = 10-20. **Bigger is NOT always better** — excess connections cause DB-side contention.

HikariCP settings:

- `maximumPoolSize`: Max concurrent connections (start at 10, increase under load)
- `minimumIdle`: Connections maintained when idle (set = maximumPoolSize for fixed pool)
- `connectionTimeout`: Max wait for connection from pool (30s default)
- `idleTimeout`: Remove idle connections (10 min)
- `maxLifetime`: Max connection age (30 min, less than DB's wait_timeout)
- `keepaliveTime`: Ping idle connections (60s, prevent firewall timeout)

```
@Configuration
public class DataSourceConfig {

    @Bean
    public HikariDataSource dataSource() {
        HikariConfig config = new HikariConfig();
        config.setJdbcUrl("jdbc:postgresql://db:5432/mydb");
        config.setUsername("app_user");
        config.setPassword("${DB_PASSWORD}");

        config.setMaximumPoolSize(20);
        config.setMinimumIdle(10);            // Fixed-size pool in prod
        config.setConnectionTimeout(30_000);
        config.setIdleTimeout(600_000);       // 10 min
        config.setMaxLifetime(1_800_000);     // 30 min < MySQL wait_timeout (8h default)
        config.setKeepaliveTime(60_000);      // Ping every 60s

        // Prepared statement caching
        config.addDataSourceProperty("cachePrepStmts", "true");
        config.addDataSourceProperty("prepStmtCacheSize", "250");
        config.addDataSourceProperty("prepStmtCacheSqlLimit", "2048");
        config.addDataSourceProperty("useServerPrepStmts", "true");

        config.setPoolName("OrderDB-Pool");
        return new HikariDataSource(config);
    }
}

// Monitor pool exhaustion
@EventListener(ApplicationReadyEvent.class)
public void monitorPool() {
    HikariPoolMXBean pool = hikariDataSource.getHikariPoolMXBean();
    Gauge.builder("db.pool.active", pool, HikariPoolMXBean::getActiveConnections)
```

```
        .register(meterRegistry);
    Gauge.builder("db.pool.pending", pool, HikariPoolMXBean::getThreadsAwaitingConnection)
        .register(meterRegistry);
    // Alert if db.pool.pending > 0 sustained for > 30s
}
```

■ 26. WEBSOCKETS

■ Q: When do you use WebSockets vs SSE vs Long Polling?

A: WebSockets provide full-duplex bidirectional communication over a single TCP connection. Upgraded from HTTP/1.1.

Comparison:

WebSockets	Server-Sent Events (SSE)	Long Polling	
Direction	Bidirectional	Server→Client only	Server→Client
Protocol	WS/WSS	HTTP	HTTP
Reconnection	Manual	Auto	Manual
Browser support	Excellent	Excellent	Excellent
Load balancer	Sticky sessions needed	Standard	Standard
Use case	Chat, gaming, collaboration	Notifications, feeds	Legacy fallback

Use WebSockets: Real-time chat, collaborative editing, live trading, multiplayer games.

Use SSE: Notifications, live feeds, progress tracking (simpler; HTTP-native; auto-reconnect).

Use Polling: Legacy support, simple notification systems.

```
// Spring Boot WebSocket with STOMP
@Configuration
@EnableWebSocketMessageBroker
public class WebSocketConfig implements WebSocketMessageBrokerConfigurer {

    @Override
    public void configureMessageBroker(MessageBrokerRegistry config) {
        config.enableSimpleBroker("/topic", "/queue");  // In-memory broker
        // For production: use RabbitMQ relay
        // config.enableStompBrokerRelay("/topic","/queue")
        //     .setRelayHost("rabbitmq").setRelayPort(61613);
        config.setApplicationDestinationPrefixes("/app");
        config.setUserDestinationPrefix("/user");
    }

    @Override
    public void registerStompEndpoints(StompEndpointRegistry registry) {
        registry.addEndpoint("/ws")
            .setAllowedOriginPatterns("*")
            .withSockJS();  // SockJS fallback for non-WS browsers
    }
}

// Controller for bidirectional messages
@Controller
public class ChatController {

    @Autowired
    private SimpMessagingTemplate messagingTemplate;
```

```
    // Client sends to /app/chat, broadcast to /topic/room/{id}
    @MessageMapping("/chat/{roomId}")
    public void sendMessage(@DestinationVariable String roomId,
                            @Payload ChatMessage message,
                            Principal user) {
        message.setSender(user.getName());
        message.setTimestamp(Instant.now());
        messagingTemplate.convertAndSend("/topic/room/" + roomId, message);
    }

    // Send to specific user: /user/{userId}/queue/notifications
    public void sendNotification(String userId, Notification notification) {
        messagingTemplate.convertAndSendToUser(userId, "/queue/notifications", notification);
    }
}

// SSE for simpler one-way streaming
@RestController
public class NotificationController {

    @GetMapping(value = "/notifications/stream", produces = MediaType.TEXT_EVENT_STREAM_VALUE)
    public SseEmitter stream(@RequestHeader("User-Id") String userId) {
        SseEmitter emitter = new SseEmitter(Long.MAX_VALUE);
        sseService.register(userId, emitter);
        emitter.onCompletion(() -> sseService.remove(userId));
        emitter.onTimeout(() -> sseService.remove(userId));
        return emitter;
    }
}
```

■ 27. STRANGLER PATTERN

■ Q: What is the Strangler Fig Pattern for legacy migration?

A: The Strangler Fig Pattern incrementally migrates a legacy monolith to microservices by routing requests to new services one feature at a time, until the monolith is "strangled" and can be decommissioned.

Name origin: Strangler fig tree grows around a host tree, eventually replacing it while the host dies.

Migration Steps:

1. Put a Facade/Proxy in front of the monolith
2. Identify a bounded context to extract (e.g., User Management)
3. Build new microservice for that context
4. Route that specific traffic to new service via proxy
5. Verify new service correct, remove from monolith
6. Repeat until monolith empty

Why safe: At every step, rollback is a config change in the proxy. No big-bang cutover.

```
// Spring Cloud Gateway as Strangler Facade
@Configuration
public class StranglerGatewayConfig {

    @Bean
    public RouteLocator routes(RouteLocatorBuilder builder) {
        return builder.routes()
            // MIGRATED: User service — route to new microservice
            .route("user-service", r -> r
                .path("/api/users/**")
                .filters(f -> f
                    .rewritePath("/api/users/(?<segment>.*)", "/users/${segment}")
                    .addRequestHeader("X-Migrated", "true"))
                .uri("lb://user-service"))

            // MIGRATED: Order service
            .route("order-service", r -> r
                .path("/api/orders/**")
                .uri("lb://order-service"))

            // NOT YET MIGRATED: Everything else → monolith
            .route("legacy-monolith", r -> r
                .path("/**")
                .filters(f -> f.addRequestHeader("X-Legacy", "true"))
                .uri("http://monolith:8080"))
            .build();
    }
}

// Feature flag to toggle between legacy and new implementation
@Service
public class UserServiceFacade {

    @Value("${feature.new-user-service.enabled:false}")
```

```
    private boolean useNewService;

    private final LegacyUserService legacyService;
    private final NewUserService newService;

    public User findById(Long id) {
        if (useNewService) {
            try {
                return newService.findById(id);
            } catch (Exception e) {
                log.error("New service failed, falling back to legacy", e);
                metrics.counter("strangler.fallback").increment();
                return legacyService.findById(id);
            }
        }
        return legacyService.findById(id);
    }
}
```

■ 28. BATCH PROCESSING

■ Q: When do you use batch processing vs stream processing?

A: Batch processes data in large, bounded chunks on a schedule. Stream processes data continuously as it arrives.

Use Batch when:

- Data can wait (nightly reports, billing, payroll)
- Processing requires full dataset (monthly aggregations)
- Throughput over latency (ETL pipelines)
- Resource efficiency (run during off-peak)

Use Stream when:

- Real-time decisions required (fraud detection)
- Low latency requirements (<1 second)
- Continuous data arriving (IoT, clickstream)

Batch frameworks: Spring Batch, Apache Spark, AWS Glue.

Stream frameworks: Apache Flink, Kafka Streams, Apache Spark Streaming.

```
// Spring Batch — production-grade batch job
@Configuration
@EnableBatchProcessing
public class OrderReportBatch {

    @Bean
    public Job orderReportJob(
            JobBuilderFactory jbf,
            StepBuilderFactory sbf,
            DataSource dataSource) {

        return jbf.get("orderReportJob")
            .incrementer(new RunIdIncrementer())
            .listener(new JobCompletionListener())
            .start(generateReportStep(sbf, dataSource))
            .next(emailReportStep(sbf))
            .build();
    }

    @Bean
    public Step generateReportStep(StepBuilderFactory sbf, DataSource ds) {
        return sbf.get("generateReport")
            .<OrderRecord, ReportLine>chunk(500)    // Process in chunks of 500
            .reader(orderItemReader(ds))
            .processor(orderToReportLineProcessor())
            .writer(reportLineWriter())
            .faultTolerant()
                .retryLimit(3)
                .retry(TransientDataAccessException.class)
                .skipLimit(10)
                .skip(MalformedDataException.class)
            .listener(new ChunkProcessingListener())
            .build();
```

```
    }

    @Bean
    @StepScope
    public JdbcCursorItemReader<OrderRecord> orderItemReader(DataSource ds) {
        return new JdbcCursorItemReaderBuilder<OrderRecord>()
            .name("orderReader")
            .dataSource(ds)
            .sql("""
                SELECT o.id, o.customer_id, o.amount, o.status, o.created_at,
                       c.name as customer_name, c.email
                FROM orders o JOIN customers c ON o.customer_id = c.id
                WHERE o.created_at BETWEEN :startDate AND :endDate
                """)
            .rowMapper(new BeanPropertyRowMapper<>(OrderRecord.class))
            .build();
    }

    @Bean
    public ItemProcessor<OrderRecord, ReportLine> orderToReportLineProcessor() {
        return order -> {
            if (order.getAmount().compareTo(BigDecimal.ZERO) <= 0) return null; // Skip
            return ReportLine.builder()
                .customerId(order.getCustomerId())
                .customerName(order.getCustomerName())
                .totalAmount(order.getAmount())
                .orderCount(1)
                .build();
        };
    }
}
```

■ 29. RELIABILITY

■ Q: What is Reliability and how do you measure it (SLI, SLO, SLA)?

A: Reliability is the probability a system performs its intended function over a period. Quantified via:

SLI (Service Level Indicator): Metric that measures service behavior.

Examples: request success rate, p99 latency, availability.

SLO (Service Level Objective): Target value for an SLI.

Examples: `success_rate >= 99.9%`, `p99_latency <= 500ms`.

SLA (Service Level Agreement): Business contract — consequence if SLO not met (refunds, credits).

Error Budget: `Error Budget = 1 - SLO`. If SLO = 99.9%, error budget = 0.1% = 43.8 min/month.

When error budget exhausted: freeze new deployments, focus on reliability.

Reliability vs Availability:

- Availability: System is up
- Reliability: System produces correct results consistently

```java
// SLO monitoring with Micrometer
@Component
public class SloMonitor {

    private final MeterRegistry registry;
    private final AtomicLong totalRequests = new AtomicLong(0);
    private final AtomicLong successfulRequests = new AtomicLong(0);

    @EventListener
    public void onRequest(RequestCompletedEvent event) {
        totalRequests.incrementAndGet();
        if (event.isSuccess()) successfulRequests.incrementAndGet();

        // Record success rate gauge
        double successRate = (double) successfulRequests.get() / totalRequests.get();
        registry.gauge("slo.success.rate", successRate);

        // Alert when success rate drops below SLO
        if (successRate < 0.999) {
            registry.counter("slo.breach", "type", "success_rate").increment();
            alertService.send(Alert.builder()
                .severity(Severity.CRITICAL)
                .message(String.format("Success rate %.2f%% < SLO 99.9%%", successRate * 100))
                .build());
        }
    }

    // Track error budget consumption
    @Scheduled(fixedDelay = 60_000)
    public void calculateErrorBudget() {
        double currentFailureRate = calculateFailureRateForMonth();
        double errorBudget = 0.001; // 0.1% for 99.9% SLO
        double budgetConsumed = currentFailureRate / errorBudget;
```

```
        registry.gauge("slo.error.budget.consumed.pct", budgetConsumed * 100);

        if (budgetConsumed > 0.9) { // 90% budget consumed
            alertService.send(Alert.builder()
                .severity(Severity.WARNING)
                .message("Error budget 90% consumed — consider deployment freeze")
                .build());
        }
    }
}
```

■ 30. LOAD BALANCING

■ Q: What are load balancing algorithms and when to use each?

A: Load balancing distributes incoming requests across multiple servers to maximize throughput and minimize response time.

Algorithms:

Algorithm	How	Best For
Round Robin	Rotate in order	Equal capacity servers, stateless
Weighted Round Robin	Rotate by weight	Mixed capacity servers
Least Connections	Route to fewest active connections	Long-lived connections
Least Response Time	Route to fastest server	Variable response times
IP Hash	Hash client IP	Session affinity (WebSocket)
Random	Random selection	Simple, low overhead
Resource Based	Route based on CPU/memory	Auto-scaling environments

Layer 4 (Transport) LB: Routes by IP/TCP — fast, no content inspection. (AWS NLB, HAProxy)

Layer 7 (Application) LB: Routes by HTTP headers, path, cookies — can do SSL termination, content-based routing. (AWS ALB, Nginx, Spring Cloud Gateway)

```
// Spring Cloud LoadBalancer (client-side LB)
@Configuration
public class LoadBalancerConfig {

    @Bean
    @LoadBalancerClient(name = "order-service")
    public ReactorLoadBalancer<ServiceInstance> roundRobinLB(
            Environment env, LoadBalancerClientFactory factory) {
        String name = env.getProperty(LoadBalancerClientFactory.PROPERTY_NAME);
        return new RoundRobinLoadBalancer(
            factory.getLazyProvider(name, ServiceInstanceListSupplier.class), name);
    }
}

// Custom load balancer: least connections + health-aware
@Component
public class LeastConnectionsLoadBalancer implements ReactorServiceInstanceLoadBalancer {

    private final ConcurrentHashMap<String, AtomicInteger> connectionCounts = new ConcurrentHashMap<>();

    @Override
    public Mono<Response<ServiceInstance>> choose(Request request) {
        return serviceInstanceListSupplier.get(request)
            .next()
            .map(instances -> {
                ServiceInstance best = instances.stream()
                    .filter(this::isHealthy)
```

```
                    .min(Comparator.comparingInt(i ->
                        connectionCounts.computeIfAbsent(
                            i.getInstanceId(), k -> new AtomicInteger(0)).get()))
                    .orElseThrow(() -> new IllegalStateException("No healthy instances"));
                connectionCounts.get(best.getInstanceId()).incrementAndGet();
                return new DefaultResponse(best);
            });
    }
}
```

■ 31. GRAPHQL

■ Q: When do you choose GraphQL over REST?

A: GraphQL is a query language for APIs where clients specify exactly what data they need.

GraphQL Advantages over REST:

- No over-fetching (get exactly the fields you need)
- No under-fetching (get nested resources in single request)
- Strongly typed schema = auto-generated documentation
- Single endpoint (`/graphql`)
- Introspection

GraphQL Disadvantages:

- Complex caching (POST body = no HTTP cache)
- N+1 problem on resolvers (solve with DataLoader)
- Over-permissive queries can cause performance issues
- Learning curve

When GraphQL wins: Mobile clients (bandwidth), complex data graphs, multiple client types with different field needs.

When REST wins: Public APIs, simple CRUD, team unfamiliar with GraphQL, caching requirements.

```
// Spring for GraphQL (Spring Boot 3)
// build.gradle: implementation 'org.springframework.boot:spring-boot-starter-graphql'

// schema.graphqls
/*
type Query {
    order(id: ID!): Order
    orders(filter: OrderFilter, page: Int = 0, size: Int = 20): OrderConnection!
}
type Mutation {
    createOrder(input: CreateOrderInput!): Order!
    cancelOrder(id: ID!): Order!
}
type Order {
    id: ID!
    status: OrderStatus!
    amount: Float!
    customer: Customer!
    items: [OrderItem!]!
    createdAt: String!
}
*/

@Controller
public class OrderGraphQLController {

    private final OrderService orderService;
    private final DataLoaderRegistry loaderRegistry;

    @QueryMapping
```

```
    public Order order(@Argument String id) {
        return orderService.findById(Long.parseLong(id));
    }

    @QueryMapping
    public Connection<Order> orders(
            @Argument OrderFilter filter,
            @Argument int page,
            @Argument int size) {
        Page<Order> orders = orderService.findAll(filter, PageRequest.of(page, size));
        return new DefaultConnection<>(orders.getContent(), new DefaultPageInfo(...));
    }

    @MutationMapping
    public Order createOrder(@Argument CreateOrderInput input) {
        return orderService.create(input);
    }

    // Batch load customers to prevent N+1 problem
    @SchemaMapping(typeName = "Order", field = "customer")
    public CompletableFuture<Customer> customer(Order order,
            DataLoader<Long, Customer> customerLoader) {
        return customerLoader.load(order.getCustomerId());
    }
}

// DataLoader — batches N customer IDs into 1 query
@Component
public class CustomerDataLoaderConfig {

    @Bean
    public DataLoaderRegistrar customerDataLoader(CustomerRepository repo) {
        return registry -> registry.register("customerLoader",
            DataLoaderFactory.newMappedDataLoader(
                (Set<Long> ids) -> CompletableFuture.supplyAsync(() ->
                    repo.findAllById(ids).stream()
                        .collect(Collectors.toMap(Customer::getId, c -> c))
                )
            )
        );
    }
}
```

■ 32. CAP THEOREM

■ Q: What is CAP Theorem and how does it guide database selection?

A: CAP Theorem (Brewer, 2000) states a distributed system can guarantee at most 2 of 3 properties simultaneously:

- **C (Consistency):** Every read returns the most recent write (or error)
- **A (Availability):** Every request gets a non-error response (not necessarily latest data)
- **P (Partition Tolerance):** System continues despite network partition

Critical insight: In a distributed system, network partitions will happen. Therefore P is not optional — you must choose between C and A when a partition occurs.

Real-world DB positioning:

Database	Type	Sacrifice
PostgreSQL, MySQL	CP	Availability during partition
Cassandra, CouchDB	AP	Consistency (eventual consistency)
MongoDB	CP (default)	Availability
DynamoDB	AP (default)	Consistency (tunable)
Zookeeper, etcd	CP	Availability

PACELC (extends CAP): Even without partition, there's a tradeoff between Latency and Consistency.

```
// Demonstrating consistency vs availability tradeoffs in Spring Boot

// CP choice: Strong consistency — wait for all replicas
@Repository
public class StronglyConsistentOrderRepo {

    public Order save(Order order) {
        // PostgreSQL with synchronous_commit=on
        // Waits for WAL to be written to primary + replica before acknowledging
        // Slower but guaranteed consistent
        return jpaRepo.saveAndFlush(order);
    }
}

// AP choice: Eventual consistency — prioritize availability
@Repository
public class EventuallyConsistentOrderRepo {

    private final CassandraTemplate cassandra;

    public Order save(Order order) {
        // Cassandra with ConsistencyLevel.ONE — writes to one replica, returns
        // Fast but replica may lag (eventual consistency)
        WriteOptions opts = WriteOptions.builder()
            .consistencyLevel(ConsistencyLevel.ONE)  // Fast
            .build();
        cassandra.insert(order, opts);
        return order;
```

```
    }

    public Optional<Order> findById(UUID id) {
        // Read with QUORUM: must read from majority of replicas → stronger consistency
        // But slower than LOCAL_ONE
        ReadOptions opts = ReadOptions.builder()
            .consistencyLevel(ConsistencyLevel.QUORUM)
            .build();
        return Optional.ofNullable(cassandra.selectOneById(id, Order.class, opts));
    }
}
```

■ 33. LEADER ELECTION

■ Q: What is Leader Election and why do distributed systems need it?

A: Leader Election selects one node among many to be the single coordinator for activities that must only happen once (scheduled jobs, shard assignment, sequencing).

Why needed: In a cluster of N replicas, certain operations must execute exactly once: sending a daily digest email, compacting Kafka logs, assigning partitions to consumers.

Algorithms:

- **Bully Algorithm:** Highest-ID node wins. Simple but chatty.
- **Ring Algorithm:** Token passed around ring.
- **Raft/ZooKeeper:** Production-grade, handles failures, re-election.

Tools: ZooKeeper (Curator framework), etcd, Redis (Redisson), PostgreSQL advisory locks.

```
// Leader election with Spring Integration (ZooKeeper)
@Configuration
@EnableIntegration
public class LeaderElectionConfig {

    @Bean
    public LeaderInitiator leaderInitiator(ZookeeperClient client) {
        return new LeaderInitiatorFactoryBean()
            .setClient(client)
            .setPath("/leadership/order-processor")
            .setRole("order-processor-leader")
            .getObject();
    }
}

@Component
public class LeaderAwareScheduler {

    @Autowired
    private LeaderElectionConfiguration leaderConfig;

    @Scheduled(cron = "0 0 2 * * *")  // 2 AM daily
    public void runNightlyProcessing() {
        if (!leaderConfig.isLeader()) {
            log.debug("Not leader, skipping nightly processing");
            return;
        }
        log.info("Leader: running nightly processing");
        nightlyProcessingService.run();
    }

    @EventListener(OnGrantedEvent.class)
    public void onLeadershipGranted(OnGrantedEvent event) {
        log.info("Leadership GRANTED for role: {}", event.getRole());
        // Initialize leader-specific resources
    }

```

```
    @EventListener(OnRevokedEvent.class)
    public void onLeadershipRevoked(OnRevokedEvent event) {
        log.info("Leadership REVOKED for role: {}", event.getRole());
        // Clean up leader-specific resources
    }
}

// PostgreSQL advisory lock for leader election (no extra infra needed)
@Service
public class DbLeaderElection {

    private static final long LEADER_LOCK_KEY = 12345L;

    @Scheduled(fixedDelay = 10_000)
    @Transactional
    public void tryBecomeLeader() {
        Boolean acquired = jdbcTemplate.queryForObject(
            "SELECT pg_try_advisory_xact_lock(?)", Boolean.class, LEADER_LOCK_KEY);

        if (Boolean.TRUE.equals(acquired)) {
            log.debug("Acquired leadership, running tasks");
            leaderTask.execute();
            // Lock auto-released at transaction end
        }
    }
}
```

■ 34. DENORMALIZATION

■ Q: When should you denormalize your database schema?

A: Denormalization intentionally introduces redundancy (violating normal forms) to improve read performance by pre-joining data or pre-computing aggregates.

Normalization vs Denormalization:

- **Normalized:** Low redundancy, easy writes, complex reads (many JOINs)
- **Denormalized:** Some redundancy, complex writes (update multiple places), fast reads

When to Denormalize:

- Read-heavy workloads where JOIN cost dominates
- Reporting tables / data warehouses (designed for reads)
- Caching computed values (aggregate counts, sums)
- High-cardinality joins that are expensive
- When query latency SLO cannot be met even with indexes

When NOT to Denormalize:

- Write-heavy workloads (writes become expensive — must update all copies)
- Data that changes frequently (consistency maintenance costly)
- When indexes can solve the problem

```
// Normalized: Order + Customer join required for every query
@Entity
public class Order {
    private Long customerId;  // FK only
    // Must JOIN customers to get customer name/email
}

// Denormalized: customer info embedded in order
@Entity
public class OrderDenormalized {
    private Long customerId;
    // Denormalized — copied from customer at order creation time
    private String customerName;   // Redundant!
    private String customerEmail;  // Redundant!
    private String customerTier;   // Redundant!
    // Trade: slightly stale customer info OK, but no JOIN needed
}

// Materialized aggregate: pre-computed for dashboard
@Entity
@Table(name = "customer_stats")
public class CustomerStats {
    @Id
    private String customerId;
    private int totalOrders;         // Pre-computed
    private BigDecimal totalRevenue; // Pre-computed
    private Instant lastOrderDate;   // Pre-computed
    private Instant updatedAt;
}

```

```java
// Update aggregate on each order event
@Service
public class CustomerStatsUpdater {

    @EventListener
    @Transactional
    public void onOrderCompleted(OrderCompletedEvent event) {
        customerStatsRepo.upsertStats(
            event.getCustomerId(),
            event.getAmount(),
            event.getOrderDate()
        );
    }
}

// Dashboard query: O(1) instead of O(n) with GROUP BY
@Query("SELECT cs FROM CustomerStats cs WHERE cs.customerId = :id")
Optional<CustomerStats> getCustomerDashboard(@Param("id") String customerId);
```

■ 35. PUB/SUB

■ Q: What is Pub/Sub and how does it differ from a queue?

A: In Pub/Sub, publishers emit messages to topics without knowing who will receive them. All subscribers to a topic receive every message. This is broadcast/fan-out semantics.

Queue vs Pub/Sub:

- **Queue:** Message consumed by ONE worker (task distribution)
- **Pub/Sub:** Message received by ALL subscribers (event notification)

Kafka as Pub/Sub:

- Same message delivered to all consumer groups
- Within a consumer group = queue semantics (only one instance processes each message)

When Pub/Sub is the right choice:

- Order placed → notify: payment service, inventory service, notification service, analytics service — all simultaneously
- Domain events that multiple bounded contexts care about
- Building an event-driven architecture

```
// Google Pub/Sub with Spring Boot
@Configuration
public class PubSubConfig {

    @Bean
    public Publisher orderPublisher(TopicAdminClient topicAdmin) throws IOException {
        TopicName topic = TopicName.of("my-project", "order-events");
        return Publisher.newBuilder(topic)
            .setBatchingSettings(BatchingSettings.newBuilder()
                .setElementCountThreshold(100L)
                .setDelayThreshold(Duration.ofMillis(10))
                .build())
            .build();
    }
}

@Service
public class OrderEventPublisher {

    private final Publisher publisher;

    public void publishOrderPlaced(Order order) {
        OrderEvent event = OrderEvent.of(order, EventType.ORDER_PLACED);
        ByteString data = ByteString.copyFromUtf8(jsonMapper.writeValueAsString(event));

        PubsubMessage message = PubsubMessage.newBuilder()
            .setData(data)
            .putAttributes("eventType", "ORDER_PLACED")
            .putAttributes("customerId", order.getCustomerId())
            .build();

        ApiFuture<String> future = publisher.publish(message);
        ApiFutures.addCallback(future, new ApiFutureCallback<>() {
```

```
            public void onSuccess(String messageId) {
                log.debug("Published: {}", messageId);
            }
            public void onFailure(Throwable t) {
                log.error("Publish failed", t);
                // Use outbox pattern for guaranteed delivery
            }
        }, executor);
    }
}

// Multiple independent subscribers receive same event
@Component
public class InventorySubscriber {
    @GcpPubSubListener("order-events-inventory-sub")
    public void onOrderPlaced(OrderEvent event, BasicAcknowledgeablePubsubMessage msg) {
        inventoryService.reserve(event);
        msg.ack();
    }
}

@Component
public class NotificationSubscriber {
    @GcpPubSubListener("order-events-notification-sub")
    public void onOrderPlaced(OrderEvent event, BasicAcknowledgeablePubsubMessage msg) {
        emailService.sendConfirmation(event);
        msg.ack();
    }
}
```

■ 36. TIMEOUT

■ Q: How do you implement timeouts at every layer of a distributed system?

A: Timeouts prevent indefinite blocking when downstream services are slow or unresponsive. Without timeouts, threads pile up waiting → thread pool exhaustion → cascade failure.

Timeout Layers:

1. **Client connect timeout:** Max time to establish TCP connection
2. **Client read timeout:** Max time to wait for response data
3. **Server request timeout:** Max time to process a request
4. **DB query timeout:** Max time for SQL query
5. **Async operation timeout:** Max time for CompletableFuture

Setting values: Start with P99.9 of normal operation + buffer. Too short = false timeouts. Too long = slow cascade failure.

```
// All timeout levels in Spring Boot

// 1. RestTemplate / WebClient timeouts
@Bean
public WebClient webClient() {
    HttpClient httpClient = HttpClient.create()
        .option(ChannelOption.CONNECT_TIMEOUT_MILLIS, 3_000)    // TCP connect: 3s
        .responseTimeout(Duration.ofSeconds(10))                 // Response: 10s
        .doOnConnected(conn -> conn
            .addHandlerLast(new ReadTimeoutHandler(10))
            .addHandlerLast(new WriteTimeoutHandler(10)));

    return WebClient.builder()
        .clientConnector(new ReactorClientHttpConnector(httpClient))
        .build();
}

// 2. Global request timeout via filter
@Component
public class RequestTimeoutFilter implements Filter {
    private static final Duration REQUEST_TIMEOUT = Duration.ofSeconds(30);

    @Override
    public void doFilter(ServletRequest req, ServletResponse res, FilterChain chain)
            throws IOException, ServletException {
        HttpServletResponse httpRes = (HttpServletResponse) res;
        ScheduledFuture<?> timeout = scheduler.schedule(() -> {
            if (!httpRes.isCommitted()) {
                httpRes.setStatus(503);
                try { httpRes.getWriter().write("Request timeout"); } catch (Exception ignored) {}
            }
        }, REQUEST_TIMEOUT.toMillis(), TimeUnit.MILLISECONDS);
        try {
            chain.doFilter(req, res);
        } finally {
            timeout.cancel(false);
```

```
        }
    }
}

// 3. JPA query timeout
@QueryHints(@QueryHint(name = QueryHints.SPEC_HINT_TIMEOUT, value = "5000")) // 5 sec
@Query("SELECT o FROM Order o WHERE o.customerId = :id")
List<Order> findByCustomer(@Param("id") String id);

// 4. CompletableFuture timeout
CompletableFuture<PaymentResult> future = paymentService.processAsync(request);
PaymentResult result = future.orTimeout(5, TimeUnit.SECONDS)
    .exceptionally(ex -> {
        if (ex instanceof TimeoutException) {
            return PaymentResult.timeout(request.getOrderId());
        }
        return PaymentResult.failed(ex.getMessage());
    })
    .get();

// 5. Resilience4j TimeLimiter
@TimeLimiter(name = "payment", fallbackMethod = "timeoutFallback")
public CompletableFuture<PaymentResult> processPayment(PaymentRequest req) {
    return CompletableFuture.supplyAsync(() -> gateway.charge(req));
}
```

■ 37. FEATURE FLAGS

■ Q: How do feature flags enable safe deployments and experimentation?

A: Feature flags (feature toggles) allow enabling/disabling features at runtime without code deployment. Decouple deployment from release.

Types:

- **Release Toggles:** Hide unfinished features until ready
- **Experiment Toggles:** A/B testing (show feature to X% of users)
- **Ops Toggles:** Kill switch for problematic features
- **Permission Toggles:** Show premium features to paid users only

Benefits: Deploy unfinished code safely (dark launch), instant rollback (flip flag), gradual rollout, targeted testing.

```
// Spring Boot with Unleash (open-source feature flag service)
@Configuration
public class FeatureFlagConfig {

    @Bean
    public Unleash unleash(@Value("${unleash.url}") String url,
                           @Value("${unleash.apiKey}") String apiKey,
                           @Value("${spring.application.name}") String appName) {
        return new DefaultUnleash(
            new UnleashConfig.Builder()
                .appName(appName)
                .instanceId(InetAddress.getLocalHost().getHostName())
                .unleashAPI(url)
                .customHttpHeader("Authorization", apiKey)
                .fetchTogglesInterval(10)  // Refresh every 10 seconds
                .build()
        );
    }
}

@Service
public class CheckoutService {

    @Autowired private Unleash unleash;

    public CheckoutResult checkout(CheckoutRequest request, String userId) {
        UnleashContext context = UnleashContext.builder()
            .userId(userId)
            .addProperty("customerTier", getUserTier(userId))
            .addProperty("region", request.getRegion())
            .build();

        // Gradual rollout: enable for % of users
        if (unleash.isEnabled("new-checkout-flow", context)) {
            return newCheckoutFlow.process(request);
        }
        return legacyCheckoutFlow.process(request);
    }
```

```
    // Ops toggle: emergency kill switch
    public PaymentResult processPayment(PaymentRequest req) {
        if (!unleash.isEnabled("payment-service-enabled")) {
            throw new ServiceUnavailableException("Payment temporarily disabled");
        }
        return paymentService.process(req);
    }
}

// Simple in-memory feature flag (no external service)
@Component
public class SimpleFeatureFlags {

    @Value("${features.new-search:false}")
    private boolean newSearchEnabled;

    @Value("${features.new-search.rollout-pct:0}")
    private int rolloutPercentage;

    public boolean isEnabled(String feature, String userId) {
        return switch (feature) {
            case "new-search" -> newSearchEnabled &&
                (Math.abs(userId.hashCode()) % 100) < rolloutPercentage;
            default -> false;
        };
    }
}
```

■ 38. ALERTING

■ Q: What makes a good alerting strategy? How do you avoid alert fatigue?

A: Good alerting notifies the right people about actionable problems at the right time. Bad alerting = alert fatigue = ignored alerts = missed incidents.

Alerting Principles:

- Alert on symptoms, not causes (alert on high error rate, not high CPU — CPU may be fine)
- Every alert must be actionable (if no runbook exists, don't alert)
- Alert on SLO burn rate, not individual threshold breaches
- Use multiple severity levels (P1=wake up, P2=business hours, P3=ticket)
- Avoid flapping (require sustained violation, not single point)

Alert Fatigue Solution: Multi-window burn rate alerts (SLO-based alerting) — only page when error budget is burning fast enough to matter.

```
// Prometheus alerting rules (alert.rules.yml)
/*
groups:
- name: order-service
  rules:
  # GOOD: Alert on symptoms (high error rate)
  - alert: HighErrorRate
    expr: |
      sum(rate(http_requests_total{status=~"5.."}[5m])) /
      sum(rate(http_requests_total[5m])) > 0.01
    for: 2m
    labels:
      severity: critical
    annotations:
      summary: "Error rate {{ $value | humanizePercentage }}"
      runbook: "https://wiki/runbooks/high-error-rate"

  # GOOD: SLO burn rate alert (pages only when budget burning fast)
  - alert: ErrorBudgetBurnRate
    expr: |
      (sum(rate(http_requests_total{status=~"5.."}[1h])) /
       sum(rate(http_requests_total[1h]))) > (14.4 * 0.001)
    for: 5m
    labels:
      severity: critical
    annotations:
      summary: "Burning error budget 14x faster than target"

  # BAD: Don't alert on CPU (not necessarily actionable)
  # - alert: HighCPU
  #   expr: cpu_usage > 80   <-- leads to alert fatigue
*/

// Spring Boot alerting via PagerDuty/Opsgenie
@Service
public class AlertService {
```

```
    private final PagerDutyClient pagerDuty;

    @EventListener
    public void onSloBreached(SloBreachEvent event) {
        if (event.getSeverity() == Severity.P1) {
            pagerDuty.createIncident(IncidentRequest.builder()
                .title(event.getMessage())
                .severity("critical")
                .body(IncidentBody.builder()
                    .details(event.getDetails())
                    .runbookUrl("https://wiki/runbooks/" + event.getRunbook())
                    .build())
                .build());
        }
    }
}
```

■ 39. CACHE STAMPEDE

■ Q: What is Cache Stampede and how do you prevent it?

A: Cache Stampede (Thundering Herd) occurs when a popular cache entry expires and many concurrent requests all miss the cache simultaneously, all hitting the database at once, potentially overwhelming it.

Scenario: 10,000 req/sec hit the cache. Cache expires. All 10,000 requests → DB simultaneously → DB overwhelmed.

Prevention Strategies:

1. **Mutex/Locking:** Only one request fetches from DB; others wait for cache to repopulate.
2. **Probabilistic Early Recomputation (PER):** Randomly recompute before expiry to prevent simultaneous expiry.
3. **Background Refresh:** Async refresh before expiry. Serve stale while refreshing.
4. **Jitter on TTL:** `ttl = baseTTL + random(0, jitterRange)` — prevents all keys expiring simultaneously.

```java
// Strategy 1: Redis distributed lock prevents stampede
@Service
public class CacheStampedeProtectedService {

    private final StringRedisTemplate redis;
    private final ProductRepository repo;

    public Product getProduct(Long id) {
        String cacheKey = "product:" + id;
        String lockKey = "lock:product:" + id;

        // Check cache first
        String cached = redis.opsForValue().get(cacheKey);
        if (cached != null) return deserialize(cached, Product.class);

        // Try to acquire lock for DB fetch
        Boolean lockAcquired = redis.opsForValue()
            .setIfAbsent(lockKey, "1", Duration.ofSeconds(10));

        if (Boolean.TRUE.equals(lockAcquired)) {
            try {
                // Double-check after acquiring lock
                cached = redis.opsForValue().get(cacheKey);
                if (cached != null) return deserialize(cached, Product.class);

                // Fetch from DB and populate cache
                Product product = repo.findById(id).orElseThrow();
                // TTL with jitter: 30 min ± 5 min
                long ttlSeconds = 1800 + ThreadLocalRandom.current().nextLong(-300, 300);
                redis.opsForValue().set(cacheKey, serialize(product),
                    Duration.ofSeconds(ttlSeconds));
                return product;
            } finally {
                redis.delete(lockKey);
            }
        } else {
            // Another thread is fetching — wait briefly and retry
            LockSupport.parkNanos(TimeUnit.MILLISECONDS.toNanos(50));
            return getProduct(id); // Retry (cache should be populated now)
```

```
        }
    }
}

// Strategy 2: Stale-while-revalidate (serve stale, refresh async)
@Service
public class StaleWhileRevalidateCache {

    record CacheEntry<T>(T value, Instant expiresAt, Instant softExpiresAt) {
        boolean isHardExpired() { return Instant.now().isAfter(expiresAt); }
        boolean isSoftExpired() { return Instant.now().isAfter(softExpiresAt); }
    }

    public <T> T get(String key, Supplier<T> loader, Duration ttl) {
        CacheEntry<T> entry = localCache.get(key);

        if (entry == null || entry.isHardExpired()) {
            T fresh = loader.get();  // Must fetch synchronously
            cache(key, fresh, ttl);
            return fresh;
        }

        if (entry.isSoftExpired()) {
            // Serve stale, refresh in background
            CompletableFuture.runAsync(() -> {
                T fresh = loader.get();
                cache(key, fresh, ttl);
            });
        }
        return entry.value();  // Return stale immediately
    }
}
```

■ 40. WEBRTC

■ Q: What is WebRTC and when do you use it?

A: WebRTC (Web Real-Time Communication) is a browser API enabling direct peer-to-peer audio, video, and data transfer without a media server.

Architecture:

- **Signaling Server:** Exchange SDP (Session Description Protocol) and ICE candidates (WebSocket or REST)
- **STUN Server:** Discover public IP/port (UDP hole punching)
- **TURN Server:** Relay traffic when P2P fails (behind restrictive NAT/firewall)
- **P2P Connection:** Direct media flow after signaling

When to use WebRTC: Video/audio calls, screen sharing, file transfer, P2P gaming, live collaboration.

When NOT to use: Broadcasting to many viewers (use HLS/DASH), simple chat (WebSockets simpler), one-to-many (CDN better).

```
// Spring Boot signaling server for WebRTC
@Controller
public class WebRtcSignalingController {

    private final SimpMessagingTemplate messaging;

    // Client sends SDP offer/answer, ICE candidates through this signaling server
    @MessageMapping("/signal/{roomId}")
    public void signal(@DestinationVariable String roomId,
                       @Payload SignalMessage signal,
                       Principal sender) {

        signal.setFrom(sender.getName());

        // Forward signal to target peer in room
        if (signal.getTo() != null) {
            messaging.convertAndSendToUser(signal.getTo(), "/queue/signal", signal);
        } else {
            // Broadcast to room (for ICE candidates)
            messaging.convertAndSend("/topic/room/" + roomId + "/signal", signal);
        }
    }

    @MessageMapping("/join/{roomId}")
    public void joinRoom(@DestinationVariable String roomId, Principal user) {
        roomService.addParticipant(roomId, user.getName());
        // Notify others in room
        messaging.convertAndSend("/topic/room/" + roomId + "/participants",
            roomService.getParticipants(roomId));
    }
}

// SignalMessage record
public record SignalMessage(
    String type,        // "offer", "answer", "ice-candidate"
    String from,
```

```
    String to,
    Object sdp,         // SDP for offer/answer
    Object candidate    // ICE candidate
) {}
```

■ 41. LSM TREES (Log-Structured Merge Trees)

■ Q: What is an LSM Tree and which databases use it?

A: LSM Tree is a data structure optimized for high write throughput. Writes go to an in-memory buffer (MemTable), which is periodically flushed to immutable disk files (SSTables). Background compaction merges SSTables.

Write Path:

1. Write to WAL (Write-Ahead Log) for durability
2. Write to MemTable (in-memory sorted structure)
3. When MemTable full → flush to SSTable on disk
4. Background compaction merges SSTables

Read Path:

1. Check MemTable (latest data)
2. Check SSTables newest to oldest (Bloom filters to skip non-containing files)

Advantages: Sequential writes = very high write throughput (avoids random disk I/O).

Disadvantages: Read amplification (may check multiple SSTables), write amplification from compaction, space amplification.

Databases using LSM: RocksDB, Cassandra, HBase, LevelDB, InfluxDB, ScyllaDB.

Databases using B-Tree: PostgreSQL, MySQL InnoDB, SQLite.

When to use LSM-based DB: Write-heavy workloads (IoT, logging, metrics, event sourcing), time-series data.

```
// RocksDB via Spring Boot (embedded LSM tree DB)
@Configuration
public class RocksDbConfig {

    @Bean
    public RocksDB rocksDB(@Value("${rocksdb.path}") String path) throws RocksDBException {
        RocksDB.loadLibrary();
        Options options = new Options()
            .setCreateIfMissing(true)
            .setWriteBufferSize(64 * 1024 * 1024)     // 64MB MemTable
            .setMaxWriteBufferNumber(3)                 // 3 MemTables before flush
            .setTargetFileSizeBase(64 * 1024 * 1024)   // SSTable target size
            .setLevelCompactionDynamicLevelBytes(true)  // Dynamic level compaction
            .setCompressionType(CompressionType.LZ4_COMPRESSION);  // Fast compression
        return RocksDB.open(options, path);
    }
}

@Service
public class MetricsStore {
    private final RocksDB db;

    // High-throughput write — leverages LSM sequential write optimization
    public void recordMetric(String metricKey, long timestamp, double value) throws RocksDBException {
        // Key design: metricKey:timestamp ensures sorted order = efficient range scans
        String key = metricKey + ":" + String.format("%020d", timestamp);
        byte[] valueBytes = ByteBuffer.allocate(8).putDouble(value).array();
```

```
        db.put(key.getBytes(), valueBytes);
    }

    // Range scan: get all values for metric in time range
    public List<Double> getMetricsInRange(String metricKey, long from, long to)
            throws RocksDBException {
        List<Double> results = new ArrayList<>();
        String startKey = metricKey + ":" + String.format("%020d", from);
        String endKey = metricKey + ":" + String.format("%020d", to);

        try (RocksIterator iter = db.newIterator()) {
            iter.seek(startKey.getBytes());
            while (iter.isValid() && new String(iter.key()).compareTo(endKey) <= 0) {
                results.add(ByteBuffer.wrap(iter.value()).getDouble());
                iter.next();
            }
        }
        return results;
    }
}
```

■ 42. STREAM PROCESSING

■ Q: When do you use Stream Processing vs Batch Processing?

A: Stream processing handles data continuously as it arrives, event by event or in micro-batches.

Stream vs Batch:

Aspect	Stream	Batch
Latency	Milliseconds	Minutes to hours
Throughput	Lower	Higher
Data	Unbounded	Bounded
Complexity	Higher	Lower
Use case	Fraud detection, monitoring	ETL, reporting

Processing Guarantees:

- **At-most-once:** May lose data (fast, not for financial)
- **At-least-once:** May duplicate (most common, use idempotency)
- **Exactly-once:** No loss, no duplicate (slowest, use for financial)

Frameworks: Kafka Streams, Apache Flink, Apache Spark Structured Streaming.

```
// Kafka Streams — real-time order analytics
@Configuration
public class OrderStreamProcessor {

    @Bean
    public KStream<String, OrderEvent> orderStream(StreamsBuilder builder) {

        KStream<String, OrderEvent> orders = builder
            .stream("order-events", Consumed.with(Serdes.String(), orderEventSerde));

        // Windowed aggregation: count orders per customer per hour
        KTable<Windowed<String>, Long> orderCountPerHour = orders
            .filter((key, order) -> order.getStatus().equals("COMPLETED"))
            .groupBy((key, order) -> order.getCustomerId(),
                Grouped.with(Serdes.String(), orderEventSerde))
            .windowedBy(TimeWindows.ofSizeWithNoGrace(Duration.ofHours(1)))
            .count(Materialized.as("order-counts-per-hour"));

        // Fraud detection: >10 orders in 1 hour = suspicious
        orderCountPerHour.toStream()
            .filter((windowedKey, count) -> count != null && count > 10)
            .map((windowedKey, count) -> KeyValue.pair(
                windowedKey.key(),
                new FraudAlert(windowedKey.key(), count, windowedKey.window().startTime())))
            .to("fraud-alerts", Produced.with(Serdes.String(), fraudAlertSerde));

        // Real-time revenue aggregation
        KTable<String, BigDecimal> revenueByCustomer = orders
            .filter((k, v) -> v.getStatus().equals("COMPLETED"))
```

```
            .groupBy((k, v) -> v.getCustomerId(), Grouped.with(Serdes.String(), orderEventSerde))
            .aggregate(
                BigDecimal::new,
                (customerId, order, total) -> total.add(order.getAmount()),
                Materialized.as("customer-revenue")
            );

        return orders;
    }
}
```

■ 43. LATENCY

■ Q: What is latency and how do you reduce it at each layer?

A: Latency is the time between a request being sent and the response being received. Usually measured as p50, p95, p99 (percentiles, not averages — averages hide tail latency).

Latency Sources and Solutions:

Layer	Causes	Solutions
Network	Distance, routing	CDN, edge computing, co-location
Load Balancer	TLS handshake, queueing	Keep-alive, connection pool
App Server	Processing time, GC pauses	Async, caching, profiling
Database	Query time, lock waits	Indexing, connection pool, read replicas
Cache	Cold miss	Cache warming, pre-loading

Latency numbers to know:

- L1 cache: ~1ns
- L2 cache: ~4ns
- RAM: ~100ns
- SSD random read: ~100µs
- Network same datacenter: ~500µs
- Network cross-region: ~100ms
- Network Europe→Australia: ~200ms

```
// Async processing to reduce perceived latency
@Service
public class OrderService {

    // SYNC: User waits for ALL of these to complete (~2-3s)
    public OrderResult createOrderSync(CreateOrderRequest req) {
        Order order = saveOrder(req);               // 50ms
        paymentService.charge(req.getPayment());    // 500ms
        inventoryService.reserve(req.getItems());   // 200ms
        notificationService.sendEmail(order);       // 300ms
        analyticsService.recordEvent(order);        // 100ms
        return OrderResult.of(order);
    }

    // ASYNC: User gets response in ~50ms, rest happens in background
    public OrderResult createOrderAsync(CreateOrderRequest req) {
        Order order = saveOrder(req);  // 50ms — only this is synchronous

        // Fire-and-forget non-critical operations
        CompletableFuture.runAsync(() -> paymentService.charge(req.getPayment()), ioPool);
        CompletableFuture.runAsync(() -> inventoryService.reserve(req.getItems()), ioPool);
        CompletableFuture.runAsync(() -> notificationService.sendEmail(order), ioPool);
        CompletableFuture.runAsync(() -> analyticsService.recordEvent(order), ioPool);

        return OrderResult.of(order);  // Return immediately
```

```
    }
}

// Response time histogram — know your latency distribution
@Bean
public Timer orderTimer(MeterRegistry registry) {
    return Timer.builder("order.processing.time")
        .publishPercentiles(0.50, 0.90, 0.95, 0.99, 0.999) // p50→p99.9
        .publishPercentileHistogram()
        .serviceLevelObjectives(
            Duration.ofMillis(100),
            Duration.ofMillis(500),
            Duration.ofSeconds(1))
        .register(registry);
}
```

■ 44. CACHING

■ Q: What are the caching strategies and when to use each?

A: Caching stores frequently accessed data in fast storage (memory) to reduce latency and database load.

Caching Strategies:

1. **Cache-Aside (Lazy Loading):** App checks cache → miss → read DB → write to cache. Most common.
   - Pro: Only caches what's needed
   - Con: Cache miss penalty (read-your-writes inconsistency briefly)
2. **Read-Through:** Cache manages DB reads automatically.
   - Pro: Application simplicity
   - Con: First-read miss always goes to DB
3. **Write-Through:** Write to cache AND DB synchronously.
   - Pro: Always consistent
   - Con: Write penalty (must write to DB even for hot-path writes)
4. **Write-Behind (Write-Back):** Write to cache only; async flush to DB.
   - Pro: Fast writes
   - Con: Data loss risk if cache fails before flush
5. **Refresh-Ahead:** Pre-emptively refresh cache before expiry.
   - Pro: No cache miss on hot entries
   - Con: May cache stale data

```java
// Cache-aside with Spring Cache
@Service
public class ProductService {

    // Spring manages cache-aside automatically
    @Cacheable(value = "products", key = "#id",
               unless = "#result == null")
    public Product findById(Long id) {
        return productRepo.findById(id).orElse(null);
    }

    @CachePut(value = "products", key = "#product.id")
    public Product update(Product product) {
        return productRepo.save(product);
    }

    @CacheEvict(value = "products", key = "#id")
    public void delete(Long id) {
        productRepo.deleteById(id);
    }

    @CacheEvict(value = "products", allEntries = true)
    @Scheduled(fixedDelay = 3_600_000) // Evict all hourly
    public void evictAllCaches() {}
}

// Redis cache configuration with TTL per cache
@Configuration
```

```java
public class CacheConfig {

    @Bean
    public RedisCacheManager cacheManager(RedisConnectionFactory factory) {
        Map<String, RedisCacheConfiguration> configs = new HashMap<>();
        configs.put("products",
            RedisCacheConfiguration.defaultCacheConfig()
                .entryTtl(Duration.ofMinutes(30))
                .serializeValuesWith(RedisSerializationContext.SerializationPair
                    .fromSerializer(new GenericJackson2JsonRedisSerializer())));
        configs.put("sessions",
            RedisCacheConfiguration.defaultCacheConfig()
                .entryTtl(Duration.ofHours(24)));
        configs.put("rate-limits",
            RedisCacheConfiguration.defaultCacheConfig()
                .entryTtl(Duration.ofMinutes(1)));

        return RedisCacheManager.builder(factory)
            .withInitialCacheConfigurations(configs)
            .build();
    }
}

// Multi-level caching: L1 (local) + L2 (Redis)
@Service
public class MultiLevelCacheService {

    private final Caffeine<Object, Object> l1Cache = Caffeine.newBuilder()
        .maximumSize(1000)
        .expireAfterWrite(1, TimeUnit.MINUTES)
        .recordStats()
        .build();

    private final RedisTemplate<String, Object> l2Cache;

    public Product getProduct(Long id) {
        // L1: in-process cache (nanoseconds)
        return (Product) l1Cache.get(id, key -> {
            // L1 miss → check L2 (Redis, sub-millisecond)
            Object l2 = l2Cache.opsForValue().get("product:" + key);
            if (l2 != null) return l2;
            // L2 miss → DB
            Product p = productRepo.findById((Long) key).orElse(null);
            if (p != null) l2Cache.opsForValue().set("product:" + key, p, Duration.ofMinutes(30));
            return p;
        });
    }
}
```

■ 45. gRPC

■ Q: When do you choose gRPC over REST?

A: gRPC is a high-performance RPC framework using Protocol Buffers (binary serialization) over HTTP/2.

gRPC Advantages over REST:

- ~7x faster serialization (Protobuf binary vs JSON text)
- HTTP/2: multiplexing, header compression, full-duplex streaming
- Strongly typed contract (`.proto` file = documentation + validation)
- Bi-directional streaming
- Code generation for all languages

gRPC Disadvantages:

- Not browser-native (need gRPC-Web proxy)
- Less human-readable than JSON
- Harder to debug (binary protocol)
- Ecosystem less mature than REST

When to use gRPC: Internal microservice communication (not user-facing), polyglot services, real-time streaming (bidirectional), high-performance requirements.

```
// order.proto
syntax = "proto3";
package order;
option java_package = "com.example.order.grpc";

service OrderService {
  rpc GetOrder (GetOrderRequest) returns (OrderResponse);
  rpc CreateOrder (CreateOrderRequest) returns (OrderResponse);
  rpc StreamOrderUpdates (GetOrderRequest) returns (stream OrderEvent); // Server streaming
  rpc BatchCreateOrders (stream CreateOrderRequest) returns (BatchResponse); // Client streaming
}

message GetOrderRequest { string order_id = 1; }
message CreateOrderRequest {
  string customer_id = 1;
  repeated OrderItem items = 2;
}
message OrderItem {
  string product_id = 1;
  int32 quantity = 2;
  double price = 3;
}
message OrderResponse {
  string id = 1;
  string status = 2;
  double total_amount = 3;
  int64 created_at = 4;
}

```

```java
// gRPC Server implementation
@GrpcService
public class OrderGrpcService extends OrderServiceGrpc.OrderServiceImplBase {

    @Override
    public void getOrder(GetOrderRequest request,
                         StreamObserver<OrderResponse> responseObserver) {
        try {
            Order order = orderService.findById(request.getOrderId());
            responseObserver.onNext(toProto(order));
            responseObserver.onCompleted();
        } catch (OrderNotFoundException e) {
            responseObserver.onError(Status.NOT_FOUND
                .withDescription("Order not found: " + request.getOrderId())
                .asRuntimeException());
        }
    }

    @Override
    public void streamOrderUpdates(GetOrderRequest request,
                                   StreamObserver<OrderEvent> responseObserver) {
        // Server streaming: send updates as they happen
        String orderId = request.getOrderId();
        orderEventStream.subscribe(orderId, event -> {
            responseObserver.onNext(toProto(event));
            if (event.isFinal()) responseObserver.onCompleted();
        });
    }
}

// gRPC Client with interceptor for auth + tracing
@Configuration
public class GrpcClientConfig {

    @Bean
    public OrderServiceGrpc.OrderServiceBlockingStub orderServiceStub() {
        ManagedChannel channel = ManagedChannelBuilder
            .forAddress("order-service", 9090)
            .usePlaintext()
            .intercept(new AuthInterceptor(), new TracingInterceptor())
            .build();
        return OrderServiceGrpc.newBlockingStub(channel)
            .withDeadlineAfter(5, TimeUnit.SECONDS);
    }
}
```

■ 46. CONSISTENCY MODELS

■ Q: What are the different consistency models and when does each apply?

A: Consistency models define the rules about when and how updates become visible to readers in a distributed system.

Consistency Spectrum (strongest to weakest):

1. **Strict Consistency:** Every read sees the most recent write across all nodes. Impossible in distributed systems (would require instantaneous propagation).
2. **Linearizability (Strong Consistency):** Operations appear atomic and in real-time order. Used by: etcd, ZooKeeper, CockroachDB (Serializability).
3. **Sequential Consistency:** All processes see operations in the same order (not necessarily real-time order). Used by: traditional databases.
4. **Causal Consistency:** Causally related operations appear in order. If A causes B, everyone sees A before B. Used by: MongoDB (causal sessions), some distributed databases.
5. **Eventual Consistency:** Given no new updates, all replicas will eventually converge. Used by: Cassandra, DynamoDB, DNS.
6. **Read Your Own Writes:** You always see your own writes, others may not. Common in web apps.

```
// Demonstrating consistency choices in Spring Boot

// Linearizable read: use primary replica (always consistent, slower)
@Repository
public class StronglyConsistentRepo {

    @Transactional(isolation = Isolation.SERIALIZABLE)
    public Account transferFunds(Long fromId, Long toId, BigDecimal amount) {
        // Serializable isolation = linearizable within a transaction
        Account from = accountRepo.findByIdWithLock(fromId);  // SELECT FOR UPDATE
        Account to   = accountRepo.findByIdWithLock(toId);
        from.debit(amount);
        to.credit(amount);
        accountRepo.saveAll(List.of(from, to));
        return from;
    }
}

// Eventual consistency: use replica for reads (faster, may be slightly stale)
@Service
public class ProductQueryService {

    @Autowired @Qualifier("readReplica")
    private ProductRepository replicaRepo;

    // Acceptable: product list may be 1-2 seconds stale
    public List<Product> getProductCatalog() {
        return replicaRepo.findAllActive();
    }
}

// Read-Your-Writes: route user's own requests to primary after write
@Service
public class SessionAwareProductService {
```

```
    public Product createProduct(CreateProductRequest req, String sessionId) {
        Product saved = primaryRepo.save(Product.from(req));
        // Mark session as "wrote" — route to primary for next X seconds
        sessionRoutingService.markPrimaryRequired(sessionId, Duration.ofSeconds(30));
        return saved;
    }

    public Product getProduct(Long id, String sessionId) {
        if (sessionRoutingService.isPrimaryRequired(sessionId)) {
            return primaryRepo.findById(id).orElseThrow();
        }
        return replicaRepo.findById(id).orElseThrow();
    }
}

// Cassandra: tunable consistency
public Order findOrderConsistently(UUID orderId) {
    // QUORUM = read from majority of replicas → strongly consistent
    return cassandraTemplate.selectOneById(orderId, Order.class,
        ReadOptions.builder().consistencyLevel(ConsistencyLevel.QUORUM).build());
}
```

■ 47. SECRETS MANAGEMENT

■ Q: How do you manage secrets securely in a distributed system?

A: Secrets (API keys, DB passwords, TLS certs) must never be hardcoded, stored in code/config files, or in environment variables in plaintext.

Bad Practices:

- Hardcoded in source code (leaked in git history)
- In `application.properties` committed to repo
- As plain-text env vars (visible in process list, docker inspect)

Good Practices:

- HashiCorp Vault / AWS Secrets Manager / GCP Secret Manager
- Kubernetes Secrets (encrypted at rest with KMS)
- Short-lived credentials with rotation
- Audit trail of who accessed what secret

```
// Spring Boot with HashiCorp Vault
// build.gradle: implementation 'org.springframework.cloud:spring-cloud-starter-vault-config'

// bootstrap.yml (loaded before application.yml)
/*
spring:
  cloud:
    vault:
      host: vault.internal
      port: 8200
      scheme: https
      authentication: kubernetes   # Uses K8s service account token
      kubernetes:
        role: order-service
      kv:
        enabled: true
        backend: secret
        default-context: order-service
*/

// Secrets auto-injected from Vault
@Configuration
public class DatabaseConfig {

    // These values come from Vault, not application.yml
    @Value("${spring.datasource.password}")  // Injected from Vault
    private String dbPassword;

    @Value("${api.stripe.secret-key}")       // Injected from Vault
    private String stripeKey;
}

// AWS Secrets Manager via Spring Cloud AWS
@Service
public class AwsSecretsService {
```

```
    @Autowired
    private SecretsManagerClient secretsManager;

    @Cacheable(value = "secrets", key = "#secretName")
    public String getSecret(String secretName) {
        GetSecretValueRequest req = GetSecretValueRequest.builder()
            .secretId(secretName).build();
        return secretsManager.getSecretValue(req).secretString();
    }
}

// Secret rotation without restart (dynamic credentials)
@Component
public class DynamicCredentialProvider {

    @Autowired private VaultTemplate vault;

    @Scheduled(fixedDelay = 3600_000) // Check every hour
    public void refreshDbCredentials() {
        VaultResponse response = vault.read("database/creds/order-service");
        String username = (String) response.getData().get("username");
        String password = (String) response.getData().get("password");
        // Update HikariCP pool with new credentials without restart
        hikariDataSource.setUsername(username);
        hikariDataSource.setPassword(password);
        hikariDataSource.getHikariPoolMXBean().softEvictConnections();
    }
}
```

■ 48. ACID

■ Q: What does ACID mean and how do you use transactions in Spring Boot?

A: ACID defines transaction properties that guarantee database reliability:

- **Atomicity:** Transaction is all-or-nothing. Either all operations succeed or all are rolled back.
- **Consistency:** Transaction brings DB from one valid state to another. All constraints maintained.
- **Isolation:** Concurrent transactions don't see each other's intermediate states.
- **Durability:** Committed transactions persist even after system crash (WAL/journal).

Isolation Levels (weakest to strongest):

Level	Prevents	Allows
Read Uncommitted	—	Dirty read, non-repeatable, phantom
Read Committed (default)	Dirty read	Non-repeatable, phantom
Repeatable Read	Dirty + Non-repeatable	Phantom
Serializable	All anomalies	Nothing — slowest

```java
// Spring @Transactional
@Service
public class FundsTransferService {

    // SERIALIZABLE: for financial transactions
    @Transactional(isolation = Isolation.SERIALIZABLE, timeout = 30)
    public TransferResult transfer(Long fromId, Long toId, BigDecimal amount) {
        // Both reads get locked — no phantom reads, no non-repeatable reads
        Account from = accountRepo.findByIdWithLock(fromId)
            .orElseThrow(() -> new AccountNotFoundException(fromId));
        Account to = accountRepo.findByIdWithLock(toId)
            .orElseThrow(() -> new AccountNotFoundException(toId));

        if (from.getBalance().compareTo(amount) < 0)
            throw new InsufficientFundsException(fromId, amount);

        from.debit(amount);
        to.credit(amount);

        accountRepo.save(from);
        accountRepo.save(to);
        auditRepo.save(TransferAudit.of(fromId, toId, amount));  // All atomic

        return TransferResult.success(from.getBalance());
        // If any exception thrown → entire transaction rolled back
    }

    // Savepoints: partial rollback within transaction
    @Transactional
    public OrderResult createOrderWithOptionalInsurance(CreateOrderRequest req) {
        Order order = orderRepo.save(Order.from(req));  // Must succeed

        try {
```

```
            // Try to add insurance — if fails, rollback only this, not whole order
            Object savepoint = TransactionAspectSupport.currentTransactionStatus()
                .createSavepoint();
            try {
                insuranceService.addCoverage(order);
            } catch (InsuranceException e) {
                TransactionAspectSupport.currentTransactionStatus()
                    .rollbackToSavepoint(savepoint);
                log.warn("Insurance failed, continuing without it: {}", e.getMessage());
            }
        } catch (NestedTransactionNotSupportedException e) {
            // DB doesn't support savepoints — skip insurance
        }

        return OrderResult.of(order);
    }
}

// Pessimistic vs Optimistic locking
// Pessimistic: SELECT FOR UPDATE — holds DB lock
@Lock(LockModeType.PESSIMISTIC_WRITE)
@Query("SELECT a FROM Account a WHERE a.id = :id")
Optional<Account> findByIdWithLock(@Param("id") Long id);

// Optimistic: Version field — retry on conflict (better for read-heavy)
@Entity
public class Product {
    @Version  // Auto-managed by JPA
    private Long version;
    private int stockQuantity;
}
// JPA throws OptimisticLockException if version changed since read
```

■ 49. SYNC vs ASYNC

■ Q: When do you choose synchronous vs asynchronous communication?

A:

- **Synchronous:** Caller blocks waiting for response. Simple, immediate consistency, tight coupling.
- **Asynchronous:** Caller sends request and continues. Loose coupling, resilient, eventual consistency.

Use Synchronous when:

- Immediate response required (user queries)
- Strong consistency needed (payment confirmation)
- Simple request-response pattern
- Failure in callee should immediately fail the caller

Use Asynchronous when:

- Long-running operations (video processing, report generation)
- Fan-out to multiple services
- Decoupling for resilience (callee down = messages queue up)
- Rate limiting / traffic shaping

Hybrid Pattern (Async Request-Reply): Client sends async request, gets back a job ID, polls for result or receives webhook callback.

```
// Async with Spring @Async
@Service
public class ReportService {

    // Returns immediately with Future — doesn't block caller
    @Async("reportExecutor")
    public CompletableFuture<Report> generateReport(ReportRequest req) {
        log.info("Generating report async on thread: {}", Thread.currentThread().getName());
        Report report = doExpensiveGeneration(req); // Takes 30 seconds
        return CompletableFuture.completedFuture(report);
    }
}

// Async thread pool configuration
@Configuration
@EnableAsync
public class AsyncConfig {

    @Bean(name = "reportExecutor")
    public Executor reportExecutor() {
        ThreadPoolTaskExecutor executor = new ThreadPoolTaskExecutor();
        executor.setCorePoolSize(5);
        executor.setMaxPoolSize(10);
        executor.setQueueCapacity(25);
        executor.setThreadNamePrefix("report-");
        executor.setRejectedExecutionHandler(new ThreadPoolExecutor.CallerRunsPolicy());
        executor.initialize();
        return executor;
    }
```

```
}

// Async Request-Reply pattern (webhook callback)
@RestController
public class AsyncReportController {

    @PostMapping("/reports")
    public ResponseEntity<JobStatusResponse> requestReport(
            @Valid @RequestBody ReportRequest req) {
        String jobId = UUID.randomUUID().toString();
        // Store job, process async
        jobRepo.save(Job.pending(jobId));
        reportService.processAsync(jobId, req);  // Fire and forget
        // Return immediately with job ID
        return ResponseEntity.accepted()
            .body(new JobStatusResponse(jobId, "PENDING",
                "/reports/status/" + jobId));
    }

    @GetMapping("/reports/status/{jobId}")
    public ResponseEntity<JobStatusResponse> getStatus(@PathVariable String jobId) {
        Job job = jobRepo.findById(jobId).orElseThrow();
        return ResponseEntity.ok(JobStatusResponse.from(job));
    }
}
```

■ 50. SERVICE DISCOVERY

■ Q: What is Service Discovery and what are the patterns?

A: Service Discovery allows services to find each other dynamically without hardcoded IPs/ports. Critical in dynamic environments where service instances scale up/down and IPs change.

Two Patterns:

1. **Client-Side Discovery:** Client queries registry directly, selects instance, calls it.
   • Example: Netflix Eureka + Ribbon
   • Pro: Client has full control of load balancing
   • Con: Client must implement LB logic per language
2. **Server-Side Discovery:** Client calls load balancer/API gateway, which queries registry.
   • Example: AWS ALB + ECS, Kubernetes (Service DNS)
   • Pro: Simple clients, LB separate concern
   • Con: Extra hop

Tools: Consul, Eureka, etcd, ZooKeeper, Kubernetes (built-in via kube-dns + Services).

```
// Client-side discovery with Spring Cloud Netflix Eureka
// build.gradle: implementation 'org.springframework.cloud:spring-cloud-starter-netflix-eureka-client'

// Producer: Register with Eureka
@SpringBootApplication
@EnableDiscoveryClient
public class OrderService {
    public static void main(String[] args) { SpringApplication.run(OrderService.class, args); }
}

// application.yml:
/*
eureka:
  client:
    serviceUrl:
      defaultZone: http://eureka-server:8761/eureka/
  instance:
    preferIpAddress: true
    healthCheckUrlPath: /actuator/health
    statusPageUrlPath: /actuator/info
    leaseRenewalIntervalInSeconds: 10
    leaseExpirationDurationInSeconds: 30
*/

// Consumer: Discover and call via load-balanced URL
@Configuration
public class ServiceConfig {

    @Bean
    @LoadBalanced  // Makes RestTemplate use Eureka for service resolution
    public RestTemplate restTemplate() {
        return new RestTemplate();
    }
}
```

```

@Service
public class CheckoutService {

    @Autowired @LoadBalanced
    private RestTemplate restTemplate;

    public PaymentResult processPayment(PaymentRequest req) {
        // "payment-service" resolved via Eureka to actual instance IP
        return restTemplate.postForObject(
            "http://payment-service/api/payments",
            req, PaymentResult.class);
    }
}

// Modern: Spring Cloud Gateway + Kubernetes (server-side)
// Kubernetes Service automatically load-balances via kube-proxy
// App calls: http://payment-service:8080/api/payments
// k8s resolves via DNS + routes via Service selector
```

■ 51. OBSERVABILITY

■ Q: How do you implement full observability (metrics + logs + traces) in Spring Boot?

A: Observability is the ability to understand what is happening inside a system based on external outputs. The three pillars are metrics, logs, and distributed traces.

Metrics: What is happening (numbers over time) — Prometheus + Grafana

Logs: What happened (events) — ELK / Loki + Grafana

Traces: How it happened (request flow) — Jaeger / Zipkin / AWS X-Ray

Correlation: A trace ID connects logs → metrics → traces for a single request across all services.

```
// Spring Boot 3 Observability with Micrometer Tracing
// build.gradle:
// implementation 'io.micrometer:micrometer-tracing-bridge-otel'
// implementation 'io.opentelemetry.instrumentation:opentelemetry-spring-boot-starter'

// application.yml
/*
management:
  tracing:
    sampling:
      probability: 1.0  # 100% sampling in dev; 10% in prod
  otlp:
    tracing:
      endpoint: http://jaeger:4318/v1/traces
*/

// Auto-instrumented by Spring Boot Actuator
@RestController
public class OrderController {

    private final ObservationRegistry observationRegistry;

    @GetMapping("/orders/{id}")
    public Order getOrder(@PathVariable Long id) {
        return Observation.createNotStarted("order.fetch", observationRegistry)
            .lowCardinalityKeyValue("operation", "findById")
            .highCardinalityKeyValue("orderId", id.toString())
            .observe(() -> orderService.findById(id));
        // This single call creates:
        // 1. Timer metric: "order.fetch" with duration
        // 2. Span in distributed trace (with parent span from incoming request)
        // 3. Log entries with trace_id + span_id automatically included
    }
}

// Structured logging with trace correlation
// logback-spring.xml:
/*
<appender name="JSON" class="ch.qos.logback.core.ConsoleAppender">
  <encoder class="net.logstash.logback.encoder.LogstashEncoder">
```

```
    <provider class="net.logstash.logback.composite.loggingevent.MdcJsonProvider"/>
  </encoder>
</appender>
*/
// Output: {"@timestamp":"...","level":"INFO","trace_id":"abc123","span_id":"def456",
//           "service":"order-service","message":"Order processed","orderId":"123"}

// Custom span for business operations
@Service
public class PaymentService {

    @Autowired private Tracer tracer;

    public PaymentResult processPayment(PaymentRequest req) {
        Span span = tracer.nextSpan().name("payment.process");
        try (Tracer.SpanInScope ws = tracer.withSpan(span.start())) {
            span.tag("payment.method", req.getMethod());
            span.tag("payment.amount", req.getAmount().toString());

            PaymentResult result = gateway.charge(req);
            span.tag("payment.status", result.getStatus());
            return result;
        } catch (Exception e) {
            span.error(e);
            throw e;
        } finally {
            span.end();
        }
    }
}
```

■ 52. FULL-TEXT SEARCH

■ Q: When do you use Elasticsearch vs PostgreSQL full-text search?

A: Full-text search finds documents by relevance based on text content, not exact matches.

PostgreSQL FTS: Built-in, good for moderate volumes, no extra infrastructure, integrates with ACID transactions.

Elasticsearch: Dedicated search engine. Best for: large datasets, complex queries, faceting, aggregations, real-time search.

Use PostgreSQL FTS: < 10 million docs, simple search requirements, want transactional consistency with writes.

Use Elasticsearch: > 10 million docs, complex relevance tuning, real-time analytics, log analysis.

```
// PostgreSQL full-text search
@Query(value = """
    SELECT * FROM products
    WHERE to_tsvector('english', name || ' ' || description)
          @@ plainto_tsquery('english', :query)
    ORDER BY ts_rank(
        to_tsvector('english', name || ' ' || description),
        plainto_tsquery('english', :query)
    ) DESC
    LIMIT :limit
    """, nativeQuery = true)
List<Product> fullTextSearch(@Param("query") String query, @Param("limit") int limit);

// Elasticsearch with Spring Data Elasticsearch
@Document(indexName = "products")
@Setting(settingPath = "elasticsearch/product-settings.json")
public class ProductDocument {
    @Id private String id;

    @Field(type = FieldType.Text, analyzer = "english")
    private String name;

    @Field(type = FieldType.Text, analyzer = "english")
    private String description;

    @Field(type = FieldType.Keyword)
    private String category;

    @Field(type = FieldType.Double)
    private Double price;

    @Field(type = FieldType.Integer)
    private Integer stockQuantity;
}

@Service
public class ProductSearchService {

    @Autowired private ElasticsearchOperations esOps;

```

```
    public SearchHits<ProductDocument> search(ProductSearchRequest req) {
        BoolQueryBuilder query = QueryBuilders.boolQuery()
            .must(QueryBuilders.multiMatchQuery(req.getQuery(), "name", "description")
                .fuzziness(Fuzziness.AUTO)
                .operator(Operator.AND))
            .filter(QueryBuilders.rangeQuery("price")
                .gte(req.getMinPrice())
                .lte(req.getMaxPrice()))
            .filter(QueryBuilders.termQuery("category", req.getCategory()));

        NativeSearchQuery searchQuery = new NativeSearchQueryBuilder()
            .withQuery(query)
            .withPageable(req.getPageable())
            .withHighlightFields(
                new HighlightBuilder.Field("name"),
                new HighlightBuilder.Field("description"))
            .withAggregations(
                AggregationBuilders.terms("by_category").field("category"),
                AggregationBuilders.range("by_price").field("price")
                    .addRange(0, 50).addRange(50, 200).addRange(200, Double.MAX_VALUE))
            .build();

        return esOps.search(searchQuery, ProductDocument.class);
    }
}
```

■ 53. CACHE WARMING

■ Q: What is Cache Warming and why is it important?

A: Cache Warming (pre-loading) populates the cache before it's needed in production, preventing a cold-start where all initial requests miss the cache and overwhelm the database.

When needed:

- After deployment (new pod starts with empty cache)
- After cache flush/eviction
- Before major traffic events (flash sale, marketing campaign)
- After cache server restart

Strategies:

1. **Eager warming:** Load all expected data at startup
2. **Lazy warming:** Cache fills naturally via traffic (cold start penalty)
3. **Scheduled warming:** Refresh cache before entries expire
4. **Event-driven warming:** Pre-populate based on predicted access patterns

```
// Eager cache warming on application startup
@Component
@Slf4j
public class CacheWarmupService implements ApplicationRunner {

    @Autowired private ProductService productService;
    @Autowired private CategoryService categoryService;
    @Autowired private CacheManager cacheManager;

    @Override
    public void run(ApplicationArguments args) {
        log.info("Starting cache warm-up...");
        Instant start = Instant.now();

        // Warm top 1000 most-viewed products
        CompletableFuture<Void> products = CompletableFuture.runAsync(() -> {
            List<Long> topProductIds = productAnalyticsService.getTopProductIds(1000);
            topProductIds.parallelStream().forEach(id -> {
                try { productService.findById(id); } catch (Exception e) {
                    log.warn("Failed to warm product {}", id, e);
                }
            });
            log.info("Warmed {} products", topProductIds.size());
        });

        // Warm all active category trees
        CompletableFuture<Void> categories = CompletableFuture.runAsync(() ->
            categoryService.findAllActive().forEach(c ->
                categoryService.getCategoryTree(c.getId())));

        CompletableFuture.allOf(products, categories).join();
        log.info("Cache warm-up completed in {}ms",
            Duration.between(start, Instant.now()).toMillis());
    }
```

```
}

// Scheduled cache refresh before entries expire
@Service
public class CacheRefreshScheduler {

    // Refresh hot products every 25 minutes (TTL is 30 min — avoid stampede)
    @Scheduled(fixedDelay = 25 * 60 * 1000)
    public void refreshHotProducts() {
        List<Long> hotIds = hotProductTracker.getHotProductIds(100);
        hotIds.forEach(id -> {
            Product fresh = productRepo.findById(id).orElse(null);
            if (fresh != null) {
                cacheManager.getCache("products").put(id, fresh);
            }
        });
        log.info("Refreshed {} hot products in cache", hotIds.size());
    }
}
```

■ 54. CQRS (Command Query Responsibility Segregation)

■ Q: What is CQRS and when does it offer the most benefit?

A: CQRS separates read operations (Queries) from write operations (Commands) into different models, potentially different services, and even different databases.

Why separate reads from writes:

- Read and write models have different optimization needs
- Reads are typically 10x-100x more frequent than writes
- Read model can be denormalized for query performance
- Write model can be normalized for consistency
- Scale reads and writes independently

CQRS Spectrum:

- **Basic CQRS:** Separate command/query interfaces, same DB
- **Segregated CQRS:** Separate read/write services + DBs (sync via events)
- **Event Sourced CQRS:** Write side = event store; read side = materialized views

When to use: Complex domain with many query patterns, high read/write ratio difference, need to scale reads independently, reporting requirements.

When NOT to use: Simple CRUD apps, small teams, no scalability requirements — adds significant complexity.

```
// Command side: normalized, ACID, handles business rules
@Service
@Transactional
public class OrderCommandService {

    public String createOrder(CreateOrderCommand cmd) {
        // Business validation
        Product product = productRepo.findById(cmd.getProductId())
            .orElseThrow(() -> new ProductNotFoundException(cmd.getProductId()));
        if (product.getStock() < cmd.getQuantity())
            throw new InsufficientStockException(cmd.getProductId());

        Order order = Order.create(cmd);
        orderRepo.save(order);

        // Publish event for read-side to consume
        eventPublisher.publish(new OrderCreatedEvent(order));
        return order.getId();
    }
}

// Query side: denormalized, optimized for reads, eventual consistency
@Service
@Transactional(readOnly = true)
public class OrderQueryService {

    // Read from denormalized read-model table
    public OrderSummaryView getOrderSummary(String orderId) {
        return orderSummaryViewRepo.findById(orderId)
```

```
            .orElseThrow(() -> new OrderNotFoundException(orderId));
    }

    // Dashboard query: pre-aggregated, no joins needed
    public CustomerDashboardView getCustomerDashboard(String customerId) {
        return customerDashboardRepo.findByCustomerId(customerId)
            .orElse(CustomerDashboardView.empty(customerId));
    }
}

// Event handler: updates read model when write side changes
@Component
@Transactional
public class OrderReadModelUpdater {

    @EventListener
    public void on(OrderCreatedEvent event) {
        // Build denormalized view for fast reads
        OrderSummaryView view = OrderSummaryView.builder()
            .orderId(event.getOrderId())
            .customerId(event.getCustomerId())
            .customerName(customerRepo.findName(event.getCustomerId()))
            .totalAmount(event.getAmount())
            .itemCount(event.getItems().size())
            .status(OrderStatus.PLACED)
            .createdAt(event.getOccurredAt())
            .build();
        orderSummaryViewRepo.save(view);

        // Update customer dashboard aggregate
        customerDashboardRepo.incrementOrderCount(event.getCustomerId(), event.getAmount());
    }

    @EventListener
    public void on(OrderShippedEvent event) {
        orderSummaryViewRepo.updateStatus(event.getOrderId(), OrderStatus.SHIPPED);
        orderSummaryViewRepo.setTrackingNumber(event.getOrderId(), event.getTrackingNumber());
    }
}
```

■ 55. B-TREES

■ Q: How does a B-Tree index work and why is it the default for most databases?

A: B-Tree (Balanced Tree) is a self-balancing tree data structure where all leaf nodes are at the same depth. Provides O(log n) for search, insert, and delete. Values are sorted, enabling efficient range queries.

Why B-Tree is the default:

- Supports equality (=) AND range queries (>, <, `BETWEEN`, `LIKE 'abc%'`)
- Supports ORDER BY (data already sorted)
- Stable performance: O(log n) regardless of data distribution
- Works well for both point queries and scans

B-Tree vs B+Tree (what DBs actually use):

- B+Tree: Data only in leaf nodes; all leaves linked in sorted order
- Enables efficient full-scans of sorted data
- Better cache performance (smaller internal nodes → more keys per page)

When B-Tree excels: Most database query patterns — range queries, sorted results, prefix matching.

When B-Tree loses: Hash (equality only, faster); GIN (full-text, arrays); BRIN (huge sequential tables).

```
// Understanding B-Tree through Spring Data index usage

// This index makes the following queries fast (all B-tree compatible):
@Index(name = "idx_order_customer_date", columnList = "customer_id, created_at")

// Query 1: Equality on customer_id (leftmost column) ■
orderRepo.findByCustomerId("cust-123");
// QUERY: WHERE customer_id = 'cust-123' → uses index

// Query 2: Equality + range ■
orderRepo.findByCustomerIdAndCreatedAtAfter("cust-123", yesterday);
// QUERY: WHERE customer_id = 'cust-123' AND created_at > ? → uses index fully

// Query 3: Range on first column only ■
orderRepo.findByCreatedAtBetween(startDate, endDate);
// QUERY: WHERE created_at BETWEEN ? AND ? → cannot use composite index!
// Need separate index on created_at alone

// Query 4: Sort using index columns → no extra sort step ■
Pageable p = PageRequest.of(0, 20, Sort.by("customerId", "createdAt"));
orderRepo.findAll(p);

// EXPLAIN ANALYZE output interpretation
/*
Index Scan using idx_order_customer_date on orders  (cost=0.43..8.45 rows=1 width=xxx)
  Index Cond: ((customer_id = 'cust-123') AND (created_at > '2024-01-01'))
  Actual time=0.123..0.456 ms, rows=15    ← Good! Using index

vs.

Seq Scan on orders  (cost=0.00..45000.00 rows=1000000 width=xxx)
  Filter: (status = 'ACTIVE')              ← Bad! Full table scan
```

```
  Rows Removed by Filter: 990000
*/

// Composite index column order matters:
// INDEX(a, b, c) can service: (a), (a,b), (a,b,c) but NOT (b) or (c) alone
// Rule: equality columns first, range column last
@Index(name = "idx_optimal", columnList = "status, customer_id, created_at")
// Serves: WHERE status=? AND customer_id=? ORDER BY created_at  → perfect
```

■ 56. ETL (Extract, Transform, Load)

■ Q: What is ETL and how do you implement it in Java/Spring?

A: ETL is a data integration process: Extract data from sources, Transform it to fit the target schema/format, Load into destination (data warehouse, analytics DB).

ETL vs ELT:

- ETL: Transform before loading. Traditional. Used when destination has limited compute.
- ELT: Load raw data, then transform in destination. Modern (Snowflake, BigQuery). Uses destination's compute power.

When to use ETL: Data warehouse population, migration between systems, data quality enforcement, aggregation pipelines.

Challenges: Large data volumes (streaming in chunks), error handling (skip/retry/fail), idempotency (re-runnable), monitoring, schema evolution.

```
// Spring Batch ETL Pipeline
@Configuration
@EnableBatchProcessing
public class CustomerEtlPipeline {

    @Bean
    public Job customerEtlJob(JobBuilderFactory jbf, StepBuilderFactory sbf) {
        return jbf.get("customerEtlJob")
            .incrementer(new RunIdIncrementer())
            .start(extractStep(sbf))
            .next(transformStep(sbf))
            .next(loadStep(sbf))
            .build();
    }

    // EXTRACT: Read from source CRM database
    @Bean
    public Step extractStep(StepBuilderFactory sbf) {
        return sbf.get("extract")
            .<SourceCustomer, SourceCustomer>chunk(500)
            .reader(sourceDbReader())
            .writer(stagingWriter())
            .faultTolerant().skipLimit(100)
            .skip(DataAccessException.class)
            .build();
    }

    // TRANSFORM: Clean, validate, enrich
    @Bean
    public Step transformStep(StepBuilderFactory sbf) {
        return sbf.get("transform")
            .<SourceCustomer, TargetCustomer>chunk(500)
            .reader(stagingReader())
            .processor(customerTransformer())
            .writer(transformedWriter())
            .build();
    }
```

```

    // LOAD: Write to data warehouse
    @Bean
    public Step loadStep(StepBuilderFactory sbf) {
        return sbf.get("load")
            .<TargetCustomer, TargetCustomer>chunk(1000)
            .reader(transformedReader())
            .writer(warehouseWriter())
            .build();
    }

    // Transformer: business rules, data cleansing
    @Bean
    public ItemProcessor<SourceCustomer, TargetCustomer> customerTransformer() {
        return source -> {
            if (source.getEmail() == null || !isValidEmail(source.getEmail())) {
                return null;  // Skip invalid records
            }
            return TargetCustomer.builder()
                .id(source.getId())
                .fullName(normalize(source.getFirstName() + " " + source.getLastName()))
                .email(source.getEmail().toLowerCase())
                .phone(normalizePhone(source.getPhone()))
                .segment(deriveSegment(source))       // Enrichment
                .lifetimeValue(calculateLTV(source))  // Computed field
                .etlTimestamp(Instant.now())
                .build();
        };
    }
}
```

■ 57. THROUGHPUT

■ Q: What is Throughput and how do you maximize it?

A: Throughput is the number of operations (requests, transactions, messages) a system processes per unit of time. Measured as RPS (requests/sec), TPS (transactions/sec), or MB/s.

Throughput vs Latency tradeoff: Batching increases throughput but increases latency per item. Optimizing one often degrades the other.

Little's Law: `L = λ × W`

Where L = avg items in system, λ = throughput (arrival rate), W = avg time in system.

If W (latency) doubles, either L doubles (queue grows) or λ halves (throughput drops).

How to increase throughput:

- Horizontal scaling (more instances)
- Async processing (don't block threads)
- Batching (process multiple items together)
- Connection pooling (reuse expensive connections)
- Caching (avoid redundant work)
- Efficient algorithms and data structures
- Database: read replicas, indexing, query optimization

```
// Throughput optimization with batching
@Service
public class HighThroughputOrderService {

    // BAD: Individual inserts — 1000 orders = 1000 DB round trips
    public void saveOrdersSlow(List<Order> orders) {
        orders.forEach(orderRepo::save);  // N round trips
    }

    // GOOD: Batch insert — 1000 orders = 1 DB round trip
    public void saveOrdersFast(List<Order> orders) {
        // JDBC batch insert
        jdbcTemplate.batchUpdate(
            "INSERT INTO orders (id, customer_id, amount, status) VALUES (?, ?, ?, ?)",
            orders,
            500,  // Batch size
            (ps, order) -> {
                ps.setString(1, order.getId().toString());
                ps.setString(2, order.getCustomerId());
                ps.setBigDecimal(3, order.getAmount());
                ps.setString(4, order.getStatus().name());
            }
        );
    }

    // Maximize throughput with async + virtual threads (Java 21)
    @Bean
    public Executor virtualThreadExecutor() {
        return Executors.newVirtualThreadPerTaskExecutor();
```

```
    }

    // Kafka producer batching for high throughput
    @Bean
    public ProducerFactory<String, Object> highThroughputProducerFactory() {
        Map<String, Object> config = new HashMap<>();
        config.put(ProducerConfig.BOOTSTRAP_SERVERS_CONFIG, "kafka:9092");
        config.put(ProducerConfig.LINGER_MS_CONFIG, 10);          // Wait 10ms to batch
        config.put(ProducerConfig.BATCH_SIZE_CONFIG, 65536);      // 64KB batch
        config.put(ProducerConfig.COMPRESSION_TYPE_CONFIG, "lz4"); // Compress batches
        config.put(ProducerConfig.BUFFER_MEMORY_CONFIG, 67108864); // 64MB buffer
        config.put(ProducerConfig.ACKS_CONFIG, "1");               // Leader ack only (not all)
        return new DefaultKafkaProducerFactory<>(config);
    }
}
```

■ 58. CACHE INVALIDATION

■ Q: What is cache invalidation and why is it considered one of the hardest problems?

A: Phil Karlton's famous quote: "There are only two hard things in Computer Science: cache invalidation and naming things."

Cache invalidation ensures cached data is removed or updated when the underlying data changes, preventing stale reads.

Invalidation Strategies:

1. **TTL (Time-To-Live):** Cache expires after fixed duration. Simple but stale during TTL window.
2. **Event-Driven Invalidation:** Invalidate on write event. Consistent but couples write path to cache.
3. **Write-Through:** Update cache on every write. Always fresh but slower writes.
4. **Cache-Aside Eviction:** Application evicts on write. Most common pattern.
5. **CDC (Change Data Capture):** DB triggers invalidation. Decoupled but complex.

The Inconsistency Window Problem: Between cache eviction and cache repopulation, concurrent reads may create multiple DB calls.

```
// Strategy 1: Event-driven cache invalidation
@Service
@Transactional
public class ProductService {

    @CachePut(value = "products", key = "#result.id")
    public Product update(Long id, UpdateProductRequest req) {
        Product product = productRepo.findById(id).orElseThrow();
        product.update(req);
        return productRepo.save(product);
    }

    @Caching(evict = {
        @CacheEvict(value = "products", key = "#id"),
        @CacheEvict(value = "product-lists", allEntries = true),  // Evict all list caches
        @CacheEvict(value = "category-products", key = "#product.categoryId")
    })
    public void delete(Long id) {
        productRepo.deleteById(id);
    }
}

// Strategy 2: CDC with Debezium → Kafka → Cache invalidation
@Component
public class CdcCacheInvalidator {

    @KafkaListener(topics = "dbserver.public.products")
    public void onProductChange(ProductChangeEvent event) {
        switch (event.getOp()) {
            case "u", "d" -> {
                // Invalidate by ID
                cache.evict("products", event.getBefore().getId());
                // Invalidate related collections
```

```
                cache.evict("category-products", event.getBefore().getCategoryId());
                log.info("Cache invalidated for product {}", event.getBefore().getId());
            }
            case "c" -> {
                // New product: invalidate list caches
                cache.evict("product-lists");
            }
        }
    }
}

// Strategy 3: Versioned cache keys (never need explicit invalidation)
@Service
public class VersionedCacheService {

    // Cache key includes a version — bump version to "invalidate" all entries
    public Product getProduct(Long id) {
        String version = redisTemplate.opsForValue().get("product-cache-version");
        String key = "product:v" + version + ":" + id;

        return Optional.ofNullable(redisTemplate.opsForValue().get(key))
            .map(v -> (Product) v)
            .orElseGet(() -> {
                Product p = productRepo.findById(id).orElseThrow();
                redisTemplate.opsForValue().set(key, p, Duration.ofHours(1));
                return p;
            });
    }

    // "Invalidate all" = bump version (O(1) instead of scanning all keys)
    public void invalidateAllProducts() {
        redisTemplate.opsForValue().increment("product-cache-version");
    }
}
```

■ 59. AUTHENTICATION

■ Q: How do you implement stateless JWT authentication in Spring Boot?

A: Authentication verifies WHO you are. Common methods: session cookies (stateful), JWT (stateless), OAuth2, API keys, mutual TLS.

JWT (JSON Web Token):

- Header.Payload.Signature
- Stateless: server doesn't store sessions — scales horizontally trivially
- Payload contains claims: userId, roles, expiry
- Signed with secret (HMAC) or key pair (RS256)

JWT Disadvantages:

- Cannot revoke before expiry (use short-lived access tokens + refresh tokens)
- Payload visible (don't store sensitive data)
- Token grows large with many claims

When to use JWT: Stateless microservices, mobile clients, cross-domain auth.

When to use Sessions: Traditional web apps where server storage is acceptable.

```
// Spring Security + JWT
@Configuration
@EnableWebSecurity
public class SecurityConfig {

    @Bean
    public SecurityFilterChain filterChain(HttpSecurity http,
                                           JwtAuthenticationFilter jwtFilter) throws Exception {
        return http
            .sessionManagement(s -> s.sessionCreationPolicy(STATELESS)) // No sessions
            .csrf(csrf -> csrf.disable())  // JWT doesn't need CSRF protection
            .authorizeHttpRequests(auth -> auth
                .requestMatchers("/api/auth/**", "/actuator/health").permitAll()
                .requestMatchers("/api/admin/**").hasRole("ADMIN")
                .anyRequest().authenticated())
            .addFilterBefore(jwtFilter, UsernamePasswordAuthenticationFilter.class)
            .build();
    }
}

// JWT filter — validates token on every request
@Component
@RequiredArgsConstructor
public class JwtAuthenticationFilter extends OncePerRequestFilter {

    private final JwtTokenService jwtService;

    @Override
    protected void doFilterInternal(HttpServletRequest request,
                                    HttpServletResponse response,
                                    FilterChain filterChain) throws ServletException, IOException {
        String authHeader = request.getHeader("Authorization");
```

```
        if (authHeader == null || !authHeader.startsWith("Bearer ")) {
            filterChain.doFilter(request, response);
            return;
        }

        String token = authHeader.substring(7);
        try {
            Claims claims = jwtService.validateAndParse(token);
            String userId = claims.getSubject();
            List<String> roles = claims.get("roles", List.class);

            UsernamePasswordAuthenticationToken auth = new UsernamePasswordAuthenticationToken(
                userId, null,
                roles.stream().map(SimpleGrantedAuthority::new).collect(toList()));
            auth.setDetails(new WebAuthenticationDetailsSource().buildDetails(request));
            SecurityContextHolder.getContext().setAuthentication(auth);
        } catch (JwtException e) {
            response.setStatus(HttpStatus.UNAUTHORIZED.value());
            response.getWriter().write("{\"error\":\"Invalid token\"}");
            return;
        }
        filterChain.doFilter(request, response);
    }
}

// JWT service
@Service
public class JwtTokenService {
    private final SecretKey key = Keys.hmacShaKeyFor(Decoders.BASE64.decode(secretBase64));
    private static final Duration ACCESS_TTL = Duration.ofMinutes(15);
    private static final Duration REFRESH_TTL = Duration.ofDays(30);

    public String generateAccessToken(User user) {
        return Jwts.builder()
            .setSubject(user.getId().toString())
            .claim("roles", user.getRoles())
            .claim("email", user.getEmail())
            .setIssuedAt(new Date())
            .setExpiration(Date.from(Instant.now().plus(ACCESS_TTL)))
            .signWith(key, SignatureAlgorithm.HS256)
            .compact();
    }

    public Claims validateAndParse(String token) {
        return Jwts.parserBuilder().setSigningKey(key).build()
            .parseClaimsJws(token).getBody();
    }
}
```

■ 60. REPLICATION

■ Q: What are the database replication modes and when to use each?

A: Replication copies data from primary to one or more replicas for high availability and read scaling.

Replication Modes:

Mode	Durability	Performance	Use Case
Synchronous	High (no data loss)	Slower writes	Financial, critical data
Asynchronous	Risk of data loss on failover	Fast writes	Analytics replicas
Semi-synchronous	Balance	Moderate	Most production systems

Replication Topologies:

- **Single Primary, Multiple Replicas:** Most common. Writes → primary; reads → replicas.
- **Multi-Primary:** All nodes accept writes. Complex conflict resolution.
- **Cascading Replication:** Replica replicates to another replica (reduces primary load).

Replication Lag: Async replicas can be seconds behind. Don't read your own writes from replica.

```
// Spring Boot read/write splitting with AbstractRoutingDataSource
public class ReadWriteRoutingDataSource extends AbstractRoutingDataSource {

    private static final ThreadLocal<DbType> contextHolder = new ThreadLocal<>();

    public enum DbType { PRIMARY, REPLICA }

    public static void usePrimary() { contextHolder.set(DbType.PRIMARY); }
    public static void useReplica() { contextHolder.set(DbType.REPLICA); }
    public static void clear() { contextHolder.remove(); }

    @Override
    protected Object determineCurrentLookupKey() {
        return contextHolder.get() != null ? contextHolder.get() : DbType.PRIMARY;
    }
}

// AOP: route @Transactional(readOnly=true) to replica automatically
@Aspect
@Component
public class ReadWriteDataSourceAspect {

    @Around("@annotation(transactional)")
    public Object routeDataSource(ProceedingJoinPoint pjp,
                                  Transactional transactional) throws Throwable {
        try {
            if (transactional.readOnly()) {
                ReadWriteRoutingDataSource.useReplica();
            } else {
                ReadWriteRoutingDataSource.usePrimary();
            }
            return pjp.proceed();
```

```
        } finally {
            ReadWriteRoutingDataSource.clear();
        }
    }
}

// Usage: @Transactional(readOnly=true) → replica, @Transactional → primary
@Service
public class ProductService {

    @Transactional(readOnly = true)  // → replica
    public List<Product> findAll() { return productRepo.findAll(); }

    @Transactional  // → primary
    public Product create(CreateProductRequest req) { return productRepo.save(Product.from(req)); }
}
```

■ 61. RBAC (Role-Based Access Control)

■ Q: How do you implement RBAC in Spring Boot?

A: RBAC assigns permissions to roles, and roles to users. Users get permissions through their roles.

RBAC vs ABAC:

- **RBAC:** Access based on role. Simple, coarse-grained. "Admins can edit all orders."
- **ABAC:** Access based on attributes. Fine-grained. "Managers can edit orders in their department created today."

When to use RBAC: Most enterprise apps, clear role boundaries, team-level access control.

When to use ABAC: Complex access policies, multi-tenant, compliance-heavy systems.

```
// Role and Permission entities
@Entity
public class Role {
    @Id
    private String name;  // ROLE_ADMIN, ROLE_MANAGER, ROLE_USER

    @ManyToMany(fetch = FetchType.EAGER)
    private Set<Permission> permissions;
}

@Entity
public class Permission {
    @Id
    private String name;  // ORDER_READ, ORDER_WRITE, USER_ADMIN, REPORT_VIEW
}

// Spring Security method-level security
@Configuration
@EnableMethodSecurity(prePostEnabled = true, securedEnabled = true)
public class MethodSecurityConfig {}

@RestController
@RequestMapping("/api/orders")
public class OrderController {

    @GetMapping("/{id}")
    @PreAuthorize("hasAnyRole('ADMIN','MANAGER','USER')")
    public Order getOrder(@PathVariable Long id) {
        return orderService.findById(id);
    }

    @DeleteMapping("/{id}")
    @PreAuthorize("hasRole('ADMIN')")
    public void deleteOrder(@PathVariable Long id) {
        orderService.delete(id);
    }

    // Fine-grained: user can only view their own orders unless admin
    @GetMapping("/my")
    @PreAuthorize("isAuthenticated()")
```

```
    public List<Order> getMyOrders() {
        String userId = SecurityContextHolder.getContext().getAuthentication().getName();
        return orderService.findByCustomerId(userId);
    }

    // SpEL with custom bean for complex rules
    @PutMapping("/{id}/approve")
    @PreAuthorize("@orderSecurityService.canApprove(authentication, #id)")
    public Order approveOrder(@PathVariable Long id) {
        return orderService.approve(id);
    }
}

@Service("orderSecurityService")
public class OrderSecurityService {
    public boolean canApprove(Authentication auth, Long orderId) {
        Order order = orderRepo.findById(orderId).orElseThrow();
        boolean isAdmin = auth.getAuthorities().stream()
            .anyMatch(a -> a.getAuthority().equals("ROLE_ADMIN"));
        boolean isOwnerManager = order.getDepartmentId()
            .equals(userService.getDepartment(auth.getName()));
        return isAdmin || isOwnerManager;
    }
}
```

■ 62. BASE

■ Q: What is BASE and how does it contrast with ACID?

A: BASE is the consistency model used by most NoSQL/distributed databases, in contrast to ACID.

- **B**asically **A**vailable: System guarantees availability (responses, even if stale)
- **S**oft State: State may change over time even without input (due to replication)
- **E**ventually Consistent: System will become consistent given enough time and no new updates

ACID vs BASE:

ACID	BASE	
Consistency	Strong (immediate)	Eventual
Availability	May sacrifice for consistency	Prioritized
Performance	Slower	Faster
Scale	Harder to scale out	Designed for horizontal scale
Use case	Payments, inventory	Social feeds, recommendations

When BASE is acceptable: User profiles, social feeds, product recommendations, shopping cart (eventual is OK for reads), analytics counters.

When ACID required: Money transfers, inventory reservation, order creation.

```
// BASE example: Cassandra eventual consistency
@Service
public class ActivityFeedService {

    private final CassandraTemplate cassandra;

    // Write: eventually consistent, available even during partition
    public void recordActivity(String userId, Activity activity) {
        cassandra.insert(UserActivity.from(userId, activity),
            InsertOptions.builder()
                .consistencyLevel(ConsistencyLevel.ONE)  // Available immediately
                .build());
    }

    // Read: may see slightly stale data (BASE)
    public List<UserActivity> getFeed(String userId) {
        return cassandra.select(
            Query.query(Criteria.where("user_id").is(userId))
                .sort(Sort.by(Sort.Direction.DESC, "occurred_at"))
                .limit(50),
            UserActivity.class,
            ReadOptions.builder()
                .consistencyLevel(ConsistencyLevel.ONE)  // Fast but may be stale
                .build());
    }

    // For important reads, use QUORUM (stronger consistency, slower)
```

```
    public long getExactFollowerCount(String userId) {
        return cassandra.selectOne(
            Query.query(Criteria.where("user_id").is(userId)),
            UserStats.class,
            ReadOptions.builder()
                .consistencyLevel(ConsistencyLevel.QUORUM)  // Read from majority
                .build()
        ).map(UserStats::getFollowerCount).orElse(0L);
    }
}

// Counter with Redis (BASE, eventually consistent, high performance)
@Service
public class LikeCounter {

    private final RedisTemplate<String, String> redis;

    // Extremely fast, eventually synced to DB
    public long incrementLike(String postId) {
        Long count = redis.opsForValue().increment("likes:" + postId);
        // Async persistence to DB every N increments or on schedule
        if (count % 100 == 0) asyncPersistToDb(postId, count);
        return count;
    }
}
```

■ 63. IDEMPOTENCY

■ Q: What is Idempotency and how do you implement it?

A: An operation is idempotent if calling it multiple times produces the same result as calling it once. Critical for safe retries in distributed systems.

HTTP Idempotency:

- GET, PUT, DELETE: Idempotent by definition
- POST: Not idempotent (each call creates a new resource)
- PATCH: Not inherently idempotent

Why matters: Networks fail. Clients retry. Without idempotency, a retry creates duplicate orders, charges.

Implementation Pattern:

1. Client generates `Idempotency-Key` (UUID) per unique operation
2. Server stores `(idempotency_key → result)` on first processing
3. Duplicate request with same key returns stored result without re-processing

```
// Idempotency key interceptor
@Service
public class IdempotentOrderService {

    private final OrderRepository orderRepo;
    private final IdempotencyKeyRepository idempotencyRepo;

    @Transactional
    public OrderResult createOrder(CreateOrderRequest request, String idempotencyKey) {
        // Check if we've already processed this key
        if (idempotencyKey != null) {
            Optional<IdempotencyRecord> existing =
                idempotencyRepo.findByKey(idempotencyKey);
            if (existing.isPresent()) {
                // Return the original result — don't process again
                log.info("Duplicate request detected for key: {}", idempotencyKey);
                return existing.get().getResult(OrderResult.class);
            }
        }

        // First time processing
        Order order = processOrder(request);
        OrderResult result = OrderResult.of(order);

        // Store result for future duplicate requests
        if (idempotencyKey != null) {
            idempotencyRepo.save(IdempotencyRecord.builder()
                .key(idempotencyKey)
                .result(serialize(result))
                .expiresAt(Instant.now().plus(Duration.ofDays(7)))
                .build());
        }

        return result;
```

```
    }
}

// Idempotent Redis increment (atomic — safe for concurrent calls)
@Service
public class WalletService {

    private final RedisTemplate<String, Long> redis;
    private final WalletRepository walletRepo;

    // Use idempotency key to prevent double-charges
    public boolean chargeWallet(String userId, long amount, String transactionId) {
        String idempotencyKey = "tx:" + transactionId;

        // SET NX (set if not exists) — atomic in Redis
        Boolean firstTime = redis.opsForValue()
            .setIfAbsent(idempotencyKey, 1L, Duration.ofDays(1));

        if (!Boolean.TRUE.equals(firstTime)) {
            log.info("Duplicate charge attempt for transaction: {}", transactionId);
            return true; // Already processed, idempotent success
        }

        // Actually charge
        return walletRepo.debit(userId, amount, transactionId);
    }
}

// Database-level idempotency via unique constraint
@Entity
@Table(uniqueConstraints = {
    @UniqueConstraint(name = "uk_order_idempotency",
                      columnNames = "idempotency_key")
})
public class Order {
    @Column(name = "idempotency_key")
    private String idempotencyKey;  // Must be unique — DB enforces idempotency
}
```

■ 64. API GATEWAY

■ Q: What does an API Gateway do and what should you put in it?

A: An API Gateway is a single entry point for all client requests to a microservices backend. It handles cross-cutting concerns.

Functions of API Gateway:

- **Routing:** Route requests to correct microservice
- **Authentication/Authorization:** Validate JWT, API keys
- **Rate Limiting:** Protect backends from overload
- **SSL Termination:** Handle HTTPS centrally
- **Load Balancing:** Distribute among service instances
- **Request/Response Transformation:** Header manipulation, body transformation
- **Caching:** Cache responses at gateway level
- **Circuit Breaking:** Fail fast if backend is down
- **Logging/Tracing:** Centralized access logs, correlation IDs

What NOT to put in API Gateway: Business logic (belongs in services), complex data aggregation (use BFF instead).

```
// Spring Cloud Gateway
@Configuration
public class GatewayConfig {

    @Bean
    public RouteLocator routes(RouteLocatorBuilder builder,
                               AuthenticationFilter authFilter,
                               RateLimitFilter rateFilter) {
        return builder.routes()
            .route("user-service", r -> r
                .path("/api/users/**")
                .filters(f -> f
                    .filter(authFilter)                          // Auth check
                    .filter(rateFilter.apply(100, 1))           // 100 req/sec
                    .addRequestHeader("X-Service", "user")
                    .addResponseHeader("X-Response-Time", LocalDateTime.now().toString())
                    .rewritePath("/api/users/(?<seg>.*)", "/users/${seg}")
                    .circuitBreaker(c -> c.setName("user-cb")
                        .setFallbackUri("forward:/fallback/user")))
                .uri("lb://user-service"))

            .route("order-service", r -> r
                .path("/api/orders/**")
                .and().method("GET","POST","PUT","DELETE")
                .filters(f -> f
                    .filter(authFilter)
                    .requestRateLimiter(c -> c
                        .setRateLimiter(redisRateLimiter())
                        .setKeyResolver(userKeyResolver())))
                .uri("lb://order-service"))
            .build();
    }

```

```
    // Redis-based rate limiter (per user)
    @Bean
    public RedisRateLimiter redisRateLimiter() {
        return new RedisRateLimiter(50, 100, 1); // 50 req/s, burst 100
    }

    @Bean
    public KeyResolver userKeyResolver() {
        return exchange -> Mono.just(
            exchange.getRequest().getHeaders().getFirst("X-User-Id") != null
                ? exchange.getRequest().getHeaders().getFirst("X-User-Id")
                : exchange.getRequest().getRemoteAddress().getHostString()
        );
    }

    // Fallback controller
    @RestController
    public class FallbackController {
        @GetMapping("/fallback/user")
        public ResponseEntity<Map<String,String>> userFallback() {
            return ResponseEntity.status(503)
                .body(Map.of("error", "User service temporarily unavailable"));
        }
    }
}

// Global filter: inject Correlation ID on every request
@Component
public class CorrelationIdGatewayFilter implements GlobalFilter {
    @Override
    public Mono<Void> filter(ServerWebExchange exchange, GatewayFilterChain chain) {
        String id = exchange.getRequest().getHeaders()
            .getFirst("X-Correlation-Id");
        if (id == null) id = UUID.randomUUID().toString();
        final String correlationId = id;
        return chain.filter(exchange.mutate()
            .request(r -> r.header("X-Correlation-Id", correlationId))
            .response(r -> { r.getHeaders().add("X-Correlation-Id", correlationId); return r; })
            .build());
    }
}
```

■ 65. LOGGING

■ Q: What are the best practices for production logging?

A: Good logging is essential for debugging production issues. Bad logging creates noise, costs money, and slows down systems.

Logging Best Practices:

- **Structured logging:** JSON format for machine parsing (not plain text)
- **Log levels:** ERROR=action needed, WARN=investigate, INFO=business events, DEBUG=development
- **Include context:** Trace ID, user ID, request ID in every log
- **Don't log sensitive data:** Passwords, credit cards, PII
- **Avoid logging in hot paths:** Logging has I/O cost — use async appenders
- **Log at service boundaries:** Request in, response out, external calls

Log aggregation: ELK Stack (Elasticsearch + Logstash + Kibana) or Grafana Loki (cheaper).

```
// Structured logging with Logstash Encoder
// build.gradle: implementation 'net.logstash.logback:logstash-logback-encoder:7.4'

// logback-spring.xml
/*
<appender name="ASYNC_JSON" class="ch.qos.logback.classic.AsyncAppender">
  <discardingThreshold>0</discardingThreshold>
  <queueSize>512</queueSize>
  <appender-ref ref="JSON_CONSOLE"/>
</appender>

<appender name="JSON_CONSOLE" class="ch.qos.logback.core.ConsoleAppender">
  <encoder class="net.logstash.logback.encoder.LogstashEncoder">
    <customFields>{"service":"order-service","environment":"${SPRING_PROFILES_ACTIVE}"}</customFields>
  </encoder>
</appender>
*/

@Slf4j
@Service
public class OrderService {

    public Order createOrder(CreateOrderRequest request) {
        // Log business events at INFO with structured context
        log.info("Creating order",
            kv("customerId", request.getCustomerId()),
            kv("itemCount", request.getItems().size()),
            kv("totalAmount", request.getTotalAmount()));

        try {
            Order order = doCreate(request);

            log.info("Order created successfully",
                kv("orderId", order.getId()),
                kv("customerId", order.getCustomerId()),
                kv("processingTimeMs", timer.elapsed()));
```

```
            return order;
        } catch (InsufficientFundsException e) {
            // Warn: expected business exception
            log.warn("Order rejected: insufficient funds",
                kv("customerId", request.getCustomerId()),
                kv("requiredAmount", request.getTotalAmount()),
                kv("reason", "INSUFFICIENT_FUNDS"));
            throw e;
        } catch (Exception e) {
            // Error: unexpected exception, needs investigation
            log.error("Order creation failed unexpectedly",
                kv("customerId", request.getCustomerId()),
                e);
            throw e;
        }
    }
}

// AOP-based request/response logging at controller boundary
@Aspect
@Component
@Slf4j
public class RequestResponseLoggingAspect {

    @Around("@within(org.springframework.web.bind.annotation.RestController)")
    public Object logRequest(ProceedingJoinPoint pjp) throws Throwable {
        long start = System.currentTimeMillis();
        String method = pjp.getSignature().toShortString();

        try {
            Object result = pjp.proceed();
            log.info("API call completed",
                kv("method", method),
                kv("durationMs", System.currentTimeMillis() - start),
                kv("status", "SUCCESS"));
            return result;
        } catch (Exception e) {
            log.error("API call failed",
                kv("method", method),
                kv("durationMs", System.currentTimeMillis() - start),
                kv("error", e.getMessage()));
            throw e;
        }
    }
}
```

■ 66. TIME SERIES DATA

■ Q: What is a Time Series database and when should you use one?

A: Time Series databases are optimized for storing and querying data indexed by time — metrics, events, IoT readings, financial prices.

Why specialized DB: Regular inserts ordered by time, never updated or deleted individually, queried by time range, aggregated (avg, max, min per window).

Time Series DBs vs RDBMS:

- TSDB: Automatic time partitioning, built-in compression, native time-window queries, high write throughput.
- RDBMS: General purpose — time series possible but requires custom partitioning, slower for high cardinality.

Tools: InfluxDB, TimescaleDB (PostgreSQL extension), Prometheus, OpenTSDB, ClickHouse.

```
// TimescaleDB (PostgreSQL extension) with Spring Boot
@Entity
@Table(name = "metrics")  // Hypertable in TimescaleDB
public class Metric {
    @Id
    private UUID id;

    @Column(nullable = false)
    private Instant time;         // Partitioning key

    @Column(nullable = false)
    private String metricName;    // e.g., "cpu.usage", "order.latency"

    @Column(nullable = false)
    private String serviceId;

    @Column(nullable = false)
    private Double value;

    @Column(columnDefinition = "jsonb")
    private Map<String, String> tags;
}

// Flyway migration: convert to hypertable
/*
SELECT create_hypertable('metrics', 'time',
    chunk_time_interval => INTERVAL '1 day',
    if_not_exists => TRUE);

-- Automated compression: compress data older than 7 days
SELECT add_compression_policy('metrics', INTERVAL '7 days');

-- Continuous aggregate: pre-compute hourly averages
CREATE MATERIALIZED VIEW metrics_hourly
WITH (timescaledb.continuous) AS
SELECT
    time_bucket('1 hour', time) AS bucket,
    metric_name,
```

```
    service_id,
    AVG(value) as avg_val,
    MAX(value) as max_val,
    MIN(value) as min_val,
    COUNT(*) as sample_count
FROM metrics
GROUP BY bucket, metric_name, service_id;
*/

// Repository with time-window queries
public interface MetricRepository extends JpaRepository<Metric, UUID> {

    @Query(value = """
        SELECT time_bucket(:interval::interval, time) as bucket,
               metric_name, AVG(value) as avg, MAX(value) as max
        FROM metrics
        WHERE service_id = :serviceId
          AND metric_name = :metric
          AND time BETWEEN :from AND :to
        GROUP BY bucket, metric_name
        ORDER BY bucket
        """, nativeQuery = true)
    List<MetricBucket> getTimeBucketedMetrics(
        @Param("serviceId") String serviceId,
        @Param("metric") String metric,
        @Param("from") Instant from,
        @Param("to") Instant to,
        @Param("interval") String interval  // "1 minute", "1 hour", "1 day"
    );
}
```

■ 67. CDN CACHING

■ Q: What is CDN Caching and what headers control it?

A: CDN (Content Delivery Network) caches static and dynamic content at edge nodes close to users, reducing latency and origin server load.

CDN Caching Headers:

- `Cache-Control: public, max-age=86400` — Cache for 1 day, allow CDN caching
- `Cache-Control: private, no-store` — Never cache (sensitive data)
- `Cache-Control: stale-while-revalidate=60` — Serve stale for 60s while fetching fresh
- `ETag` — Entity tag for conditional requests
- `Last-Modified` — For conditional GET (If-Modified-Since)
- `Vary` — Cache different versions based on headers (e.g., Vary: Accept-Encoding)

Cache Invalidation at CDN: Path-based invalidation, tag-based invalidation (Cloudflare Cache Tags), versioned URLs (best practice).

```
// Spring Boot: setting cache headers per endpoint
@RestController
public class ProductController {

    // Long-lived: product images, static catalog
    @GetMapping("/products/{id}/image")
    public ResponseEntity<Resource> getImage(@PathVariable Long id) {
        Resource image = productImageService.getImage(id);
        return ResponseEntity.ok()
            .cacheControl(CacheControl.maxAge(365, TimeUnit.DAYS)
                .cachePublic())
            .eTag(generateETag(image))     // Conditional GET support
            .body(image);
    }

    // Medium: product catalog (changes infrequently)
    @GetMapping("/products")
    public ResponseEntity<List<ProductDTO>> getProducts() {
        return ResponseEntity.ok()
            .cacheControl(CacheControl.maxAge(5, TimeUnit.MINUTES)
                .cachePublic()
                .staleWhileRevalidate(30, TimeUnit.SECONDS))
            .body(productService.findAll());
    }

    // Short: prices (change often, but stale OK briefly)
    @GetMapping("/products/{id}/price")
    public ResponseEntity<PriceDTO> getPrice(@PathVariable Long id) {
        return ResponseEntity.ok()
            .cacheControl(CacheControl.maxAge(30, TimeUnit.SECONDS)
                .cachePublic()
                .staleIfError(Duration.ofMinutes(5))) // Serve stale on error
            .body(priceService.getPrice(id));
    }

```

```
    // No cache: cart, orders, user-specific data
    @GetMapping("/cart")
    public ResponseEntity<Cart> getCart() {
        return ResponseEntity.ok()
            .cacheControl(CacheControl.noStore())  // Never cache
            .body(cartService.getCart());
    }
}

// Versioned URLs = instant CDN cache busting
@Service
public class AssetVersioningService {
    // /static/app.js → /static/app.a1b2c3d4.js
    public String getVersionedUrl(String assetPath) {
        String hash = computeContentHash(assetPath); // MD5 of file content
        return assetPath.replaceFirst("(\\.[^.]+)$", "." + hash + "$1");
    }
}
```

■ 68. EVENT SOURCING

■ Q: What is Event Sourcing and when does it provide the most value?

A: Event Sourcing persists the state of a domain object as a sequence of immutable events (facts) rather than just current state. To get current state: replay all events from beginning.

Traditional DB: Store current state. `UPDATE orders SET status='SHIPPED' WHERE id=1`

Event Sourcing: Append event. `INSERT INTO events (order_id, type, data) VALUES (1, 'OrderShipped', {...})`

Benefits:

- Complete audit trail: know every state change, who made it, when
- Time travel: reconstruct state at any point in time
- Event replay: rebuild read models, debug bugs with production data
- Natural fit with Event-Driven Architecture

Drawbacks:

- Complexity: more code, unfamiliar pattern
- Event versioning: schema evolution of events is hard
- Snapshot needed: replaying 10M events to get current state is slow
- Eventually consistent read models

```
// Event Store with Spring Boot
@Entity
@Table(name = "domain_events")
public class DomainEvent {
    @Id
    private UUID eventId;
    private String aggregateId;
    private String aggregateType;
    private String eventType;
    @Column(columnDefinition = "jsonb")
    private String payload;
    private long sequenceNumber;
    private Instant occurredAt;
    private String causedBy;  // User/system that caused the event
}

// Order aggregate with event sourcing
public class OrderAggregate {
    private String id;
    private OrderStatus status;
    private BigDecimal amount;
    private List<OrderItem> items;
    private long version = 0;

    private final List<DomainEvent> uncommittedEvents = new ArrayList<>();

    // Apply events to reconstruct state
    public void apply(DomainEvent event) {
        switch (event.getEventType()) {
```

```
            case "OrderPlaced" -> {
                OrderPlacedPayload p = deserialize(event.getPayload(), OrderPlacedPayload.class);
                this.status = OrderStatus.PLACED;
                this.amount = p.getAmount();
                this.items = p.getItems();
            }
            case "OrderPaid" -> this.status = OrderStatus.PAID;
            case "OrderShipped" -> this.status = OrderStatus.SHIPPED;
            case "OrderCancelled" -> this.status = OrderStatus.CANCELLED;
        }
        this.version++;
    }

    // Commands produce events
    public void cancel(String reason) {
        if (!status.isCancellable())
            throw new OrderNotCancellableException(id, status);

        DomainEvent event = DomainEvent.builder()
            .eventId(UUID.randomUUID())
            .aggregateId(id)
            .eventType("OrderCancelled")
            .payload(serialize(new OrderCancelledPayload(reason)))
            .sequenceNumber(version + 1)
            .occurredAt(Instant.now())
            .build();
        uncommittedEvents.add(event);
    }
}

// Repository: load by replaying events
@Service
public class OrderEventStore {

    public OrderAggregate load(String orderId) {
        List<DomainEvent> events = eventRepo.findByAggregateIdOrderBySequenceNumber(orderId);
        if (events.isEmpty()) throw new AggregateNotFoundException(orderId);

        OrderAggregate order = new OrderAggregate(orderId);
        events.forEach(order::apply);
        return order;
    }

    @Transactional
    public void save(OrderAggregate order) {
        List<DomainEvent> events = order.getUncommittedEvents();
        eventRepo.saveAll(events);
        // Publish for read-model consumers
        events.forEach(eventPublisher::publish);
        order.clearUncommittedEvents();
    }
}

```

```java
// Snapshot for performance (avoid replaying millions of events)
@Service
public class SnapshotService {
    private static final int SNAPSHOT_EVERY = 100; // Every 100 events

    public Optional<OrderAggregate> loadSnapshot(String orderId) {
        return snapshotRepo.findLatestByAggregateId(orderId)
            .map(s -> deserialize(s.getState(), OrderAggregate.class));
    }

    public OrderAggregate loadFromSnapshot(String orderId) {
        Optional<Snapshot> snapshot = loadSnapshot(orderId);
        long fromSequence = snapshot.map(Snapshot::getVersion).orElse(0L);

        OrderAggregate order = snapshot
            .map(s -> deserialize(s.getState(), OrderAggregate.class))
            .orElse(new OrderAggregate(orderId));

        // Only replay events after snapshot
        List<DomainEvent> recentEvents = eventRepo.findAfterSequence(orderId, fromSequence);
        recentEvents.forEach(order::apply);
        return order;
    }
}
```

■ 69. MERKLE TREES

■ Q: What is a Merkle Tree and where is it used in distributed systems?

A: A Merkle Tree is a binary tree where every leaf node contains the hash of a data block, and every non-leaf node contains the hash of its children. The root hash represents the integrity of all data.

Properties:

- Verify data integrity: if root hash matches, all data is intact
- Efficient diff: compare two trees to find exactly which leaves differ — O(log n) instead of O(n)
- Tamper-evident: changing any data changes the root hash

Where used:

- **Bitcoin/Blockchain:** Each block contains Merkle root of all transactions. Verify a transaction is in a block without downloading all transactions (SPV proof).
- **Git:** Every commit is a Merkle tree of file hashes.
- **Cassandra/DynamoDB:** Anti-entropy repair — compare Merkle trees across replicas to find inconsistencies efficiently.
- **IPFS:** Content-addressed storage.
- **Certificate Transparency Logs.**

```java
// Merkle Tree implementation
public class MerkleTree {

    private final List<String> dataBlocks;
    private List<String> tree;

    public MerkleTree(List<String> dataBlocks) {
        this.dataBlocks = dataBlocks;
        this.tree = buildTree(dataBlocks);
    }

    private List<String> buildTree(List<String> data) {
        List<String> current = data.stream()
            .map(this::hash).collect(Collectors.toList());

        List<String> allNodes = new ArrayList<>(current);

        while (current.size() > 1) {
            List<String> nextLevel = new ArrayList<>();
            for (int i = 0; i < current.size(); i += 2) {
                String left = current.get(i);
                String right = i + 1 < current.size() ? current.get(i + 1) : left; // Duplicate odd
                nextLevel.add(hash(left + right));
            }
            allNodes.addAll(nextLevel);
            current = nextLevel;
        }
        return allNodes;
    }

    public String getRootHash() {
        return tree.get(tree.size() - 1);
    }
```

```
    // Generate proof that dataBlocks[index] is in the tree
    public List<String> getProof(int index) {
        List<String> proof = new ArrayList<>();
        int levelSize = dataBlocks.size();
        int levelStart = 0;

        while (levelSize > 1) {
            int siblingIndex = (index % 2 == 0) ? index + 1 : index - 1;
            if (siblingIndex < levelSize) proof.add(tree.get(levelStart + siblingIndex));
            index /= 2;
            levelStart += levelSize;
            levelSize = (levelSize + 1) / 2;
        }
        return proof;
    }

    // Compare two trees to find differing blocks (anti-entropy)
    public List<Integer> findDifferences(MerkleTree other) {
        List<Integer> diffs = new ArrayList<>();
        compareNodes(tree.size() - 1, other.tree, diffs, 0, dataBlocks.size());
        return diffs;
    }

    private String hash(String data) {
        try {
            MessageDigest md = MessageDigest.getInstance("SHA-256");
            return HexFormat.of().formatHex(md.digest(data.getBytes()));
        } catch (NoSuchAlgorithmException e) { throw new RuntimeException(e); }
    }
}
```

■ 70. DATA PIPELINE

■ Q: What is a Data Pipeline and how do you design one?

A: A Data Pipeline is an automated series of steps that moves and transforms data from sources to destinations for analytics, reporting, or ML.

Components:

- **Ingestion:** Collect data from databases, APIs, event streams (Kafka, Kinesis)
- **Processing:** Clean, transform, enrich (Spark, Flink, Beam)
- **Storage:** Data warehouse (Snowflake, BigQuery, Redshift) or data lake
- **Serving:** BI tools, APIs, dashboards

Pipeline Types:

- **Batch:** Scheduled, process large chunks (nightly ETL)
- **Streaming:** Real-time, low latency (fraud detection, recommendations)
- **Lambda:** Both batch (accuracy) + streaming (freshness) → merge results
- **Kappa:** Streaming only, reprocess from Kafka for historical (simpler than Lambda)

```
// Kafka Streams data pipeline
@Configuration
public class AnalyticsPipeline {

    @Bean
    public KStream<String, Object> buildPipeline(StreamsBuilder builder) {
        // Ingest from multiple topics
        KStream<String, OrderEvent> orders = builder.stream("order-events");
        KStream<String, PaymentEvent> payments = builder.stream("payment-events");
        KStream<String, ShipmentEvent> shipments = builder.stream("shipment-events");

        // Transform: enrich orders with customer data
        KStream<String, EnrichedOrder> enriched = orders
            .filter((key, order) -> order.getStatus().equals("COMPLETED"))
            .mapValues(order -> EnrichedOrder.builder()
                .orderId(order.getId())
                .customerId(order.getCustomerId())
                .amount(order.getAmount())
                .region(order.getShippingRegion())
                .processedAt(Instant.now())
                .build());

        // Aggregate: revenue by region, per hour
        enriched.groupBy((key, order) -> order.getRegion(),
                Grouped.with(Serdes.String(), enrichedOrderSerde))
            .windowedBy(TimeWindows.ofSizeWithNoGrace(Duration.ofHours(1)))
            .aggregate(
                () -> new RegionalRevenue(),
                (region, order, acc) -> acc.add(order.getAmount()),
                Materialized.as("regional-revenue-store")
            )
            .toStream()
            .to("regional-revenue", Produced.with(windowedStringSerde, revenueSerde));

```

```
        // Join orders with payments (enrichment)
        enriched.join(
            payments,
            (order, payment) -> EnrichedOrderWithPayment.of(order, payment),
            JoinWindows.ofTimeDifferenceWithNoGrace(Duration.ofMinutes(5)),
            StreamJoined.with(Serdes.String(), enrichedOrderSerde, paymentEventSerde)
        ).to("enriched-order-payments");

        return orders;
    }
}
```

■ 71. BANDWIDTH

■ Q: How do you reduce bandwidth consumption in a distributed system?

A: Bandwidth is the data transfer capacity of a network link. Reducing bandwidth reduces costs, improves performance, and benefits mobile users.

Techniques:

1. **Compression:** gzip/Brotli for HTTP (typically 70-90% size reduction for text)
2. **Binary protocols:** gRPC/Protobuf (3-7x smaller than JSON)
3. **CDN:** Serve content from edge, not origin
4. **Pagination:** Never return unbounded data sets
5. **Field selection:** Only return requested fields (GraphQL, sparse fieldsets)
6. **Delta sync:** Send only changes, not full state
7. **Image optimization:** WebP, proper sizing, lazy loading
8. **HTTP/2 header compression (HPACK):** Reduces redundant headers

```
// Spring Boot: enable gzip compression
// application.yml:
// server.compression.enabled=true
// server.compression.mime-types=application/json,application/xml,text/html,text/plain
// server.compression.min-response-size=1024

// Response filtering: only return requested fields
@RestController
public class ProductController {

    @GetMapping("/products/{id}")
    public ResponseEntity<Map<String, Object>> getProduct(
            @PathVariable Long id,
            @RequestParam(required = false) Set<String> fields) {

        Product product = productService.findById(id);
        Map<String, Object> full = objectMapper.convertValue(product, new TypeReference<>(){});

        if (fields != null && !fields.isEmpty()) {
            // Return only requested fields (reduces payload)
            Map<String, Object> filtered = new LinkedHashMap<>();
            fields.forEach(f -> { if (full.containsKey(f)) filtered.put(f, full.get(f)); });
            return ResponseEntity.ok(filtered);
        }
        return ResponseEntity.ok(full);
    }
}

// Delta sync: only send changes after last sync
@GetMapping("/sync/products")
public ResponseEntity<SyncResponse> syncProducts(
        @RequestParam Instant lastSyncTime) {
    List<Product> changed = productRepo.findUpdatedAfter(lastSyncTime);
    List<Long> deleted = deletedProductRepo.findDeletedAfter(lastSyncTime);

```

```
    return ResponseEntity.ok(SyncResponse.builder()
        .changed(changed)
        .deleted(deleted)
        .syncTime(Instant.now())
        .build());
    // Client only downloads delta, not full catalog
}
```

■ 72. CDN

■ Q: How does a CDN work and what should be served from CDN vs origin?

A: A CDN is a geographically distributed network of servers (PoPs — Points of Presence) that cache and serve content close to users.

How it works:

1. User requests `https://cdn.example.com/image.jpg`
2. DNS resolves to nearest CDN edge node (anycast routing)
3. Edge: cache HIT → serve immediately
4. Edge: cache MISS → fetch from origin, cache, serve

CDN Serves: Static assets (JS, CSS, images, fonts), media files, cacheable API responses, HTML pages (SSG).

Origin Serves: Authenticated content, real-time data, user-specific data, write operations.

CDN Providers: Cloudflare, AWS CloudFront, Akamai, Fastly, Azure CDN.

```
// Spring Boot: CDN URL resolution for assets
@Configuration
public class CdnConfig {

    @Value("${cdn.base-url:}")
    private String cdnBaseUrl;

    @Bean
    public AssetResolver assetResolver() {
        return assetPath -> {
            if (cdnBaseUrl.isBlank()) return assetPath; // Dev: serve locally
            // Prod: prefix with CDN URL + version hash
            String version = assetVersionService.getVersion(assetPath);
            return cdnBaseUrl + "/" + version + assetPath;
        };
    }
}

// Dynamic CDN cache invalidation via Cloudflare API
@Service
public class CloudflareCacheService {

    private final WebClient cloudflare = WebClient.builder()
        .baseUrl("https://api.cloudflare.com/client/v4")
        .defaultHeader("Authorization", "Bearer " + cfToken)
        .build();

    // Invalidate specific paths (e.g., after product update)
    public Mono<Void> invalidate(List<String> paths) {
        return cloudflare.post()
            .uri("/zones/{zoneId}/purge_cache", zoneId)
            .bodyValue(Map.of("files", paths))
            .retrieve()
            .bodyToMono(Void.class);
    }
```

```
    // Invalidate by cache tag (Cloudflare Enterprise)
    public Mono<Void> invalidateByTag(String tag) {
        return cloudflare.post()
            .uri("/zones/{zoneId}/purge_cache", zoneId)
            .bodyValue(Map.of("tags", List.of(tag)))
            .retrieve()
            .bodyToMono(Void.class);
    }
}

// Set CDN cache tags in response header for tag-based invalidation
@GetMapping("/products/{id}")
public ResponseEntity<ProductDTO> getProduct(@PathVariable Long id) {
    Product p = productService.findById(id);
    return ResponseEntity.ok()
        .header("Cache-Tag", "product-" + id, "category-" + p.getCategoryId())
        .cacheControl(CacheControl.maxAge(1, TimeUnit.HOURS).cachePublic())
        .body(mapper.toDTO(p));
}
```

■ 73. AUTHORIZATION

■ Q: How does Authorization differ from Authentication and how do you implement fine-grained authorization?

A:

- **Authentication (AuthN):** Proves WHO you are. (Login, JWT validation)
- **Authorization (AuthZ):** Determines WHAT you can do. (Permission checks)

Authorization Models:

- **RBAC:** Role-based. "Admin can do X."
- **ABAC:** Attribute-based. "Managers can edit their team's orders."
- **ReBAC:** Relationship-based (Google Zanzibar). "User can view document if they own it or it's shared with them."
- **Policy-based (OPA):** Externalized policy rules evaluated at runtime.

```
// OPA (Open Policy Agent) integration for externalized authz
@Service
public class OpaAuthorizationService {

    private final WebClient opaClient = WebClient.builder()
        .baseUrl("http://opa:8181/v1/data/authz")
        .build();

    public boolean isAllowed(String userId, String action, String resource) {
        AuthzRequest req = AuthzRequest.builder()
            .input(Map.of(
                "user", getUserAttributes(userId),
                "action", action,
                "resource", resource,
                "context", Map.of("time", Instant.now().toString())))
            .build();

        Boolean allowed = opaClient.post().uri("/allow")
            .bodyValue(req)
            .retrieve()
            .bodyToMono(OpaResponse.class)
            .map(OpaResponse::getResult)
            .blockOptional()
            .orElse(false);

        auditLog(userId, action, resource, allowed);
        return Boolean.TRUE.equals(allowed);
    }
}

// OPA policy file (Rego language)
/*
package authz

default allow = false

# Admins can do anything
allow {
    input.user.roles[_] == "ADMIN"
```

```
}

# Users can read their own orders
allow {
    input.action == "ORDER_READ"
    input.resource.owner_id == input.user.id
}

# Managers can approve orders in their department
allow {
    input.action == "ORDER_APPROVE"
    input.user.roles[_] == "MANAGER"
    input.resource.department == input.user.department
    to_number(input.resource.amount) <= 10000  # Up to $10k
}
*/

// Spring Security with OPA
@Component
public class OpaPermissionEvaluator implements PermissionEvaluator {

    @Override
    public boolean hasPermission(Authentication auth, Object targetDomainObject,
                                 Object permission) {
        String userId = auth.getName();
        String action = permission.toString();
        String resource = targetDomainObject.getClass().getSimpleName() + ":" +
            getResourceId(targetDomainObject);

        return opaService.isAllowed(userId, action, resource);
    }
}

// Usage
@PreAuthorize("hasPermission(#order, 'ORDER_APPROVE')")
public Order approveOrder(Order order) {
    return orderService.approve(order);
}
```

■ 74. PARTITIONING

■ Q: What is the difference between Partitioning and Sharding?

A:

- **Partitioning:** Dividing data within a SINGLE database instance into logical or physical parts (partition pruning reduces scan scope).
- **Sharding:** Distributing data across MULTIPLE database instances.

Partitioning Types:

Type	How	Best For
Range	By value range	Time-series (partition by month)
List	By explicit value list	Geographic regions, status values
Hash	By hash of column	Even distribution, no hot spots
Composite	Combination	Range by month + hash by customer

Benefits of Partitioning: Partition pruning (queries only scan relevant partitions), faster maintenance (vacuum/delete per partition), parallel query across partitions.

```
// Flyway migration: PostgreSQL declarative partitioning
/*
-- Range partitioning by month
CREATE TABLE orders (
    id BIGSERIAL,
    customer_id VARCHAR NOT NULL,
    amount DECIMAL(19,4),
    status VARCHAR NOT NULL,
    created_at TIMESTAMPTZ NOT NULL
) PARTITION BY RANGE (created_at);

-- Create monthly partitions
CREATE TABLE orders_2024_01 PARTITION OF orders
    FOR VALUES FROM ('2024-01-01') TO ('2024-02-01');

CREATE TABLE orders_2024_02 PARTITION OF orders
    FOR VALUES FROM ('2024-02-01') TO ('2024-03-01');

-- Indexes on partition key for partition pruning
CREATE INDEX ON orders (created_at, customer_id);

-- Query: only scans 2024_01 partition automatically
-- EXPLAIN ANALYZE SELECT * FROM orders WHERE created_at BETWEEN '2024-01-01' AND '2024-01-31';
*/

// Automated partition creation with Spring Boot
@Service
@Scheduled(cron = "0 0 1 1 * *")  // 1st of each month
public class PartitionManager {

    public void createNextMonthPartition() {
```

```
        YearMonth nextMonth = YearMonth.now().plusMonths(1);
        String partitionName = "orders_" + nextMonth.format(DateTimeFormatter.ofPattern("yyyy_MM"));
        LocalDate start = nextMonth.atDay(1);
        LocalDate end = nextMonth.plusMonths(1).atDay(1);

        jdbcTemplate.execute(String.format("""
            CREATE TABLE IF NOT EXISTS %s PARTITION OF orders
            FOR VALUES FROM ('%s') TO ('%s')
            """, partitionName, start, end));

        log.info("Created partition: {}", partitionName);
    }

    // Drop old partitions (data retention policy)
    @Scheduled(cron = "0 0 2 1 * *")
    public void dropOldPartitions() {
        YearMonth retentionCutoff = YearMonth.now().minusMonths(24); // Keep 2 years
        String oldPartition = "orders_" + retentionCutoff.format(DateTimeFormatter.ofPattern("yyyy_MM"))

        jdbcTemplate.execute("DROP TABLE IF EXISTS " + oldPartition);
        log.info("Dropped old partition: {}", oldPartition);
    }
}
```

■ 75. SSO (Single Sign-On)

■ Q: How does SSO work and what protocols implement it?

A: SSO lets users authenticate once and access multiple applications without re-logging in. Central Identity Provider (IdP) manages authentication.

Protocols:

- **SAML 2.0:** XML-based, enterprise, browser-based SSO. Used by Okta, Azure AD, Google Workspace.
- **OAuth 2.0:** Authorization framework (not AuthN). Used for delegated access ("Login with Google").
- **OpenID Connect (OIDC):** Identity layer on top of OAuth 2.0. Issues ID tokens. Modern SSO.
- **Kerberos:** Ticket-based, enterprise Windows environments (Active Directory).

OAuth 2.0 Flows:

- Authorization Code + PKCE: Web/mobile apps (most common)
- Client Credentials: Machine-to-machine (no user involved)
- Device Code: Smart TVs, CLIs

```
// Spring Boot OIDC SSO
@Configuration
@EnableWebSecurity
public class OidcSecurityConfig {

    @Bean
    public SecurityFilterChain filterChain(HttpSecurity http) throws Exception {
        return http
            .oauth2Login(oauth -> oauth
                .loginPage("/login")
                .defaultSuccessUrl("/dashboard")
                .userInfoEndpoint(u -> u
                    .oidcUserService(customOidcUserService())))
            .oauth2ResourceServer(rs -> rs
                .jwt(jwt -> jwt.decoder(jwtDecoder())))
            .authorizeHttpRequests(auth -> auth
                .requestMatchers("/login", "/oauth2/**").permitAll()
                .anyRequest().authenticated())
            .build();
    }
}

// application.yml — Okta as OIDC provider
/*
spring:
  security:
    oauth2:
      client:
        registration:
          okta:
            client-id: ${OKTA_CLIENT_ID}
            client-secret: ${OKTA_CLIENT_SECRET}
            scope: openid,profile,email,groups
            redirect-uri: "{baseUrl}/login/oauth2/code/okta"
        provider:
```

```
          okta:
            issuer-uri: https://{yourOktaDomain}/oauth2/default
*/

// Custom OIDC user service: map IdP groups to Spring roles
@Service
public class CustomOidcUserService extends OidcUserService {

    @Override
    public OidcUser loadUser(OidcUserRequest req) throws OAuth2AuthenticationException {
        OidcUser oidcUser = super.loadUser(req);

        // Map IdP groups to Spring Security roles
        List<GrantedAuthority> authorities = new ArrayList<>(oidcUser.getAuthorities());
        List<String> groups = oidcUser.getAttribute("groups");
        if (groups != null) {
            groups.stream()
                .map(g -> new SimpleGrantedAuthority("ROLE_" + g.toUpperCase()))
                .forEach(authorities::add);
        }

        // Provision user in local DB if first login
        userProvisioningService.provisionIfAbsent(oidcUser);

        return new DefaultOidcUser(authorities, oidcUser.getIdToken(), oidcUser.getUserInfo());
    }
}
```

■ 76. MICROSERVICES

■ Q: What are the key patterns for building production microservices?

A: Microservices decompose a monolith into small, independently deployable services aligned with business domains.

When microservices: Large teams, independent deployment needs, different scaling requirements per service, polyglot tech requirements.

When monolith (first): Small team, early stage, unclear domain boundaries, operational complexity unaffordable.

Key Patterns:

- **Service per business domain** (not per technical layer)
- **Async communication** for most service-to-service (Kafka/RabbitMQ)
- **Sync (REST/gRPC)** for user-facing, time-critical interactions
- **Saga pattern** for distributed transactions
- **Circuit breaker** for resilience
- **Observability** (logs + metrics + traces) across all services
- **API Gateway** as single entry point

```
// Saga pattern: distributed transaction across microservices
// Choreography-based (event-driven, no central coordinator)
@Service
public class OrderSagaOrchestrator {

    // SAGA STEPS for "Place Order":
    // 1. Reserve inventory → 2. Process payment → 3. Confirm order
    // If any fails → compensate all previous steps

    @KafkaListener(topics = "inventory-reserved")
    @Transactional
    public void onInventoryReserved(InventoryReservedEvent event) {
        Order order = orderRepo.findById(event.getOrderId()).orElseThrow();
        order.setStatus(OrderStatus.INVENTORY_RESERVED);
        orderRepo.save(order);

        // Proceed to next saga step
        kafkaTemplate.send("process-payment",
            new ProcessPaymentCommand(order.getId(), order.getAmount()));
    }

    @KafkaListener(topics = "payment-completed")
    @Transactional
    public void onPaymentCompleted(PaymentCompletedEvent event) {
        Order order = orderRepo.findById(event.getOrderId()).orElseThrow();
        order.setStatus(OrderStatus.COMPLETED);
        orderRepo.save(order);
        kafkaTemplate.send("order-completed", new OrderCompletedEvent(order.getId()));
    }

    // COMPENSATION: payment failed → release inventory
    @KafkaListener(topics = "payment-failed")
    @Transactional
```

```
    public void onPaymentFailed(PaymentFailedEvent event) {
        Order order = orderRepo.findById(event.getOrderId()).orElseThrow();
        order.setStatus(OrderStatus.PAYMENT_FAILED);
        orderRepo.save(order);

        // Compensate: release reserved inventory
        kafkaTemplate.send("release-inventory",
            new ReleaseInventoryCommand(order.getId(), order.getItems()));
    }
}
```

■ 77. BACKPRESSURE

■ Q: What is Backpressure and how do you implement it?

A: Backpressure is a mechanism where a slow consumer signals to a fast producer to slow down, preventing the consumer from being overwhelmed.

Without backpressure: Producer → [buffer fills up] → OOM, dropped messages, or cascading failure.

With backpressure: Consumer signals "I'm full" → Producer slows down or routes to overflow handling.

Implementation approaches:

- **Reactive streams (Project Reactor):** Built-in backpressure via `request(n)` protocol
- **Queue depth limits:** Bounded queues with rejection policies
- **Rate limiting:** Cap producer rate based on consumer capacity
- **Load shedding:** Drop low-priority requests when overwhelmed

```
// Reactive backpressure with Project Reactor
@Service
public class OrderProcessingPipeline {

    public void startProcessing() {
        Flux.fromIterable(orderSource.getAllOrders())
            .onBackpressureBuffer(1000,     // Buffer up to 1000 items
                dropped -> log.warn("Order dropped due to backpressure: {}", dropped),
                BufferOverflowStrategy.DROP_OLDEST)
            // Or: .onBackpressureDrop(dropped -> sendToDeadLetter(dropped))
            // Or: .onBackpressureLatest()  // Keep only most recent
            .flatMap(order -> processOrder(order), 10)  // Max 10 concurrent
            .doOnError(e -> log.error("Pipeline error", e))
            .retryWhen(Retry.backoff(3, Duration.ofSeconds(1)))
            .subscribe();
    }

    // Reactive HTTP endpoint with backpressure
    @GetMapping(value = "/orders/stream", produces = MediaType.APPLICATION_NDJSON_VALUE)
    public Flux<Order> streamOrders(
            @RequestParam(defaultValue = "10") int prefetch) {
        return orderRepository.findAllAsFlux()
            .limitRate(prefetch)  // Subscriber controls rate
            .delayElements(Duration.ofMillis(10)); // Throttle
    }
}

// Bounded queue with backpressure in thread pool
@Service
public class BoundedOrderProcessor {

    private final BlockingQueue<Order> queue = new ArrayBlockingQueue<>(500);

    // Producer: submits orders with backpressure
    public boolean submit(Order order) {
        boolean accepted = queue.offer(order);  // Returns false if full
        if (!accepted) {
```

```
            metrics.counter("orders.rejected.backpressure").increment();
            // Options: reject, send to overflow queue, wait, or drop
        }
        return accepted;
    }

    @PostConstruct
    public void startConsumers() {
        // Fixed number of consumers — natural backpressure via bounded queue
        IntStream.range(0, 5).forEach(i ->
            Thread.ofVirtual().start(() -> {
                while (!Thread.currentThread().isInterrupted()) {
                    try {
                        Order order = queue.poll(1, TimeUnit.SECONDS);
                        if (order != null) processOrder(order);
                    } catch (InterruptedException e) { Thread.currentThread().interrupt(); }
                }
            }));
    }
}
```

■ 78. LOAD SHEDDING

■ Q: What is Load Shedding and how does it differ from Rate Limiting?

A:

- **Rate Limiting:** Cap requests from a specific client/API key. "You get max 100 req/s."
- **Load Shedding:** Drop lowest-priority requests when system is overloaded. "System at 95% — drop non-critical requests globally."

Load shedding protects the service itself when total load exceeds capacity, regardless of source.

Priority tiers (example):

1. Health checks (always pass)
2. Payment/checkout (critical business)
3. Product browsing (important)
4. Analytics/reporting (can wait)
5. Background jobs (shed first)

```
// Adaptive load shedding based on system metrics
@Component
public class LoadSheddingFilter implements Filter {

    private final MeterRegistry registry;

    @Override
    public void doFilter(ServletRequest req, ServletResponse res, FilterChain chain)
            throws IOException, ServletException {
        HttpServletRequest httpReq = (HttpServletRequest) req;
        HttpServletResponse httpRes = (HttpServletResponse) res;

        if (shouldShed(httpReq)) {
            metrics.counter("requests.shed", "path", httpReq.getRequestURI()).increment();
            httpRes.setStatus(503);
            httpRes.setHeader("Retry-After", "30");
            httpRes.getWriter().write("{\"error\":\"Service overloaded, please retry\"}");
            return;
        }
        chain.doFilter(req, res);
    }

    private boolean shouldShed(HttpServletRequest req) {
        double cpuLoad = getSystemCpuLoad();
        double memUsage = getMemoryUsagePercent();
        int queueDepth = getRequestQueueDepth();

        // Never shed health checks
        if (req.getRequestURI().startsWith("/actuator/health")) return false;

        // Always shed if critically overloaded
        if (cpuLoad > 0.95 || memUsage > 0.90) return true;

        // Probabilistic shedding: higher load → more shedding
        if (cpuLoad > 0.80) {
```

```
            double shedProbability = (cpuLoad - 0.80) / 0.15; // 0% → 100% as CPU 80% → 95%
            if (!isHighPriority(req) && Math.random() < shedProbability) return true;
        }

        return false;
    }

    private boolean isHighPriority(HttpServletRequest req) {
        String path = req.getRequestURI();
        return path.startsWith("/api/checkout") || path.startsWith("/api/payment");
    }
}
```

■ 79. METRICS

■ Q: What types of metrics should you collect and how do you alert on them?

A: Metrics are numeric measurements collected over time to understand system behavior.

Metric Types (Prometheus / Micrometer):

Type	Description	Examples
Counter	Monotonically increasing	requests total, errors total
Gauge	Current value (up and down)	active connections, queue depth
Histogram	Value distribution in buckets	request duration, response size
Summary	Percentiles (client-side)	p50, p95, p99 latency

What to measure: USE (Utilization, Saturation, Errors) for resources + RED (Rate, Errors, Duration) for services.

```
// Comprehensive metrics setup with Micrometer
@Configuration
public class MetricsConfig {

    // Custom application metrics
    @Bean
    public MeterBinder orderMetrics(OrderRepository orderRepo) {
        return registry -> {
            // Counter: total orders created
            registry.counter("orders.total", "status", "created");

            // Gauge: current active orders
            Gauge.builder("orders.active", orderRepo, OrderRepository::countByStatusProcessing)
                .description("Orders currently being processed")
                .register(registry);

            // Distribution summary: order amount distribution
            DistributionSummary.builder("order.amount.usd")
                .publishPercentiles(0.5, 0.95, 0.99)
                .publishPercentileHistogram()
                .baseUnit("dollars")
                .register(registry);

            // Timer with SLO buckets
            Timer.builder("order.processing.duration")
                .publishPercentiles(0.5, 0.90, 0.95, 0.99)
                .serviceLevelObjectives(
                    Duration.ofMillis(100),
                    Duration.ofMillis(500),
                    Duration.ofSeconds(1),
                    Duration.ofSeconds(5))
                .description("Time to process an order")
                .register(registry);
        };
    }
}
```

```
// USE model metrics for resource monitoring
@Component
public class SystemMetrics {

    @Scheduled(fixedDelay = 15_000)
    public void recordSystemMetrics() {
        OperatingSystemMXBean os = ManagementFactory.getOperatingSystemMXBean();
        // Utilization
        meterRegistry.gauge("system.cpu.utilization",
            ((com.sun.management.OperatingSystemMXBean) os).getCpuLoad());

        // Saturation (run queue length)
        ThreadMXBean threads = ManagementFactory.getThreadMXBean();
        meterRegistry.gauge("jvm.threads.runnable",
            Arrays.stream(threads.getAllThreadIds())
                .filter(id -> threads.getThreadInfo(id).getThreadState() == Thread.State.RUNNABLE)
                .count());
    }
}
```

■ 80. VECTOR DB

■ Q: What is a Vector Database and when do you need one?

A: A Vector Database stores and queries high-dimensional vectors (embeddings) by semantic similarity, not exact match.

Why needed: Machine learning models represent text, images, audio as dense vectors (embeddings). Traditional DBs can't efficiently find "similar" vectors.

Use cases:

- Semantic search (find documents similar in meaning, not keywords)
- Recommendation systems (find products similar to what user liked)
- RAG (Retrieval-Augmented Generation) for LLMs
- Image similarity, anomaly detection, duplicate detection

Similarity metrics: Cosine similarity (angle), Euclidean distance, dot product.

Vector DBs: Pinecone, Weaviate, Milvus, pgvector (PostgreSQL extension), Chroma, Qdrant.

```
// Spring AI with pgvector (PostgreSQL extension)
// build.gradle: implementation 'org.springframework.ai:spring-ai-pgvector-store-spring-boot-starter'

@Service
public class SemanticSearchService {

    @Autowired private VectorStore vectorStore;  // pgvector backend
    @Autowired private EmbeddingClient embeddingClient;  // OpenAI text-embedding-ada-002

    // Index product descriptions as vectors
    public void indexProduct(Product product) {
        String text = product.getName() + " " + product.getDescription() +
            " " + String.join(" ", product.getTags());

        List<Float> embedding = embeddingClient.embed(text);  // 1536-dimensional vector

        Document doc = new Document(
            text,
            Map.of("productId", product.getId(),
                   "category", product.getCategory(),
                   "price", product.getPrice()),
            embedding
        );
        vectorStore.add(List.of(doc));
    }

    // Semantic search: "affordable running shoes for flat feet"
    public List<ProductResult> search(String query, int topK) {
        List<Float> queryEmbedding = embeddingClient.embed(query);

        SearchRequest searchReq = SearchRequest.query(query)
            .withTopK(topK)
            .withSimilarityThreshold(0.7)  // Minimum cosine similarity
            .withFilterExpression("category == 'FOOTWEAR' && price < 200");
```

```
        List<Document> similar = vectorStore.similaritySearch(searchReq);

        return similar.stream()
            .map(doc -> ProductResult.builder()
                .productId((String) doc.getMetadata().get("productId"))
                .score(doc.getScore())
                .build())
            .collect(Collectors.toList());
    }

    // RAG: Retrieve relevant context for LLM
    public String answerQuestion(String userQuestion) {
        List<Document> context = vectorStore.similaritySearch(
            SearchRequest.query(userQuestion).withTopK(5));

        String contextText = context.stream()
            .map(Document::getContent)
            .collect(Collectors.joining("\n\n"));

        return aiClient.call("Answer based on context:\n" + contextText +
            "\nQuestion: " + userQuestion);
    }
}
```

■ 81. DATA COMPRESSION

■ Q: What compression algorithms should you use and when?

A: Compression reduces storage and bandwidth at the cost of CPU time.

Algorithms Comparison:

Algorithm	Ratio	Speed	Use Case
gzip	Good	Moderate	HTTP, files
Brotli	Better	Slower	HTTP/2 web
LZ4	Moderate	Very Fast	Real-time streaming
Snappy	Moderate	Very Fast	Kafka, columnar storage
Zstandard (zstd)	Excellent	Fast	Everything (modern choice)
GZIP deflate	Good	Moderate	Legacy compatibility

Rule: CPU-bound systems → less compression. I/O-bound or bandwidth-bound → more compression.

```
// HTTP response compression (Spring Boot)
// application.yml:
// server.compression.enabled=true
// server.compression.mime-types=application/json,text/html,text/plain
// server.compression.min-response-size=2048

// Kafka message compression
@Bean
public ProducerFactory<String, Object> compressedProducerFactory() {
    return new DefaultKafkaProducerFactory<>(Map.of(
        ProducerConfig.BOOTSTRAP_SERVERS_CONFIG, "kafka:9092",
        ProducerConfig.COMPRESSION_TYPE_CONFIG, "zstd",  // Best ratio+speed
        ProducerConfig.BATCH_SIZE_CONFIG, 131072           // 128KB batches
    ));
}

// Custom zstd compression for large payloads
@Service
public class PayloadCompressor {

    private final ZstdCompressor compressor = new ZstdCompressor();
    private final ZstdDecompressor decompressor = new ZstdDecompressor();

    public byte[] compress(byte[] data) {
        int maxCompressedLength = Zstd.compressBound(data.length);
        byte[] compressed = new byte[(int) maxCompressedLength];
        int compressedSize = (int) Zstd.compress(compressed, data, 3); // Level 3: balanced
        return Arrays.copyOf(compressed, compressedSize);
    }

    public byte[] decompress(byte[] compressed, int originalSize) {
        byte[] decompressed = new byte[originalSize];
```

```
        Zstd.decompress(decompressed, compressed);
        return decompressed;
    }

    // Store original size in header for decompression
    public byte[] compressWithHeader(byte[] data) {
        byte[] compressed = compress(data);
        ByteBuffer result = ByteBuffer.allocate(4 + compressed.length);
        result.putInt(data.length);  // Original size header
        result.put(compressed);
        return result.array();
    }
}
```

■ 82. SERVICE MESH

■ Q: What is a Service Mesh and when do you need one?

A: A Service Mesh is an infrastructure layer that handles service-to-service communication: routing, load balancing, retries, circuit breaking, mTLS, observability — without changing application code.

How it works: Sidecar proxy (Envoy) injected alongside each service container. All traffic goes through sidecars.

Service Mesh Provides:

- mTLS between services (automatic encryption + authentication)
- Traffic management (canary, A/B, circuit breaking)
- Distributed tracing (automatic span creation)
- Observability (metrics, logs per service pair)
- Retry policies without application code changes

Popular Meshes: Istio (most features, complex), Linkerd (simpler, lighter), Consul Connect, AWS App Mesh.

When to use: 10+ microservices, need mTLS, complex traffic management, observability across services, polyglot services.

When NOT to: Small number of services (overhead not worth it), simple architectures.

```
# Istio VirtualService: traffic management without code changes
apiVersion: networking.istio.io/v1alpha3kind: VirtualServicemetadata:  name: order-servicespec:  hosts:  - order-servi
  http:  - match:
    - headers:
        x-version:          exact: v2    route:    - destination:
        host: order-service        subset: v2  - route:
    - destination:
        host: order-service        subset: v1      weight: 90    - destination:
        host: order-service        subset: v2      weight: 10    retries:      attempts: 3      perTryTimeout
# DestinationRule: mTLS + circuit breaking (no app code needed)
apiVersion: networking.istio.io/v1alpha3kind: DestinationRulemetadata:  name: order-servicespec:  host: order-service
```

■ 83. BLOOM FILTER

■ Q: What is a Bloom Filter and when is it useful?

A: A Bloom Filter is a space-efficient probabilistic data structure that tests whether an element is in a set. It can have false positives but never false negatives.

- "Definitely NOT in set" → 100% certain
- "Might be in set" → need to verify (possible false positive)

Space efficiency: Storing 1 million items with 1% FP rate requires only ~1.1MB vs millions of bytes for HashSet.

Use cases:

- Database: Check if record likely exists before disk read (avoid I/O for non-existent keys)
- Cache: Avoid caching items that will never be requested again (prevent cache pollution)
- URL shortener: Quickly check if custom alias exists
- Network: Routers checking if destination was recently seen
- Google BigTable, Cassandra, HBase use Bloom filters per SSTable

```
// Google Guava Bloom Filter
@Service
public class ProductExistenceFilter {

    // 10M items, 1% false positive rate
    private final BloomFilter<Long> productFilter =
        BloomFilter.create(Funnels.longFunnel(), 10_000_000, 0.01);

    @PostConstruct
    public void loadExistingProducts() {
        // Load all product IDs at startup
        productRepo.findAllIds().forEach(productFilter::put);
        log.info("Bloom filter loaded with {} products", productRepo.count());
    }

    public Optional<Product> findProduct(Long id) {
        // Bloom filter check: O(1), no DB call
        if (!productFilter.mightContain(id)) {
            // Definitely not in DB — skip the DB call entirely
            return Optional.empty();
        }
        // Might exist — check DB (handles false positives)
        return productRepo.findById(id);
    }

    @EventListener
    public void onProductCreated(ProductCreatedEvent event) {
        productFilter.put(event.getProductId());
    }
}

// Distributed Bloom filter with Redis (for multi-instance)
@Service
public class DistributedBloomFilter {

```

```
    private final RedisTemplate<String, byte[]> redis;
    private static final String KEY = "bloom:products";
    private static final int HASH_FUNCTIONS = 7;  // k hash functions
    private static final long FILTER_SIZE = 100_000_000L;  // 100M bits

    public void add(Long id) {
        List<Long> positions = getHashPositions(id);
        positions.forEach(pos ->
            redis.opsForValue().setBit(KEY, pos, true));
    }

    public boolean mightContain(Long id) {
        List<Long> positions = getHashPositions(id);
        return positions.stream().allMatch(pos ->
            Boolean.TRUE.equals(redis.opsForValue().getBit(KEY, pos)));
    }

    private List<Long> getHashPositions(Long id) {
        return IntStream.range(0, HASH_FUNCTIONS)
            .mapToLong(i -> Math.abs(hash(id, i)) % FILTER_SIZE)
            .boxed().collect(Collectors.toList());
    }
}
```

■ 84. DATA LAKE

■ Q: What is a Data Lake and how does it differ from a Data Warehouse?

A:

- **Data Lake:** Store raw, unprocessed data in native format (files, JSON, Parquet, images) at massive scale. Schema-on-read. Cheap storage. For exploration.
- **Data Warehouse:** Structured, processed, optimized for analytics queries. Schema-on-write. Expensive. For BI/reporting.
- **Data Lakehouse:** Combines both — raw lake + warehouse-quality ACID queries (Delta Lake, Apache Iceberg).

Data Lake	Data Warehouse	Data Lakehouse	
Data	Raw, any format	Structured, processed	Both
Schema	On read	On write	Both
Storage	Cheap (S3, HDFS)	Expensive (Redshift)	Cheap + ACID
Users	Data scientists	Business analysts	Both
Examples	S3 + Glue	Snowflake, BigQuery	Delta Lake, Iceberg

```
// Writing to Data Lake (S3 + Parquet via Apache Spark)
@Service
public class DataLakeWriter {

    @Autowired private SparkSession spark;

    public void writeOrdersToDataLake(List<Order> orders, String date) {
        // Convert to DataFrame
        Dataset<Row> df = spark.createDataFrame(orders, Order.class)
            .withColumn("year", functions.year(functions.col("createdAt")))
            .withColumn("month", functions.month(functions.col("createdAt")))
            .withColumn("day", functions.dayofmonth(functions.col("createdAt")));

        // Write as Parquet with Hive-style partitioning (partition pruning for queries)
        df.write()
            .mode(SaveMode.Append)
            .partitionBy("year", "month", "day")
            .option("compression", "snappy")  // Good balance for analytics
            .parquet("s3://my-data-lake/orders/");
    }

    // Register as Glue catalog table for Athena/Presto queries
    public void registerGlueCatalog() {
        spark.sql("""
            CREATE TABLE IF NOT EXISTS orders_raw
            USING parquet
            PARTITIONED BY (year, month, day)
            LOCATION 's3://my-data-lake/orders/'
            """);
        spark.sql("MSCK REPAIR TABLE orders_raw"); // Discover new partitions
    }

    // Delta Lake: ACID on data lake
```

```
    public void writeWithDeltaLake(List<Order> orders) {
        Dataset<Row> df = spark.createDataFrame(orders, Order.class);
        df.write()
            .format("delta")
            .mode(SaveMode.Merge)  // ACID upsert!
            .save("s3://my-data-lake/delta/orders/");
    }
}
```

■ 85. CLIENT-SERVER ARCHITECTURE

■ Q: What are the variants of Client-Server architecture?

A: Client-Server is the foundational architecture where clients request services and servers provide them.

Variants:

- **2-Tier:** Client ↔ Server (thin client directly accesses DB — legacy)
- **3-Tier:** Presentation → Application → Data (most web apps)
- **N-Tier:** Multiple specialized tiers (CDN → LB → API Gateway → Services → Cache → DB)
- **Peer-to-Peer:** Every node is both client and server (BitTorrent, blockchain)

Modern distributed 3-tier with Spring Boot:

- **Presentation:** React SPA / Mobile App
- **Application:** Spring Boot REST API / Microservices
- **Data:** PostgreSQL + Redis + Elasticsearch

■ 86. DNS

■ Q: How does DNS work and what are the record types?

A: DNS (Domain Name System) translates human-readable domain names to IP addresses. Hierarchical, distributed, cached.

Resolution Flow:

1. Browser checks local cache → OS cache → Router cache
2. Recursive resolver (ISP) queries root nameservers → TLD nameservers → Authoritative nameservers
3. Returns IP, caches with TTL

Important DNS Record Types:

Record	Purpose	Example
A	Domain → IPv4	`api.example.com → 1.2.3.4`
AAAA	Domain → IPv6	`api.example.com → 2001::1`
CNAME	Alias to another domain	`www → example.com`
MX	Mail exchange server	`example.com → mail.example.com`
TXT	Text data (SPF, DKIM, verification)	`v=spf1 include:...`
SRV	Service location	gRPC service discovery
NS	Nameserver delegation	

DNS for load balancing: Round-robin DNS returns multiple A records. Simple but ignores server health.

DNS failover: Low TTL + health checks = fast failover.

```java
// DNS-based service discovery in Spring Boot
@Service
public class DnsServiceDiscovery {

    // Resolve all IP addresses for a service (round-robin DNS)
    public List<InetAddress> resolveServiceInstances(String serviceName) {
        try {
            return Arrays.asList(InetAddress.getAllByName(serviceName + ".default.svc.cluster.local"));
        } catch (UnknownHostException e) {
            log.error("Service not found: {}", serviceName);
            return Collections.emptyList();
        }
    }

    // DNS-based health check + failover
    @Scheduled(fixedDelay = 10_000)
    public void refreshServiceRegistry() {
        List<InetAddress> addresses = resolveServiceInstances("payment-service");
        serviceRegistry.updateInstances("payment-service", addresses.stream()
            .map(a -> new ServiceInstance(a.getHostAddress(), 8080))
            .collect(Collectors.toList()));
    }
}
```

```

// Custom DNS resolver for connection pooling with DNS TTL respect
@Configuration
public class DnsAwareHttpClient {

    @Bean
    public HttpClient httpClient() {
        // Respect DNS TTL changes (important for blue-green, failover)
        return HttpClient.newBuilder()
            .connectTimeout(Duration.ofSeconds(5))
            .build();
        // Note: Java's default DNS caching ignores TTL — set networkaddress.cache.ttl=30 in JVM
    }
}
```

■ 87. RATE LIMITING

■ Q: What are the rate limiting algorithms and how do you implement them?

A: Rate limiting controls how many requests a client can make in a time window, protecting the service from abuse and overload.

Algorithms:

Algorithm	How	Pros	Cons
Fixed Window	Count per time window	Simple	Burst at window boundary
Sliding Window	Rolling count	No boundary burst	More memory
Token Bucket	Tokens refilled at rate	Allows controlled bursts	Slightly complex
Leaky Bucket	Queue drains at fixed rate	Smooth output rate	No burst handling

Distributed rate limiting: Use Redis for shared counter across multiple instances.

```
// Redis sliding window rate limiter
@Service
public class RateLimiterService {

    private final StringRedisTemplate redis;

    // Sliding window: count requests in last N seconds
    public boolean allowRequest(String clientId, int limit, Duration window) {
        String key = "ratelimit:" + clientId;
        long now = System.currentTimeMillis();
        long windowStart = now - window.toMillis();

        // Lua script for atomicity (atomic check-and-increment)
        String luaScript = """
            local key = KEYS[1]
            local now = tonumber(ARGV[1])
            local window = tonumber(ARGV[2])
            local limit = tonumber(ARGV[3])
            local windowStart = now - window

            redis.call('ZREMRANGEBYSCORE', key, '-inf', windowStart)
            local count = redis.call('ZCARD', key)

            if count < limit then
                redis.call('ZADD', key, now, now .. '-' .. math.random())
                redis.call('EXPIRE', key, math.ceil(window / 1000))
                return 1
            end
            return 0
            """;

        Long allowed = redis.execute(
            new DefaultRedisScript<>(luaScript, Long.class),
            List.of(key),
```

```
            String.valueOf(now),
            String.valueOf(window.toMillis()),
            String.valueOf(limit));

        return Long.valueOf(1L).equals(allowed);
    }
}

// Token bucket (allows bursts)
@Service
public class TokenBucketRateLimiter {

    private final StringRedisTemplate redis;

    public boolean tryAcquire(String clientId, int tokensPerSecond, int maxBurst) {
        String key = "bucket:" + clientId;
        long now = System.currentTimeMillis();

        String script = """
            local key = KEYS[1]
            local now = tonumber(ARGV[1])
            local rate = tonumber(ARGV[2])      -- tokens per ms
            local capacity = tonumber(ARGV[3])

            local data = redis.call('HMGET', key, 'tokens', 'last_refill')
            local tokens = tonumber(data[1]) or capacity
            local lastRefill = tonumber(data[2]) or now

            -- Refill tokens based on elapsed time
            local elapsed = now - lastRefill
            local newTokens = math.min(capacity, tokens + elapsed * rate / 1000)

            if newTokens >= 1 then
                redis.call('HMSET', key, 'tokens', newTokens - 1, 'last_refill', now)
                redis.call('EXPIRE', key, 60)
                return 1
            end
            return 0
            """;

        Long allowed = redis.execute(
            new DefaultRedisScript<>(script, Long.class),
            List.of(key), String.valueOf(now),
            String.valueOf(tokensPerSecond),
            String.valueOf(maxBurst));

        return Long.valueOf(1L).equals(allowed);
    }
}

// Spring Boot filter for rate limiting
@Component
@Order(2)
```

```java
public class RateLimitFilter implements Filter {

    @Override
    public void doFilter(ServletRequest req, ServletResponse res, FilterChain chain)
            throws IOException, ServletException {
        HttpServletRequest httpReq = (HttpServletRequest) req;
        HttpServletResponse httpRes = (HttpServletResponse) res;

        String clientId = extractClientId(httpReq);
        int limit = getLimit(httpReq.getRequestURI()); // Different limits per endpoint

        if (!rateLimiter.allowRequest(clientId, limit, Duration.ofSeconds(1))) {
            httpRes.setStatus(429);  // Too Many Requests
            httpRes.setHeader("Retry-After", "1");
            httpRes.setHeader("X-RateLimit-Limit", String.valueOf(limit));
            httpRes.setHeader("X-RateLimit-Remaining", "0");
            httpRes.getWriter().write("{\"error\":\"Rate limit exceeded\"}");
            return;
        }
        chain.doFilter(req, res);
    }
}
```

■ 88. CHECKSUM

■ Q: What is a Checksum and when do you use it?

A: A Checksum is a value derived from a block of data used to detect errors or verify data integrity.

Algorithms:

Algorithm	Speed	Collision resistance	Use Case
CRC32	Very fast	Weak	Network error detection, file checksums
MD5	Fast	Broken for security	File integrity (non-security), ETag
SHA-1	Fast	Weak for security	Git commit IDs
SHA-256	Moderate	Strong	Digital signatures, TLS, Bitcoin
xxHash	Extremely fast	Moderate	High-throughput data integrity
MurmurHash3	Extremely fast	Moderate	Hash tables, Bloom filters

```java
// Data integrity verification service
@Service
public class ChecksumService {

    // Fast checksums for data integrity (not cryptographic)
    public String computeCRC32(byte[] data) {
        CRC32 crc = new CRC32();
        crc.update(data);
        return Long.toHexString(crc.getValue());
    }

    // Strong checksum for security/deduplication
    public String computeSHA256(byte[] data) throws NoSuchAlgorithmException {
        MessageDigest md = MessageDigest.getInstance("SHA-256");
        return HexFormat.of().formatHex(md.digest(data));
    }

    // File upload integrity verification
    @PostMapping("/upload")
    public ResponseEntity<String> uploadFile(
            @RequestParam MultipartFile file,
            @RequestHeader("Content-MD5") String expectedChecksum) throws Exception {

        byte[] data = file.getBytes();
        String actualChecksum = Base64.getEncoder().encodeToString(
            MessageDigest.getInstance("MD5").digest(data));

        if (!actualChecksum.equals(expectedChecksum)) {
            return ResponseEntity.badRequest()
                .body("Checksum mismatch: file corrupted in transit");
        }
        storageService.store(file.getOriginalFilename(), data);
        return ResponseEntity.ok("Uploaded successfully");
```

```
    }

    // ETag generation for HTTP conditional requests
    public String generateETag(Object resource) {
        try {
            String json = objectMapper.writeValueAsString(resource);
            return '"' + computeSHA256(json.getBytes()) + '"';
        } catch (Exception e) { throw new RuntimeException(e); }
    }
}
```

■ 89. ENCRYPTION

■ Q: What are the encryption types and when to use each?

A:

- **Symmetric:** Same key for encrypt/decrypt (AES-256). Fast. Problem: key distribution.
- **Asymmetric:** Public key encrypts, private key decrypts (RSA, EC). Slower. Solves key distribution.
- **Hybrid:** Asymmetric to exchange symmetric key, symmetric for data. Used by TLS.
- **Hashing:** One-way, not reversible. For passwords (bcrypt, argon2), integrity checks.

Encryption at Rest vs In Transit:

- In Transit: TLS 1.3 (HTTPS, gRPC)
- At Rest: AES-256 for stored data, database encryption, disk encryption

```java
// Encrypting sensitive data at rest
@Service
public class EncryptionService {

    @Value("${encryption.key}")  // 256-bit key from Vault
    private String base64Key;

    public String encrypt(String plaintext) throws Exception {
        byte[] key = Base64.getDecoder().decode(base64Key);
        SecretKeySpec secretKey = new SecretKeySpec(key, "AES");

        Cipher cipher = Cipher.getInstance("AES/GCM/NoPadding");
        byte[] iv = new byte[12]; // 96-bit IV for GCM
        SecureRandom.getInstanceStrong().nextBytes(iv);
        cipher.init(Cipher.ENCRYPT_MODE, secretKey, new GCMParameterSpec(128, iv));

        byte[] encrypted = cipher.doFinal(plaintext.getBytes(StandardCharsets.UTF_8));

        // Prepend IV to ciphertext for storage (IV is not secret)
        byte[] combined = new byte[iv.length + encrypted.length];
        System.arraycopy(iv, 0, combined, 0, iv.length);
        System.arraycopy(encrypted, 0, combined, iv.length, encrypted.length);
        return Base64.getEncoder().encodeToString(combined);
    }

    public String decrypt(String ciphertext) throws Exception {
        byte[] combined = Base64.getDecoder().decode(ciphertext);
        byte[] key = Base64.getDecoder().decode(base64Key);

        byte[] iv = Arrays.copyOfRange(combined, 0, 12);
        byte[] encrypted = Arrays.copyOfRange(combined, 12, combined.length);

        SecretKeySpec secretKey = new SecretKeySpec(key, "AES");
        Cipher cipher = Cipher.getInstance("AES/GCM/NoPadding");
        cipher.init(Cipher.DECRYPT_MODE, secretKey, new GCMParameterSpec(128, iv));

        return new String(cipher.doFinal(encrypted), StandardCharsets.UTF_8);
    }
}
```

```java

// Password hashing (NEVER encrypt passwords — only hash)
@Service
public class PasswordService {

    private final PasswordEncoder encoder = new BCryptPasswordEncoder(12);
    // Or use Argon2: new Argon2PasswordEncoder(16, 32, 1, 65536, 3);

    public String hashPassword(String rawPassword) {
        return encoder.encode(rawPassword);
    }

    public boolean verifyPassword(String rawPassword, String hashedPassword) {
        return encoder.matches(rawPassword, hashedPassword);
    }
}
```

■ 90. MONOLITH

■ Q: When is a Monolith better than Microservices?

A: A monolith packages all functionality in a single deployable unit. Often the right starting architecture.

Monolith Advantages:

- Simple development (one codebase, one repo)
- Simple deployment (one artifact)
- Simple testing (no distributed test complexity)
- No network latency for internal calls
- Strong consistency (one DB transaction)
- Low operational complexity

When Monolith is correct: Early stage startup, small team (<10 engineers), domain boundaries unclear, speed to market critical, operational simplicity required.

Modular Monolith: Best of both — monolith deployment with clear module boundaries. Easy to split into microservices later. Recommended first architecture.

```
// Well-structured modular monolith
// Each module has clear boundaries and interfaces

// Order Module
@Module(name = "order", dependencies = {"inventory", "payment"})
public class OrderModule {
    // Only exposes interface to other modules
    public interface OrderService {
        Order createOrder(CreateOrderRequest request);
        Optional<Order> findById(Long id);
    }
}

// Inventory Module — no direct DB access from order module
@Module(name = "inventory")
public class InventoryModule {
    public interface InventoryService {
        boolean reserve(List<OrderItem> items);
        void release(List<OrderItem> items);
    }
}

// Internal event bus — same semantics as Kafka but in-process
@Configuration
public class ModularMonolithConfig {

    @Bean
    public ApplicationEventPublisher eventPublisher() {
        return new SimpleApplicationEventPublisher();
    }
}

@Service
```

```java
public class OrderServiceImpl implements OrderModule.OrderService {

    @Autowired private InventoryModule.InventoryService inventoryService;
    @Autowired private PaymentModule.PaymentService paymentService;
    @Autowired private ApplicationEventPublisher events;

    @Override
    @Transactional  // Single DB transaction — strong consistency
    public Order createOrder(CreateOrderRequest request) {
        boolean reserved = inventoryService.reserve(request.getItems());
        if (!reserved) throw new InsufficientStockException();

        Order order = orderRepo.save(Order.from(request));
        events.publishEvent(new OrderCreatedEvent(order)); // In-process event
        return order;
    }
}
```

■ 91. CIRCUIT BREAKER

■ Q: How does the Circuit Breaker pattern work in detail?

A: Circuit Breaker monitors calls to external services. When failure rate exceeds threshold, it "opens" and fails fast (no actual call made), giving the failing service time to recover.

States:

- **CLOSED:** Normal operation. Failures tracked.
- **OPEN:** Failures exceeded threshold. All calls fail immediately (fast fail). After `waitDuration`, transitions to HALF_OPEN.
- **HALF_OPEN:** Allow limited test calls. If successful → CLOSED. If fail → OPEN.

Why it prevents cascade failure: Without CB, thread pool fills waiting for slow external service → entire application becomes unresponsive for all operations.

```
// Resilience4j Circuit Breaker detailed configuration
@Configuration
public class CircuitBreakerConfig {

    @Bean
    public CircuitBreakerRegistry circuitBreakerRegistry() {
        io.github.resilience4j.circuitbreaker.CircuitBreakerConfig config =
            io.github.resilience4j.circuitbreaker.CircuitBreakerConfig.custom()
                .slidingWindowType(COUNT_BASED)
                .slidingWindowSize(20)              // Track last 20 calls
                .minimumNumberOfCalls(10)           // Need 10 calls before evaluating
                .failureRateThreshold(50)           // Open if >50% failure rate
                .slowCallRateThreshold(80)          // Also open if >80% calls are slow
                .slowCallDurationThreshold(Duration.ofSeconds(3))
                .waitDurationInOpenState(Duration.ofSeconds(30))
                .permittedNumberOfCallsInHalfOpenState(5)
                .automaticTransitionFromOpenToHalfOpenEnabled(true)
                .recordExceptions(IOException.class, TimeoutException.class)
                .ignoreExceptions(BusinessException.class)  // Don't count business errors
                .build();

        return CircuitBreakerRegistry.of(config);
    }
}

@Service
public class ResilientPaymentService {

    @Autowired private CircuitBreakerRegistry registry;

    public PaymentResult processPayment(PaymentRequest request) {
        CircuitBreaker cb = registry.circuitBreaker("payment-gateway");

        // Add event listeners for monitoring
        cb.getEventPublisher()
            .onStateTransition(e -> {
                log.warn("CB state transition: {} → {}",
                    e.getStateTransition().getFromState(),
```

```
                    e.getStateTransition().getToState());
                alertService.notify("Circuit breaker state change: " + e.getStateTransition());
            })
            .onCallNotPermitted(e -> metrics.counter("cb.rejected").increment());

        return CircuitBreaker.decorateSupplier(cb,
                () -> paymentGateway.charge(request))
            .andThen(result -> { /* success */ return result; })
            .recover(CallNotPermittedException.class,
                ex -> PaymentResult.rejected("Circuit open")) // Fast fail fallback
            .recover(Exception.class,
                ex -> PaymentResult.failed(ex.getMessage()))
            .get();
    }
}
```

■ 92. AUTOSCALING

■ Q: What are the autoscaling types and how do you configure them?

A: Autoscaling automatically adjusts the number of running instances based on load, maintaining performance while minimizing cost.

Types:

- **Horizontal Pod Autoscaler (HPA):** Scale number of replicas based on CPU/memory/custom metrics.
- **Vertical Pod Autoscaler (VPA):** Adjust CPU/memory requests of existing pods.
- **KEDA (Kubernetes Event-Driven Autoscaling):** Scale based on Kafka lag, queue depth, custom metrics.
- **Cluster Autoscaler:** Add/remove nodes from Kubernetes cluster.

```
# HPA based on CPU + custom metric
apiVersion: autoscaling/v2kind: HorizontalPodAutoscalermetadata:  name: order-service-hpaspec:  scaleTargetRef
    resource:      name: cpu      target:        type: Utilization        averageUtilization: 70  - type: External
    external:      metric:        name: kafka_consumer_lag        selector:          matchLabels:
        value: 4           # Add max 4 pods at once        periodSeconds: 60    scaleDown:      stabilizationWindowSec
        value: 10          # Remove max 10% per minute        periodSeconds: 60
```

```
// Spring Boot: expose custom metrics for HPA
@Component
public class OrderQueueDepthMetric {

    @Autowired private MeterRegistry registry;
    @Autowired private OrderRepository orderRepo;

    @PostConstruct
    public void registerMetric() {
        // Expose queue depth metric for HPA
        Gauge.builder("order.processing.queue.depth",
            orderRepo, repo -> repo.countByStatus(OrderStatus.PENDING))
            .description("Orders waiting to be processed")
            .register(registry);
    }
}
```

■ 93. TRACING

■ Q: What is distributed tracing and how do you implement it?

A: Distributed tracing tracks a request as it flows through multiple services, showing the complete timeline and where time was spent.

Concepts:

- **Trace:** Complete end-to-end journey of a request (has a Trace ID)
- **Span:** Unit of work within a service (has Span ID, parent Span ID, start time, duration, tags)
- **Context propagation:** Trace ID passed via HTTP headers (`traceparent`, `b3-trace-id`)

Tools: Jaeger, Zipkin, AWS X-Ray, Datadog APM, Grafana Tempo.

```
// Spring Boot 3 with OpenTelemetry (auto-instrumented)
// application.yml:
// management.tracing.sampling.probability=0.1  # 10% sampling in prod
// management.otlp.tracing.endpoint=http://jaeger:4318/v1/traces

// Custom spans for business operations
@Service
public class OrderFulfillmentService {

    @Autowired private Tracer tracer;
    @NewSpan("order.fulfillment")  // Creates new span automatically
    public FulfillmentResult fulfill(
            @SpanTag("order.id") String orderId) {
        // This method creates a span with the orderId tag automatically
        return doFulfill(orderId);
    }

    // Manual span creation for fine-grained tracing
    public FulfillmentResult fulfillManual(String orderId) {
        Span span = tracer.nextSpan()
            .name("order.fulfillment")
            .tag("order.id", orderId)
            .start();

        try (Tracer.SpanInScope ws = tracer.withSpan(span)) {
            // Child spans for each step
            Span inventorySpan = tracer.nextSpan().name("inventory.reserve").start();
            try (Tracer.SpanInScope s = tracer.withSpan(inventorySpan)) {
                inventoryService.reserve(orderId);
            } finally { inventorySpan.end(); }

            Span paymentSpan = tracer.nextSpan().name("payment.charge").start();
            try (Tracer.SpanInScope s = tracer.withSpan(paymentSpan)) {
                paymentService.charge(orderId);
            } finally { paymentSpan.end(); }

            span.tag("fulfillment.status", "completed");
            return FulfillmentResult.success();
        } catch (Exception e) {
            span.error(e);
```

```
            throw e;
        } finally {
            span.end();
        }
    }
}
```

■ 94. MATERIALIZED VIEW

■ Q: What is a Materialized View and when should you use it?

A: A Materialized View stores the result of a query physically on disk (unlike regular views which re-execute on every access). Dramatically speeds up expensive aggregate queries at the cost of staleness.

When to use: Expensive GROUP BY / JOIN queries run frequently; dashboard queries over large tables; reporting that can tolerate slight staleness.

When NOT to use: Data that must be real-time fresh; write-heavy tables (refresh overhead); small datasets.

```
// Flyway migration: Materialized View in PostgreSQL
/*
CREATE MATERIALIZED VIEW customer_revenue_mv AS
SELECT
    c.id as customer_id,
    c.name,
    c.email,
    COUNT(o.id) as total_orders,
    SUM(o.amount) as total_revenue,
    AVG(o.amount) as avg_order_value,
    MAX(o.created_at) as last_order_date,
    MIN(o.created_at) as first_order_date
FROM customers c
LEFT JOIN orders o ON o.customer_id = c.id AND o.status = 'COMPLETED'
GROUP BY c.id, c.name, c.email;

-- Index for fast lookups
CREATE UNIQUE INDEX ON customer_revenue_mv (customer_id);

-- Concurrent refresh: doesn't lock read queries
REFRESH MATERIALIZED VIEW CONCURRENTLY customer_revenue_mv;
*/

// Service with scheduled refresh
@Service
public class MaterializedViewRefreshService {

    @Scheduled(fixedDelay = 300_000)  // Refresh every 5 minutes
    @Transactional
    public void refreshRevenueView() {
        jdbcTemplate.execute(
            "REFRESH MATERIALIZED VIEW CONCURRENTLY customer_revenue_mv");
        log.info("Materialized view refreshed");
    }

    // On-demand refresh when critical update occurs
    @EventListener
    @Async
    public void onMajorOrderEvent(HighValueOrderEvent event) {
        if (event.getAmount().compareTo(new BigDecimal("10000")) > 0) {
            refreshRevenueView();  // Refresh immediately for high-value orders
        }
```

```
    }
}

@Repository
public interface CustomerRevenueRepository extends JpaRepository<CustomerRevenueMv, String> {
    // Reads from materialized view — millisecond response for precomputed aggregates
    @Query("SELECT m FROM CustomerRevenueMv m ORDER BY m.totalRevenue DESC")
    List<CustomerRevenueMv> getTopCustomersByRevenue(Pageable pageable);
}
```

■ 95. SERIALIZATION

■ Q: What serialization formats should you choose and why?

A: Serialization converts objects to bytes for storage or network transfer. Deserialization reverses it.

Formats Comparison:

Format	Size	Speed	Schema	Human-readable	Use Case
JSON	Large	Slow	No	Yes	REST APIs, config
XML	Largest	Slowest	XSD	Yes	Legacy SOAP, enterprise
Protobuf	Small (~3x JSON)	Very Fast	Yes (.proto)	No	gRPC, internal services
Avro	Small	Fast	Yes	No	Kafka, big data
MessagePack	Medium	Fast	No	No	Binary JSON
Java Native	Large	Slow	No	No	Never use for APIs

Schema evolution: Protobuf/Avro handle field additions/removals gracefully (backward/forward compatible).

```
// Protobuf serialization
// order.proto: message OrderEvent { string order_id = 1; double amount = 2; }

@Service
public class OrderEventSerializer {

    // Protobuf: fast, compact, schema-validated
    public byte[] serializeProtobuf(OrderEventProto.OrderEvent event) {
        return event.toByteArray();
    }

    public OrderEventProto.OrderEvent deserializeProtobuf(byte[] bytes) throws Exception {
        return OrderEventProto.OrderEvent.parseFrom(bytes);
    }
}

// Kafka with Avro Schema Registry
@Configuration
public class AvroKafkaConfig {

    @Bean
    public ProducerFactory<String, GenericRecord> avroProducerFactory() {
        return new DefaultKafkaProducerFactory<>(Map.of(
            ProducerConfig.BOOTSTRAP_SERVERS_CONFIG, "kafka:9092",
            "schema.registry.url", "http://schema-registry:8081",
            ProducerConfig.KEY_SERIALIZER_CLASS_CONFIG, StringSerializer.class,
            ProducerConfig.VALUE_SERIALIZER_CLASS_CONFIG, KafkaAvroSerializer.class
        ));
    }
}

```

```java
// JSON with field filtering for API responses
@JsonView(Views.Public.class)
public class Order {
    @JsonView(Views.Public.class)
    private Long id;

    @JsonView(Views.Public.class)
    private String status;

    @JsonView(Views.Internal.class)  // Not exposed externally
    private String internalNotes;

    @JsonView(Views.Admin.class)
    private BigDecimal costPrice;
}

// Controller returns only public fields to external clients
@GetMapping("/{id}")
@JsonView(Views.Public.class)
public Order getOrder(@PathVariable Long id) {
    return orderService.findById(id);
}
```

■ 96. SIDECAR PATTERN

■ Q: What is the Sidecar pattern and what belongs in a sidecar?

A: The Sidecar pattern attaches a helper container to the main application container in the same pod. They share network namespace and can share filesystem.

What belongs in a Sidecar:

- Service mesh proxy (Envoy/Linkerd) — traffic management, mTLS
- Log collection agent (Fluentd, Filebeat)
- Metrics collector (Prometheus Node Exporter)
- Config sync (Vault Agent, AWS Secrets Manager Agent)
- Service discovery registration

Benefits: Application code stays clean; cross-cutting concerns in sidecar; different update lifecycle.

Drawbacks: Additional resource overhead; complexity; debugging across containers.

```
# Kubernetes: Application + Sidecar containers
apiVersion: apps/v1kind: Deploymentspec:  template:    spec:      containers:      # Main application
      - name: order-service
        image: order-service:1.0.0        ports:        - containerPort: 8080
        volumeMounts:        - name: logs
          mountPath: /var/log/app        - name: secrets
          mountPath: /vault/secrets          readOnly: true
      # Sidecar 1: Log collector
      - name: fluentd
        image: fluentd:v1.14        volumeMounts:        - name: logs
          mountPath: /var/log/app          readOnly: true        env:        - name: ELASTICSEARCH_HOST
          value: elasticsearch:9200
      # Sidecar 2: Vault Agent (secret injection without app code changes)
      - name: vault-agent
        image: hashicorp/vault:1.15        args: ["agent", "-config=/vault/config/agent.hcl"]        volumeMounts
          mountPath: /vault/config        - name: secrets
          mountPath: /vault/secrets
      volumes:      - name: logs
        emptyDir: {}      - name: secrets
        emptyDir:          medium: Memory  # Secrets in tmpfs — never written to disk
```

■ 97. HYPERLOGLOG

■ Q: What is HyperLogLog and what problem does it solve?

A: HyperLogLog (HLL) is a probabilistic algorithm for estimating the cardinality (count of distinct elements) of a very large dataset using very little memory.

The Problem: COUNT(DISTINCT user_id) for 1 billion records requires storing all 1 billion IDs to count exactly.

HLL Solution: ~1.5KB of memory to estimate 1 billion distinct values with ~0.81% error. Incredibly space-efficient.

Use cases:

- Unique visitor counting (DAU, MAU)
- Distinct event counting
- A/B test reach estimation
- Network: unique IP counting

```
// Redis HyperLogLog for unique visitor counting
@Service
public class UniqueVisitorService {

    private final StringRedisTemplate redis;

    public void trackVisit(String userId, String date) {
        String key = "uv:" + date;
        redis.opsForHyperLogLog().add(key, userId);
        // Memory: constant ~12KB regardless of billions of users
    }

    public long getUniqueVisitors(String date) {
        // Approximate count with ~0.81% error
        return redis.opsForHyperLogLog().size("uv:" + date);
    }

    // Merge multiple days (union of unique visitors)
    public long getUniqueVisitorRange(String startDate, String endDate) {
        String mergeKey = "uv:merged:" + startDate + ":" + endDate;
        List<String> keys = getDatesInRange(startDate, endDate).stream()
            .map(d -> "uv:" + d).collect(Collectors.toList());

        redis.opsForHyperLogLog().union(mergeKey, keys.toArray(String[]::new));
        long result = redis.opsForHyperLogLog().size(mergeKey);
        redis.delete(mergeKey);
        return result;
    }

    // Custom HyperLogLog for embedded use (no Redis needed)
    public long estimateCardinality(Stream<String> elements) {
        // Using stream-lib HyperLogLog
        HyperLogLog hll = new HyperLogLog(0.01); // 1% error
        elements.forEach(hll::offer);
        return hll.cardinality();
    }
}
```

■ 98. DATA WAREHOUSE

■ Q: What is a Data Warehouse and how do you design one?

A: A Data Warehouse is a centralized repository for structured, integrated data from multiple sources, optimized for complex analytical queries (OLAP — Online Analytical Processing).

OLTP vs OLAP:

OLTP (Operational DB)	OLAP (Data Warehouse)	
Purpose	Transactions	Analytics
Queries	Simple, fast, indexed	Complex, slow, aggregations
Data	Current	Historical
Schema	Normalized (3NF)	Denormalized (Star/Snowflake)
Examples	PostgreSQL, MySQL	Snowflake, BigQuery, Redshift
Record count	Thousands per query	Billions per query

Star Schema: Fact table (events/transactions) surrounded by Dimension tables (who, what, where, when).

```
-- Star Schema for Order Analytics Data Warehouse
-- FACT TABLE: Order facts (events/measures)
CREATE TABLE fact_orders (
    order_key        BIGINT,           -- Surrogate key
    customer_key     INT,              -- FK to dim_customer
    product_key      INT,              -- FK to dim_product
    date_key         INT,              -- FK to dim_date
    geography_key    INT,              -- FK to dim_geography
    order_amount     DECIMAL(19,4),
    quantity         INT,
    discount_pct     DECIMAL(5,2),
    profit_margin    DECIMAL(19,4)
);

-- DIMENSION TABLES: Context
CREATE TABLE dim_customer (
    customer_key    INT PRIMARY KEY,   -- Surrogate key
    customer_id     VARCHAR,           -- Business key
    customer_name   VARCHAR,
    segment         VARCHAR,           -- Consumer, Corporate, Home Office
    city            VARCHAR,
    country         VARCHAR
);

CREATE TABLE dim_date (
    date_key        INT PRIMARY KEY,
    full_date       DATE,
    day_of_week     VARCHAR,
    month           INT,
    quarter         INT,
    year            INT,
```

```
    is_weekend      BOOLEAN,
    is_holiday      BOOLEAN
);
```

```
// Spring Batch: populate data warehouse from OLTP DB (nightly ETL)
@Configuration
@EnableBatchProcessing
public class DwETLJob {

    @Bean
    public Job orderDwJob(JobBuilderFactory jbf, StepBuilderFactory sbf) {
        return jbf.get("orderDwJob")
            .start(extractAndTransformOrders(sbf))
            .next(loadToWarehouse(sbf))
            .next(refreshAggregations(sbf))
            .build();
    }

    @Bean
    public Step extractAndTransformOrders(StepBuilderFactory sbf) {
        return sbf.get("extract")
            .<Order, FactOrder>chunk(5000)
            .reader(new JpaPagingItemReaderBuilder<Order>()
                .name("orderReader")
                .queryString("SELECT o FROM Order o WHERE o.updatedAt > :lastRun")
                .build())
            .processor(order -> {
                // Transform OLTP record to warehouse fact record
                return FactOrder.builder()
                    .orderKey(order.getId())
                    .customerKey(lookupCustomerKey(order.getCustomerId()))
                    .productKey(lookupProductKey(order.getProductId()))
                    .dateKey(toDateKey(order.getCreatedAt()))
                    .orderAmount(order.getAmount())
                    .quantity(order.getQuantity())
                    .profitMargin(calculateMargin(order))
                    .build();
            })
            .writer(warehouseJdbcWriter())
            .build();
    }

    // Refresh OLAP aggregations for fast dashboard queries
    @Bean
    public Step refreshAggregations(StepBuilderFactory sbf) {
        return sbf.get("refreshAggregations")
            .<String, String>chunk(1)
            .reader(new ListItemReader<>(List.of(
                "REFRESH MATERIALIZED VIEW monthly_revenue_by_segment",
                "REFRESH MATERIALIZED VIEW product_performance_ytd",
                "REFRESH MATERIALIZED VIEW customer_cohort_analysis")))
            .writer(items -> items.forEach(jdbcTemplate::execute))
```

```
            .build();
    }
}
```

SYSTEM DESIGN — HIGH LEVEL REFERENCE

■ Quick Selection Guide

Requirement	Choose
Strong consistency + ACID transactions	PostgreSQL, CockroachDB
High write throughput, time-series	Cassandra, InfluxDB
Full-text search	Elasticsearch
Caching, sessions	Redis
Event streaming, audit log	Kafka
Task queue, complex routing	RabbitMQ
Internal service comms	gRPC + Protobuf
Public APIs	REST + JSON
Mobile/web clients with complex data	GraphQL
Service mesh, mTLS	Istio/Linkerd
Real-time bidirectional	WebSockets + STOMP
One-way server push	SSE
Semantic search, RAG	pgvector, Pinecone
Cardinality estimation	HyperLogLog (Redis)
Set membership	Bloom Filter
Batch analytics	Spark, Spring Batch
Stream analytics	Kafka Streams, Flink
Data warehouse	Snowflake, BigQuery
Feature flags	Unleash, LaunchDarkly
Secrets management	HashiCorp Vault
Distributed tracing	OpenTelemetry + Jaeger
Metrics	Prometheus + Grafana
Logs	ELK Stack / Grafana Loki

■ System Design Framework (for interviews)

1. **Clarify requirements:** Functional + Non-functional (scale, latency, consistency)
2. **Estimate scale:** DAU, QPS, storage, bandwidth
3. **Define API:** Key endpoints/events
4. **HLD diagram:** Components + data flow
5. **Data model:** Schema design
6. **Deep dive:** Most critical component
7. **Bottlenecks + mitigations:** Where will it fail?
8. **Operational concerns:** Monitoring, deployment, failure modes